Rockchip DRM Display Driver 开发指南

文件标识：RK-YH-YF-455

发布版本：V5.9.0

日期：2025-09-28

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

本文主要介绍 Rockchip 平台处理器基于 DRM 显示框架 VOP 以及相关显示接口的基本特性、工作流程和常见问题分析。目的是为了相关工程师能对 DRM 显示驱动框架和硬件接口有更好的理解，并通过常见问题的分析能快速定位问题、解决问题。

产品版本

芯片名称	内核版本
RK3036	Linux kernel 4.4 及以上内核
RK312X/PX3SE	Linux kernel 4.4 及以上内核
RK3288	Linux kernel 4.4 及以上内核
RK322X/RK312XH	Linux kernel 4.4 及以上内核
RK3308	Linux kernel 4.4 及以上内核
RK322XH/RK332X	Linux kernel 4.4 及以上内核
RK3326/PX30	Linux kernel 4.4 及以上内核
RK3368/PX5	Linux kernel 4.4 及以上内核
RK3399	Linux kernel 4.4 及以上内核
RK1808	Linux kernel 4.4 及以上内核
RV1109/RV1126	Linux kernel 4.19 及以上内核
RK356X	Linux kernel 4.19 及以上内核
RK3588	Linux kernel 5.10 及以上内核
RV1103/RV1106	Linux kernel 5.10 及以上内核
RK3528	Linux kernel 4.19 及以上内核
RK3562	Linux kernel 5.10 及以上内核
RK3576	Linux kernel 6.1 及以上内核
RK3506	Linux kernel 6.1 及以上内核
RV1126B	Linux kernel 6.1 及以上内核

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

硬件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	黄家钗	2019-1-5	初始版本
V2.0.0	黄家钗	2019-11-15	针对 Linux 4.19 更新
V3.0.0	黄家钗	2022-03-17	针对 RK356X/RK3588/RV1103/RV1106 以及 Linux kernel 5.10 更新
V3.1.0	闫孝军	2022-04-24	加入更多内容
V3.1.2	闫孝军	2022-05-09	修正一些描述错误
V3.1.3	闫孝军	2022-05-30	补充相关引用参考文档
V3.2.0	黄家钗	2022-06-12	更新 VOP 和接口 feature
V3.3.0	张玉炳	2022-06-23	添加 RK3588 VOP DCLK 分配策略
V3.4.0	黄家钗	2022-06-29	加入 RK3566 图层分配策略说明
V3.4.1	闫孝军	2022-07-06	修正 connector-split 相关描述
V3.5.0	闫孝军	2022-07-08	加入一些定位问题的方法和建议
V3.5.1	张玉炳	2022-07-13	修正 dclk 相关描述
V3.6.0	黄家钗	2023-05-29	添加 RK3528 和 RK3562 支持并更新 RK3588 DSC 支持情况说明
V3.7.0	闫孝军	2023-09-14	更新 connector-mirror 软件支持版本情况
V3.8.0	丁凌崧	2023-09-28	添加 RK3308 和 RV1106 开启显示功能的配置说明
V3.9.0	黄家钗	2023-10-20	添加获取 VOP 显示状态信息的方法
V4.0.0	黄家钗	2024-04-23	添加 RK3576 以及 Linux kernel 6.1 支持
V4.1.0	黄家钗	2024-06-21	更新 connector split 模式说明
V4.2.0	闫孝军	2024-07-17	添加关于如何准确打开 DRM 显示设备的描述
V4.3.0	丁凌崧	2024-08-27	添加 x-mirror 功能说明和 4K logo 显示相关 FAQ
V4.4.0	丁凌崧	2024-08-28	添加 RK3506 支持
V4.5.0	丁凌崧	2024-09-10	添加 RV1106/RV1103 小分辨率显示异常相关 FAQ
V4.6.0	黄家钗	2024-11-07	添加 RK3399 VOP DCLK 分配策略
V4.7.0	黄家钗	2024-11-11	添加色域相关配置说明
V4.8.0	闫孝军	2024-12-09	修改 drm-cursor 库描述的位置
V4.9.0	黄家钗	2025-01-07	增加 logo 配置说明
V5.0.0	闫孝军	2025-02-11	修正一些时钟分配相关的描述
v5.1.0	陈潮毅	2025-02-13	添加 RK3528/RK3562/RK3568/RK3576 VOP DCLK 分配策略
v5.2.0	陈潮毅	2025-02-14	添加 RK3528/RK3562 VP 和各显示接口连接关系说明
v5.3.0	闫孝军	2025-02-25	添加关于禁止 RK3588/RK3568 图层迁移的描述。
v5.4.0	陈潮毅	2025-02-26	添加 RK3566/RK3576s/RK3588s 显示规格的说明
v5.5.0	闫孝军	2025-02-28	强调 RK3568/RK3588 的 Cluster 图层不支持非 AFBC 格式
v5.5.1	黄家钗	2025-03-06	调整 VOP2 平台显示通路章节按数字增序排列
v5.6.0	黄家钗	2025-03-12	增加 VP 自测模式和显示接口文档说明

版本号	作者	修改日期	修改说明
v5.7.0	丁凌崧	2025-04-07	添加 U-Boot Video Console 模式配置说明
v5.8.0	张玉炳	2025-09-22	补充 RK3576/RK3588 DCIK 分配策略说明
v5.9.0	陈潮毅	2025-09-28	添加 RV1126B 支持

目录

Rockchip DRM Display Driver 开发指南

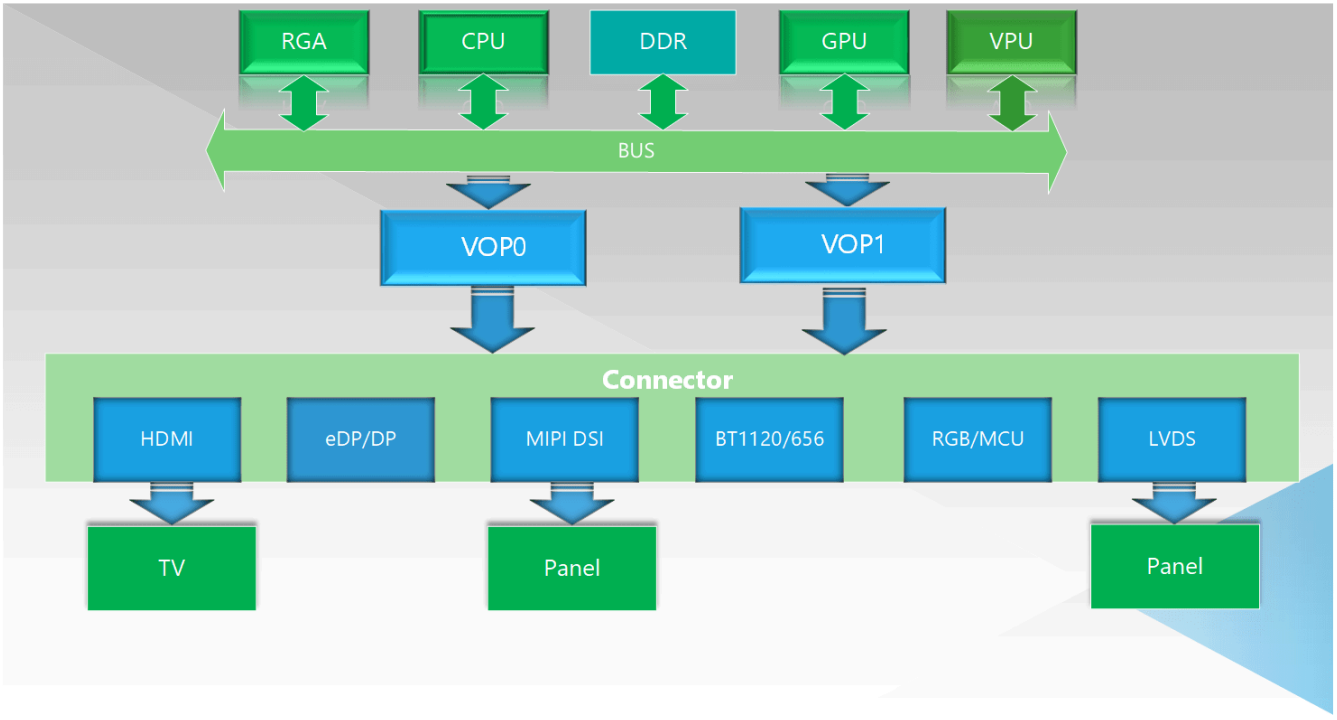
1. Rockchip 平台显示子系统（DSS） 概述
2. DRM 概述
   - 2.1 基本概念
   - 2.2 显示通路
   - 2.3 DRM 驱动和 libdrm 的交互过程
3. 软件驱动
   - 3.1 U-Boot 驱动
     - 3.1.1 驱动目录
     - 3.1.2 驱动文件
     - 3.1.3 接口说明
     - 3.1.4 Logo Display 应用说明
     - 3.1.5 Video Console 应用说明
     - 3.1.6 分析 U-Boot logo 过渡到 Linux Kernel logo 过程中出现的闪屏或者无法显示的问题
   - 3.2 kernel 驱动
     - 3.2.1 驱动目录
     - 3.2.2 驱动文件
     - 3.2.3 驱动加载流程
     - 3.2.4 DTS 配置
       - 3.2.4.1 基础配置
       - 3.2.4.2 指定图层分配策略
         - 3.2.4.2.1 RK3566 图层分配策略
       - 3.2.4.3 把某个图层设置为鼠标层
         - 3.2.4.3.1 图层和 VP 之间的的连接关系
       - 3.2.4.4 esmart 图层分割
         - 3.2.4.4.1 RK3528 esmart 图层分割
         - 3.2.4.4.2 RK3576 esmart 图层分割
       - 3.2.4.5 禁止图层迁移
     - 3.2.5 色域相关配置说明
       - 3.2.5.1 Plane 色域配置说明
       - 3.2.5.2 Connector 色域配置说明
       - 3.2.5.3 其他说明
4. Display feature
   - 4.1 各平台 VOP 基础特性
   - 4.2 各平台显示接口最大输出分辨率和协议标准
   - 4.3 VOP2 平台显示通路
     - 4.3.1 RK3528 VP 和各显示接口的连接关系
     - 4.3.2 RK3562 VP 和各显示接口的连接关系
     - 4.3.3 RK3568 VP 和各显示接口的连接关系
     - 4.3.4 RK3576 VP 和各显示接口的连接关系
     - 4.3.5 RK3588 VP 和各显示接口的连接关系
   - 4.4 延伸芯片的显示规格
     - 4.4.1 RK3566
     - 4.4.2 RK3576s
     - 4.4.3 RK3588s
   - 4.5 VOP 平台 DCLK 时钟分配策略
     - 4.5.1 RK3399 VOP DCLK 分配策略
     - 4.5.2 RK3528 VOP DCLK 分配策略
       - 4.5.2.1 RK3528 DCLK 概述
     - 4.5.3 RK3562 VOP DCLK 分配策略
       - 4.5.3.1 RK3562 DCLK 概述
     - 4.5.4 RK3568 VOP DCLK 分配策略
       - 4.5.4.1 RK3568 DCLK 概述
     - 4.5.5 RK3576 VOP DCLK 分配策略
       - 4.5.5.1 RK3576 DCLK 概述
       - 4.5.5.2 RK3576 VOP DCLK 静态分配策略
       - 4.5.5.3 RK3576 VOP DCLK 和 HDMI 接口绑定的动态分配策略
       - 4.5.5.4 RK3576 VOP DCLK 动态分配策略
       - 4.5.5.5 RK3576 VOP DCLK 分配策略选用
     - 4.5.6 RK3588 VOP DCLK 分配策略
       - 4.5.6.1 RK3588 DCLK 概述

4.5.6.2 RK3588 VOP DCLK 静态分配策略
4.5.6.3 RK3588 VOP DCLK 和 HDMI 接口绑定的动态分配策略
4.5.6.4 RK3588 VOP DCLK 动态分配策略
4.5.6.5 RK3588 VOP DCLK 分配策略选用
5. VOP 2.0 架构下的多屏显示
5.1 Connector-mirror
5.2 Connector-split
6. 硬件相关
6.1 RGB 输出/TTL 模式硬件连接
6.1.1 VOP 1.0 RGB 接口硬件连接方式
6.1.2 VOP 2.0 及之后的版本 RGB 接口硬件连接方式
6.1.3 不同 display mode index 对应的软件配置
6.1.4 BT.656 和 BT.1120 的硬件连接方式
6.2 LVDS Data Mapping
7. 扫描时序说明
7.1 常见的扫描时序图
7.2 DRM 对扫描时序的定义
7.3 软件配置的对应关系和转换
7.4 查看当前配置的时序
8. 带宽的计算方法
8.1 图像的带宽
8.2 显示接口的带宽
9. 常用的 debug 手段
9.1 dump 当前的显示状态
9.1.1 使用命令
9.1.2 参数说明
9.1.3 dump VOP 当前的配置信息
9.2 dump 当前显示的 buffer
9.2.1 使用说明
9.3 使用 VP 自测模式显示彩条
9.4 调整 DRM 打印 Log 等级抓 Log
9.5 查看当前显示时钟
9.6 强行开关显示设备
9.7 查看 DRM buffer 使用情况
9.8 查看 GPIO 状态
9.9 modetest 的使用
9.10 xrandr 的使用
9.11 显示进程的暂停、启动
9.12 获取 EDID 信息
9.13 查看 HDMI 状态
10. FAQ
10.1 VOP POST_BUF_EMPTY
10.2 显示效果调节
10.3 屏无法点亮/休眠唤醒显示异常/不显示问题
10.4 RK3308/RV1106/RV1103 如何打开显示功能
10.5 关闭 iommu 的方法
10.6 各种接口应用参考文档
10.7 RGB/MCU 屏帧率计算问题
10.8 如何编写第三方转换芯片驱动
10.9 RK3588 DSC 支持几个 slice，slice width 最大支持多少
10.10 超过 4kP60 对 aclk 的要求
10.11 如何默认开启显示 mirror 功能
10.12 RV1106/RV1103 小分辨率屏无法点亮/显示异常问题
11. 如何更高效的定位问题
12. 参考文档

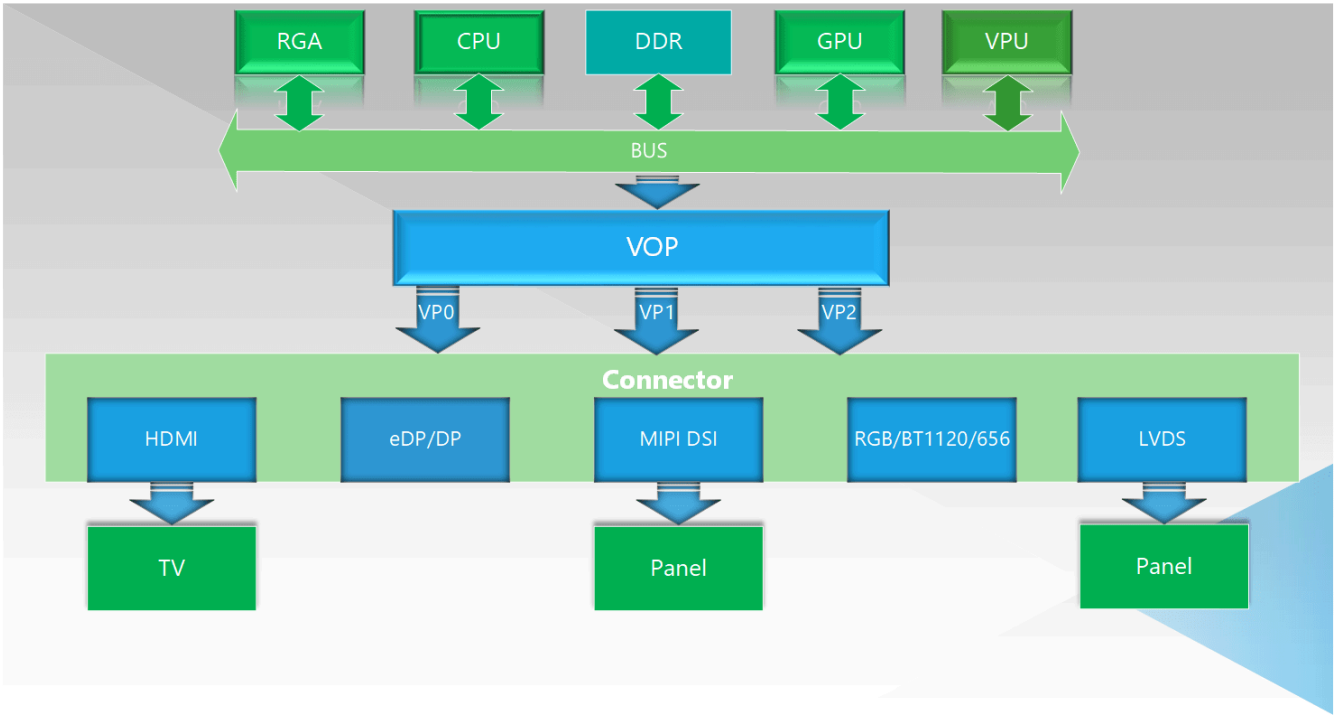
1. Rockchip 平台显示子系统（DSS） 概述

显示子系统是 Rockchip 平台显示输出相关软硬件系统的统称，它包括 VOP（比较老的平台叫 LCDC，比如 RK3188、RK3066）和 RGB、BT1120、BT656、I8080（MCU 显示接口），LVDS、MIPI DSI、EDP、DP、HDMI 等显示信号输出模块以及与之对应的软件驱动。

整个显示系统的硬件框架如下图所示：



VOP 1.0 显示子系统架构



VOP 2.0 显示子系统架构

从上面的 DSS 框图可以看到，在整个显示通路的最后端，是由 RGA，GPU、VPU 组成的显示图形加速模块，他们是专门针对图像处理优化设计的硬件 IP，能够高效的进行图像的生成和进一步处理（比如 GPU 通过 opengl 功能提供图像渲染功能，RGA 可以对图像数据进行缩放，旋转，合成等 2D 处理，VPU 可以高效的进行视频解码），从而减轻 CPU 负担。

经过这些图像加速模块处理后的数据会存放在 DDR 中，然后由 VOP 读取，根据应用需求进行 Alpha 叠加，颜色空间转换，gamma 矫正，HDR 转换 等处理后，再发送到对应的显示接口模块（HDMI/DP/DSI/RGB/LVDS），这些接口模块会把接收到的数据转换成符合各自协议的数据流，发送到显示器或者屏幕上，呈现在最终用户眼前。

目前 Rockchip 平台上存在两种 VOP 架构—— VOP 1.0 和 VOP 2.0，他们的主要的区别是对多显的支持方式不同，VOP 1.0 是用多 VOP 的方式来实现多屏幕显示，即正常情况下，一个 VOP 在同一时刻只能输出一路独立的显示时序，驱动一个屏幕显示独立的内容。如果需要实现双屏显示，则需要有两个 VOP 来实现，所以在 RK3288，RK3399，PX30 等支持双显的平台上，都有两个独立的 VOP。

VOP 2.0 采用了统一显示架构，即整个 SOC 上只存在一个 VOP，但是在 VOP 的后端设计了多路独立的 Video Port(简称 VP) 输出接口，这些 VP 能够同时独立工作，并且输出相互独立的显示时序。比如在上面的 VOP 2.0 框图中，有三个 VP，就能同时实现三屏异显。

如果想了解哪些芯片采用的是 VOP 1.0 架构，哪些芯片采用的是 VOP 2.0 架构，请参考后面的 **Display Feature** 章节内容。

2. DRM 概述

DRM 全称是 Direct Rendering Manager，进行显示输出管理、buffer 分配、帧缓冲。对应的 userspace 库为 libdrm，libdrm 库提供了一系列友好的控制封装，使用户可以方便的进行显示的控制和 buffer 申请。DRM 的设备节点为 "/dev/dri/cardX"， X 为 0-15 的数值，默认使用的是 /dev/dri/card0。

Rockchip 平台从 Linux 4.4 内核开始，显示驱动全部切到 DRM 显示框架。

需要注意的是，在一个系统中，可能存在多个 DRM 设备，这些设备产生的设备节点 `/dev/dri/cardX` 是不固定的，他们和 DRM 驱动的加载顺序相关。

比如 Rockchip 目前的平台上，显示子系统和 NPU 驱动都使用了 DRM 框架，所以大部分情况下，我们会在系统上看到两个设备节点：

`/dev/dri/card0` 和 `/dev/dri/card1` ， 但是 card0 和 card1 和显示子系统与 NPU 没有固定的对应关系。一般情况下哪个驱动先加载，哪个驱动会被注册为 card0。

如果某个应用需要打开 DRM 显示设备，则该应用在打开 `/dev/dri/card0` 或者 `/dev/dri/card1` 后需要做进一步判断，判断该设备是否是显示设备，在 Rockchip 平台上，目前有两种方法：

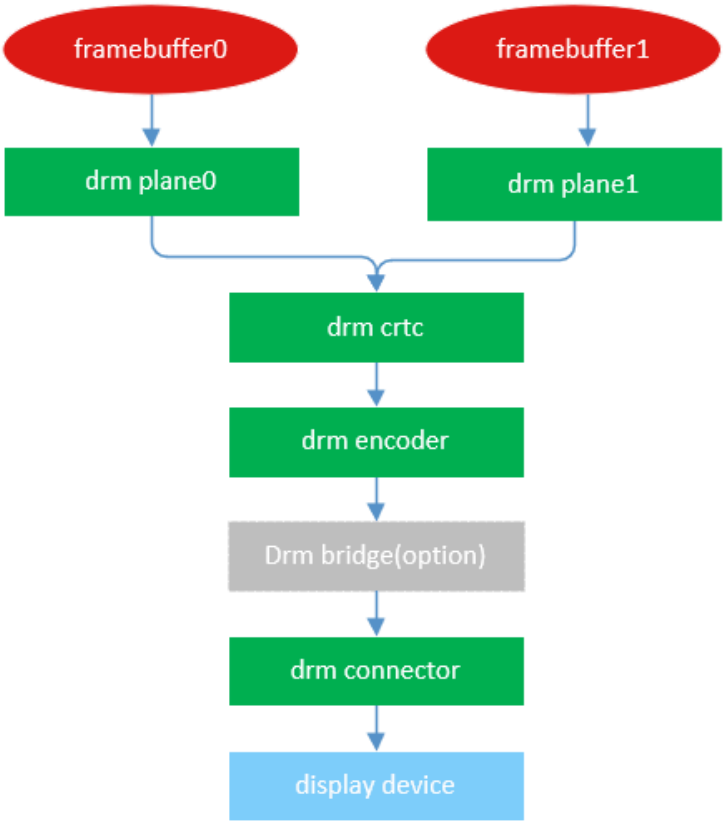
1. 使用 libdrm 提供的 `drmIsKMS(int fd)` API 对打开的设备做检查，判断该设备是否支持显示功能，NPU 设备不支持显示功能
2. 使用 libdrm 提供的 `drmOpen("rockchip", NULL)` ，强制指定打开 rockchip drm 驱动。

2.1 基本概念

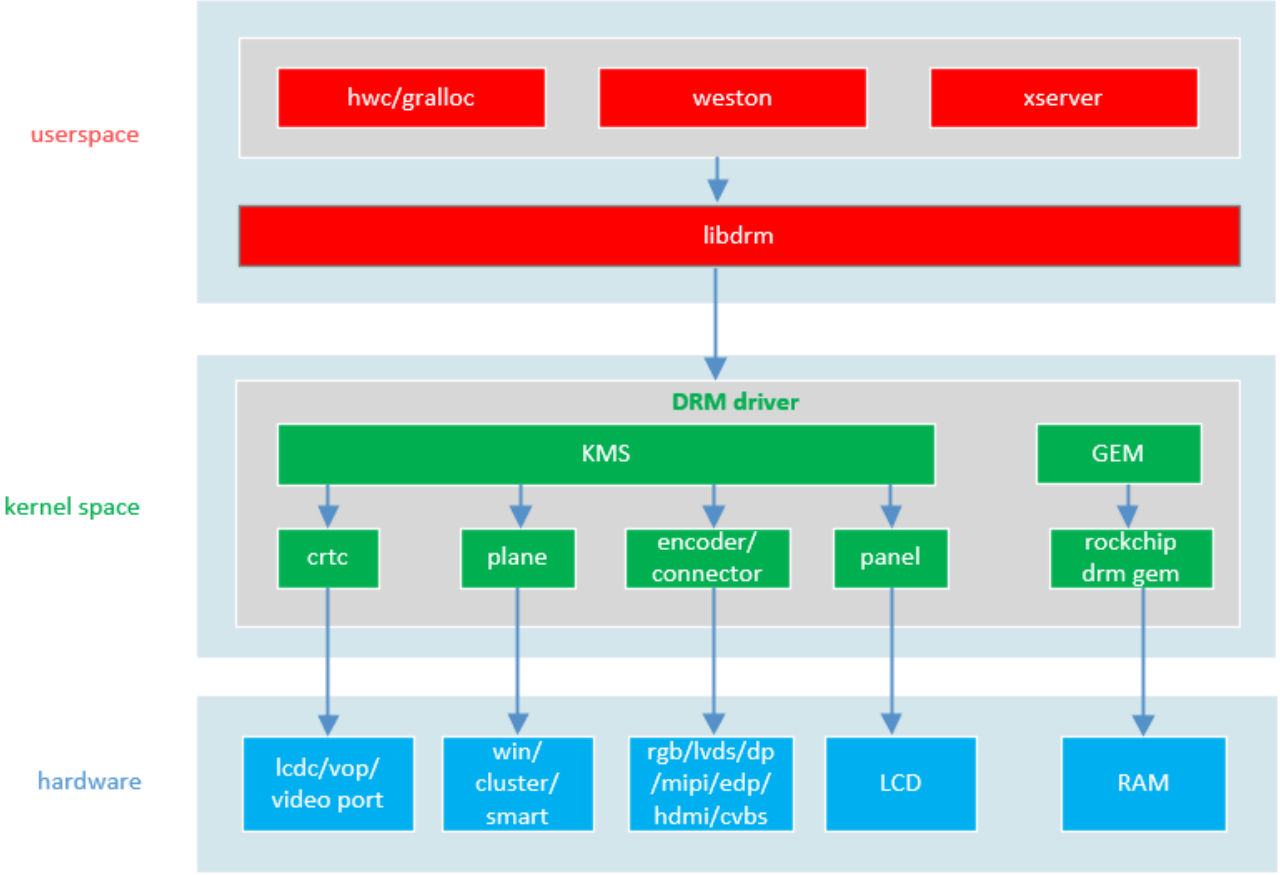
为了方便管理显示通路上的各种硬件模块，DRM 定义了以下几个概念：

基本概念	说明
CRTC	显示控制器，在 rockchip 平台是 SOC 内部 VOP（部分文档也称为 LCDC）模块或者 VOP2 中 Video Port 的抽象
Plane	图层，在 rockchip 平台是 SOC 内部 VOP（LCDC）模块 win 图层的抽象
Encoder	输出转换器，指 RGB、LVDS、DSI、eDP、DP、HDMI、CVBS、VGA 等显示接口
Connector	连接器，指 encoder 和 panel 之间交互的接口部分
Bridge	桥接设备，一般用于注册 encoder 后面另外再接的转换芯片，如 DSI2HDMI 转换芯片
Panel	泛指屏，各种 LCD 显示设备的抽象
GEM	DRM 下 buffer 管理和分配，类似 ION、DMA BUFFER

2.2 显示通路



2.3 DRM 驱动和 libdrm 的交互过程



3. 软件驱动

3.1 U-Boot 驱动

显示驱动在 U-Boot 中主要提供开机 logo 显示和充电界面显示这两个功能。

在 Rockchip 平台上，开机 logo 一般分为两个阶段：显示 U-Boot logo 和 显示 Kernel logo。

默认 U-Boot logo

默认 Kernel logo

这两个 LOGO 图片默认放在 Linux kernel 根目录下（logo.bmp 和 logo_kernel.bmp），Linux Kernel 在编译的时候会把他们打包到 resource.img 中，再打包进入 Boot.img。

U-Boot 启动的时候会把这两个文件加载到内存中，U-Boot LOGO 在 U-Boot 阶段就开始显示，Kernel LOGO 在内存中的地址会被 U-Boot 传递给 Linux kernel，在 Linux Kernel 的 drm 驱动初始化阶段显示。

3.1.1 驱动目录

```
drivers/video/drm/
```

3.1.2 驱动文件

Driver	File
Core	rockchip_display.c rockchip_crtc.c rockchip_connector.c rockchip_phy.c rockchip_panel.c rockchip_bridge.c
VOP	rockchip_vop.c rockchip_vop_reg.c rockchip_vop2.c rockchip_vop2_reg.c
RGB	rockchip_rgb.c inno_video_combo_phy.c
LVDS	rockchip_lvds.c inno_video_combo_phy.c
MIPI-DSI	drm_mipi_dsi.c dw_mipi_dsi2.c inno_mipi_phy.c inno_video_combo_phy.c samsung_mipi_dcphy.c
eDP	rockchip_analogix_dp.c rockchip_analogix_dp_reg.c
HDMI	dw_hdmi.c rockchip_dw_hdmi.c rockchip-inno-hdmi-phy.c inno_hdmi.c dw_hdmi_qp.c rockchip_dw_hdmi_qp.c phy-rockchip-samsung-hdptx-hdmi.c
TVE /CVBS	rockchip_drm_tve.c
DP	dw-dp.c drm_dp_helper.c phy-rockchip-usbdp.c

以上驱动文件在不同的 U-Boot 版本可能会做些小的调整，但多数驱动文件和框架部分基本不会变化，查阅的时候可以根据实际情况调整。

3.1.3 接口说明

1. 显示 U-Boot logo

```
void rockchip_show_logo(void)
```

2. 显示指定的 bmp 图片，目前主要用于充电 logo 的显示

```
void rockchip_show_bmp(const char *bmp)
```

3. 将 U-Boot 中确定的一些变量通过修改 dtb 文件传递给内核，包括 kernel logo 的大小、地址、格式、输出扫描时序以及过扫描的配置等信息

```
void rockchip_display_fixup(void *blob)
```

3.1.4 Logo Display 应用说明

1. 开启 U-Boot logo

   logo 通过 Linux kernel dts(U-Boot 显示模块和 Linux kernel 复用同一个 dtb) 中对应显示接口的 route_xxx 节点控制，这里的 xxx 可以是 dsi，edp，hdmi，lvds，具体请搜索 dts 中的 route 关键字。

   以 MIPI DSI0 为例，在对应的板级 dts 文件里找到 route_dsi0 节点，把 status 设置为 “okay”:

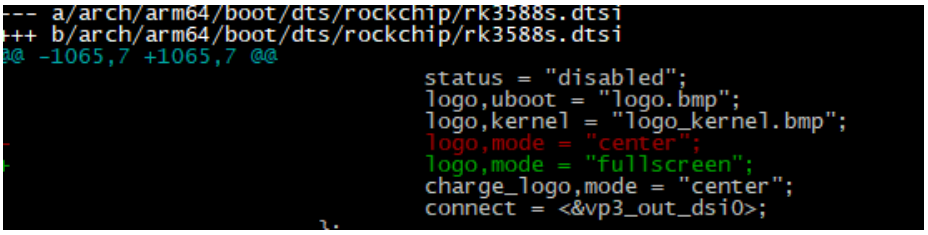
```
&route_dsi0 {
        status = "okay";
        connect = <&vp3_out_dsi0>;
};
```

   `connect` 属性参考后面 **DTS 配置** 章节的详细说明。

2. 配置 U-Boot logo 全屏显示

   logo 默认是居中（center）显示，如果需要全屏显示，修改对应接口的 route 节点。

   比如需要把 DSI0 上输出的 logo 全屏显示，则修改 route_dis0 的 logo,mode 属性为 “fullscreen”。



3. logo 图片要求

- U-Boot logo 和 Linux Kernel logo 分辨率相同，由于 VOP 虚宽是以 word 为单位，所以分辨率宽必须是 4 pixel 对齐，对于屏幕本身是非 4 pxiel 对齐同时需要全屏显示的，可以将图片向下对齐到 4 pixel，同时将配置 `logo,mode = "fullscreen"` ；
- 只支持 8bit，16bit，24bit、32bit 的 bmp 图片；
- U-Boot logo 和 Linux Kernel logo 必须同时开启，即 logo.bmp 和 logo_kernel.bmp 必须同时提供，不能只提供其中一个。

4. 启动 log 确认

   如果 logo 功能正常启用，在 U-Boot 阶段会看到类似如下的 log：

```
Rockchip UBOOT DRM driver version: v1.0.1
vp0 have layer nr:2[0 2 ], primary plane: 2
vp1 have layer nr:2[1 3 ], primary plane: 3
vp2 have layer nr:2[6 8 ], primary plane: 8
vp3 have layer nr:2[7 9 ], primary plane: 9
Using display timing dts
dsi@fde20000:  detailed mode clock 132000 kHz, flags[a]
    H: 1080 1095 1099 1129
    V: 1920 1935 1937 1952
bus_format: 100e
VOP update mode to: 1080x1920p0, type: MIPI0 for VP3
VOP VP3 enable Esmart3[654x270->654x270@213x825] fmt[2] addr[0xedf04000]
final DSI-Link bandwidth: 880000 Kbps x 4
......
hdmi_select_link_config use tmds mode
mode:1920x1080 bus_format:0x100a
hdmi@fde80000:  detailed mode clock 148500 kHz, flags[5]
    H: 1920 2008 2052 2200
    V: 1080 1084 1089 1125
bus_format: 100a
VOP update mode to: 1920x1080p0, type: HDMI0 for VP0
```

```
......
VOP VP0 enable Esmart0[654x270->654x270@633x405] fmt[2] addr[0xedf04000]
......
CLK: (uboot. arm: enter 1200000 KHz, init 1200000 KHz, kernel 0N/A)
```

可以看到在 VP0 和 VP3 上都启动了 logo 显示。

VP0 的显示分辨率为 1920 x 1080，显示接口是 HDMI。

VP3 的显示分辨率为 1080 x 1920，显示接口是 MIPI DSI。

显示的 logo 大小为 654 x 270。

5. 如何让 logo 旋转固定的角度

   如果因为横竖屏等原因，想要 logo 旋转一定角度显示，可以在 PC 上旋转好保存成新的图片，再替换到 kernel 工程下的 logo.bmp 和 kernel_logo.bmp；

6. split 模式显示 logo

   split 模式即一个 crtc 连接两个 connector 接2个屏，如果想要左右屏显示不同内容并拼接成一个完整图像的，可以在 PC 上将左右屏的图片拼接好保存成一个完整的图像，再替换到 kernel 工程下的 logo.bmp 和 kernel_logo.bmp；

7. 如何只显示一张图片

   如果想要 uboot 和 kernel 阶段只显示一张图片，只要让 logo.bmp 和 kernel_logo.bmp 使用相同的图片或者通过把 dts 文件 route 节点下的“logo,uboot” 和 “logo,kernel” 属性配置为同一张图片即可；

8. 如何支持 4K 分辨率 logo 显示功能

   各平台的 defconfig 是无法支持 4K logo 图片正常显示的，串口会有类似下述 log：

```
......
bmp[logo.bmp] size[23MB] is over the limitation MAX_IMAGE_BYTES[8MB]
failed to display uboot logo
CLK: (uboot. arm: enter 1008000 KHz, init 1008000 KHz, kernel 0N/A)
  bpll 816000 KHz
  lpll 816000 KHz
......
```

以 RK3576 平台为例，需要添加如下修改：

```
diff --git a/drivers/video/drm/rockchip_display.c b/drivers/video/drm/rockchip_display.c
index b1773ba6942..d1606db1ba5 100644
--- a/drivers/video/drm/rockchip_display.c
+++ b/drivers/video/drm/rockchip_display.c
@@ -52,7 +52,7 @@
 #define RK_BLK_SIZE 512
 #define BMP_PROCESSED_FLAG 8399
 #define BYTES_PER_PIXEL sizeof(uint32_t)
-#define MAX_IMAGE_BYTES (8 * 1024 * 1024)
+#define MAX_IMAGE_BYTES (32 * 1024 * 1024)

 DECLARE_GLOBAL_DATA_PTR;
 static LIST_HEAD(rockchip_display_list);
diff --git a/include/configs/rk3576_common.h b/include/configs/rk3576_common.h
index 16abba314c5..b4a8ec3b898 100644
--- a/include/configs/rk3576_common.h
+++ b/include/configs/rk3576_common.h
@@ -22,7 +22,7 @@
 #endif
 #define CONFIG_SPL_LOAD_FIT_ADDRESS    0x42000000

-#define CONFIG_SYS_MALLOC_LEN          (32 << 20)
+#define CONFIG_SYS_MALLOC_LEN          (32 << 21)
 #define CONFIG_SYS_CBSIZE              1024

 #ifdef CONFIG_SUPPORT_USBPLUG
```

并且需要将配置项 CONFIG_DRM_MEM_RESERVED_SIZE_MBYTES 修改为 64 MB。

上述修改的原因为：

（1）出于内存占用及默认 parameter 分区表配置等方面考虑，不建议使用太大的 BMP logo 图片，所以将 MAX_IMAGE_BYTES 限制到 8 MB，超过此大小的建议使用 BI_RLE4/BI_RLE8 格式 BMP 图片。

（2）常见的 4K BI_RGB 24bpp BMP 图片大小约为 24 MB，各平台默认的 malloc heap 大小通常为 32 MB，会出现 BMP decode 相关函数无法申请到内存问题。

（3）为了保证 U-Boot 到 Kernel logo 显示的平滑切换，通常两张 logo 图片会是相同的分辨率和格式，这样预留给 logo 显示功能的内存区域大小（由 CONFIG_DRM_MEM_RESERVED_SIZE_MBYTES 决定，默认为 32 MB）就不够了。

9. 如何让不同的屏幕显示不同的内容

可以参考以下方法修改，让 resource.img 打包新的图片，并修改对应的 dts 文件：

```diff
--- a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
@@ -854,7 +854,7 @@ &display_subsystem {
        route {
                route_dp1: route-dp1 {
                        status = "disabled";
-                       logo,uboot = "logo.bmp";
+                       logo,uboot = "logo_new.bmp";
                        logo,kernel = "logo_kernel.bmp";
                        logo,mode = "center";
                        charge_logo,mode = "center";
--- a/scripts/mkimg
+++ b/scripts/mkimg
+LOGO_NEW_PATH=${srctree}/logo_new.bmp
+[ -f ${LOGO_NEW_PATH} ] && LOGO_NEW=logo_new.bmp
+
 LOGO_KERNEL_PATH=${srctree}/logo_kernel.bmp
 [ -f ${LOGO_KERNEL_PATH} ] && LOGO_KERNEL=logo_kernel.bmp

@@ -233,6 +236,9 @@ if [ -x ${srctree}/scripts/bmpconvert ]; then
        if [ -f ${LOGO_PATH} ]; then
                ${srctree}/scripts/bmpconvert ${LOGO_PATH};
        fi
+       if [ -f ${LOGO_NEW_PATH} ]; then
+               ${srctree}/scripts/bmpconvert ${LOGO_NEW_PATH};
+       fi
@@ -242,12 +248,15 @@ if [ "${srctree}" != "${objtree}" ]; then
        if [ -f ${LOGO_PATH} ]; then
                cp -a ${LOGO_PATH} ${objtree}/;
        fi
+       if [ -f ${LOGO_NEW_PATH} ]; then
+               cp -a ${LOGO_NEW_PATH} ${objtree}/;
+       fi
 fi
-scripts/resource_tool ${DTB_PATH} ${LOGO} ${LOGO_KERNEL} >/dev/null
-echo "  Image:  resource.img (with ${DTB} ${LOGO} ${LOGO_KERNEL}) is ready"
+scripts/resource_tool ${DTB_PATH} ${LOGO} ${LOGO_NEW} ${LOGO_KERNEL} >/dev/null
+echo "  Image:  resource.img (with ${DTB} ${LOGO} ${LOGO_NEW} ${LOGO_KERNEL}) is ready"
```

3.1.5 Video Console 应用说明

该功能用于在屏幕或显示器上显示 U-Boot 的启动 logs 和命令行界面。在《U-Boot 驱动》章节中提到 Logo 显示功能分为 U-Boot Logo 和 Kernel Logo 两个阶段，使能 Video Console 功能后，U-Boot Logo 显示将被 Video Console 文本显示替换，而 Kernel Logo 仍会正常显示。

DTS 配置：参考《Logo Display 应用说明》中步骤 1 DTS 配置说明使能 U-Boot 显示功能。

defconfig 配置：

- 使能 Video Console 配置：

```
/* enable the video console */
CONFIG_ROCKCHIP_VIDCONSOLE=y
```

- 配置 Video Console 的分辨率和 BPP（Bits Per Pixel），以 1920x1080 16bpp 为例：

```
/* the resolution of the video console */
CONFIG_ROCKCHIP_VIDCONSOLE_WIDTH=1920
CONFIG_ROCKCHIP_VIDCONSOLE_HEIGHT=1080

/* the bits per pixel the of video console */
CONFIG_ROCKCHIP_VIDCONSOLE_BPP32 is not set
CONFIG_ROCKCHIP_VIDCONSOLE_BPP16=y
CONFIG_ROCKCHIP_VIDCONSOLE_BPP8 is not set
```

  - 若未指定，默认横向分辨率为各显示通路对应屏幕或显示器横向分辨率中的最小值，纵向分辨率同样为其纵向分辨率中的最小值，这样是为了保证默认 framebuffer size 的最小化。

    以 RK3588 为例，假设此时 VP0 HDMI0 输出分辨率为 1920x1080，而 VP3 DSI0 输出分辨率为 1080x1920，则此时 Video Console 的默认分辨率为 1080x1080。
  - 若未指定，BPP 默认为 32。

- 配置 Video Console 的 framebuffer size，计算公式为：

$$\text{size} = \frac{\text{width} \times \text{height} \times \text{bpp}}{8 \times 1024 \times 1024}$$

以配置 1920x1080 32bpp 为例：

$$\text{size} = \frac{1920 \times 1080 \times 32}{8 \times 1024 \times 1024} = 8\,\text{MB}$$

```
/* the framebuffer size of video console */
CONFIG_ROCKCHIP_VIDCONSOLE_MEM_RESERVED_SIZE_MBYTES=8
```

  - 若未指定，默认的 framebuffer size 为 16M。如果 Video Console 需要的 framebuffer size （以 3840x2160 32bpp 为例）超过了默认值，将有下述报错信息并回退到 U-Boot Logo 显示：

    ```
    ......
    video console fb size [33177600 Bytes] is over reserved [16777216 Bytes], then back to show uboot
    logo
    ......
    ```

3.1.6 分析 U-Boot logo 过渡到 Linux Kernel logo 过程中出现的闪屏或者无法显示的问题

1. 确认 DRM 驱动是否有正常加载

   DRM 驱动加载过程中可能会出现一些资源没有准备好，导致 DRM 驱动 bind 失败，可能会出现类似以下 log：

   ```
   [1.792387] dw-mipi-dsi2 fde20000.dsi: [drm:dw_mipi_dsi2_bind] *ERROR* Failed to find panel or bridge:
   -517
   ```

   这个 log 说明此时 panel 或者 bridge 没准备好，正常驱动框架会在一段时间后重新开始 bind，但如果最后出现以下 log 说明此时 drm 驱动已经加载成功了，这种情况我们就不需要太在意前面一两次 bind 失败的 log：

   ```
   [2.566831] rockchip-drm display-subsystem: [drm] fb0: rockchipdrmfb frame buffer device
   ```

   如果开机 log 一直不停的刷 bind 失败的 log，那可能要检查的你 dts 配置，产品中最经常遇到的是 GPIO 口被其他设备先注册、panel 的 compatible 未正确配置导致 panel 注册失败、backlight 驱动注册失败等。

如果 Linux kernel log 中出现了如下 logo 失败相关的信息，则需要结合代码认真分析：

```
rockchip-drm display-subsystem: failed to parse resources for logo display
rockchip-drm display-subsystem: connector[HDMI-A-1] can't found any modes
rockchip-drm display-subsystem: can't not find any logo display
rockchip-drm display-subsystem: failed to show loader logo
```

2. DDR 变频导致闪屏或者显示错位

(1) 尝试关闭 dts 文件中 DDR 变频节点，保证内核阶段不做 DDR 变频，修改方法如下：

```
&dmc {
    status = “disabled”;
};
```

(2) 尝试关闭 DDR 变频时对 DCLK 频率的调整，修改方法如下：

```
&dmc {
    vop-dclk-mode = <1>;
};
```

3. clk tree 变化导致

部分平台 U-Boot 中的 clk tree 配置和内核的 clk tree 配置是独立的，如果两个驱动的 clk 策略不一致，就有可能在 Linux kernel clk 重新初始化的时候出现闪屏问题，以 RK3399 为例，可以按如下方法确认：

(1) 在 rk3399_clk_init()@kernel/drivers/clk/rockchip/clk-rk3399.c 函数入口处加上 while(1)；确认是否会有闪屏问题；

(2) 在 rk3399_clk_init()@kernel/drivers/clk/rockchip/clk-rk3399.c 函数结束处加上 while(1)，确认是否会有闪屏问题；

(3) 如果步骤 (1) 中无闪屏现象步骤 (2) 中有闪屏现象，那基本可以确认是 clk tree 变化导致闪屏问题，可以直接找对应平台 cru 负责人或者提交 redmine 并说明转给 pll 相关负责人。

4. 时钟被关闭导致

有一些流程上的问题或者软件上的 bug 可能存在一些必要的时钟在驱动注册的时候没有被使能，导致这些时钟在内核跑完后被框架自动关闭从而导致显示异常，可以尝试按下面的修改默认不关闭时钟做测试：

在 dts 文件中，找到 chosen 节点，在 bootargs 末尾加上 clk_ignore_unused：如 bootargs = "xxxx clk_ignore_unused”；

```
--- a/arch/arm64/boot/dts/rockchip/rk3399-linux.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3399-linux.dtsi
@@ -47,7 +47,7 @@
        compatible = "rockchip,linux", "rockchip,rk3399";

        chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xff1a0000";
+               bootargs = "earlycon=uart8250,mmio32,0xff1a0000 clk_ignore_unused";
        };
```

5. U-Boot logo 图片和 kernel logo 图片大小不一致

rockchip 平台有要求 U-Boot logo 和 kernel logo 的图片分辨率大小一样，如果出现 U-Boot logo 显示正常，到内核阶段显示异常，可以确认下 kernel 目录下 logo.bmp 和 logo_kernel.bmp 分辨率是否一致；

```
$file logo.bmp  logo_kernel.bmp
logo.bmp:        PC bitmap, Windows 3.x format, 654 x 258 x 8
logo_kernel.bmp: PC bitmap, Windows 3.x format, 654 x 258 x 8
```

6. 内核初始化过程一些电源 GPIO 被重新初始化

该问题涉及的可能性很多，总之在新项目 porting 过程中要及时确认显示相关的 GPIO 电源是否和其他模块有冲突；

如果 DTS 确认无误，可以在内核代码搜索串口中的关键字，通过二分法在各个模块加载的位置 while 住，逐步确认导致闪屏问题的点；

7. VOP 优先级配置问题

如果 VOP 优先级没有被配置最高，有可能在内核加载阶段被其他 IP 抢占总线导致闪屏问题，该问题一般会在 SDK 发布前 Fix。

8. 测试相关电源和信号

如果以上还未找到闪屏问题，请使用示波器抓取 CLK、DATA 和电源等相关信号从 U-Boot 到内核阶段的波形图并提交 redmine。

3.2 kernel 驱动

3.2.1 驱动目录

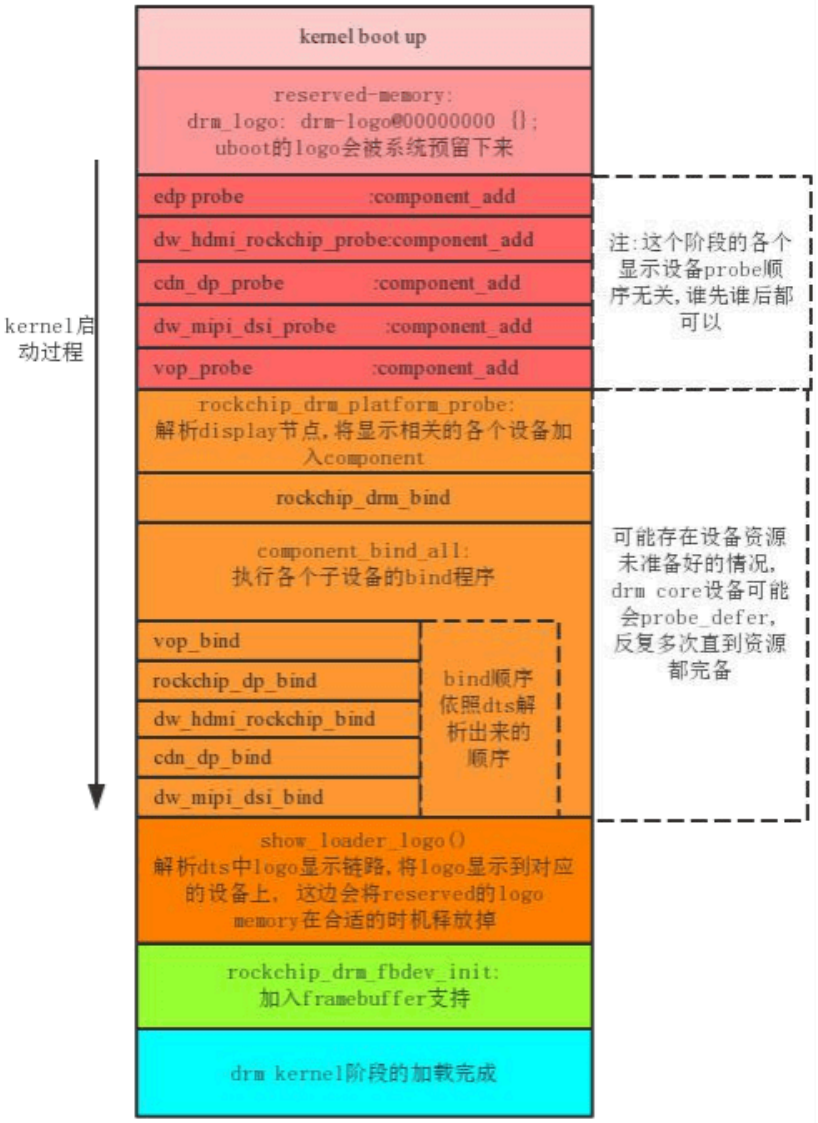
```
drivers/gpu/drm/rockchip/
drivers/gpu/drm/bridge/analogix/
drivers/gpu/drm/bridge/synopsys/
drivers/phy/rockchip/
```

3.2.2 驱动文件

Driver	File	Doc
Core	rockchip_drm_drv.c rockchip_drm_fb.c rockchip_drm_fbdev.c rockchip_drm_gem.c rockchip_drm_logo.c rockchip_drm_direct_show.c panel-simple.c	rockchip-drm.txt or rockchip-drm.yaml
VOP	rockchip_drm_vop.c rockchip_vop_reg.c rockchip_drm_vop2.c rockchip_vop2_reg.c	rockchip-vop.txt or rockchip-vop.yaml
RGB	rocckhip_rgb.c phy-rockchip-inno-video-combo-phy.c	rockchip-rgb.txt
LVDS	rockchip_lvds.c phy-rockchip-inno-video-combo-phy.c	rockchip-lvds.txt
MIPI-DSI	dw-mipi-dsi.c phy-rockchip-inno-dsidphy.c phy-rockchip-inno-video-combo-phy.c dw-mipi-dsi2-rockchip.c phy-rockchip-samsung-dcphy.c	dw_mipi_dsi_rockchip.txt phy-rockchip-inno-mipi-dphy.txt
eDP	analogix_dp_core.c analogix_dp-rockchip.c analogix_dp_reg.c phy-rockchip-dp.c	analogix_dp-rockchip.txt analogix_dp.txt rockchip-dp-phy.txt
DP	cdn-dp-core.c cdn-dp-reg.c dw-dp.c phy-rockchip-usbdp.c phy-rockchip-samsung-hdptx-hdmi.c	cdn-dp-rockchip.txt
HDMI	inno_hdmi.c dw-hdmi.c dw_hdmi-rockchip.c phy-rockchip-inno-hdmi-phy.c dw-hdmi-qp.c phy-rockchip-samsung-hdptx-hdmi.c	inno_hdmi-rockchip.txt dw_hdmi-rockchip.txt phy-rockchip-inno-hdmi-phy.txt
TVE/CVBS	rockchip_drm_tve.c	rockchip_drm_tve.txt

以上驱动文件在不同的 kernel 版本可能会做些小的调整，但多数驱动文件和框架部分基本不会变化，查阅的时候可以根据实际情况调整。

3.2.3 驱动加载流程



需要注意的是，DRM 驱动是一系列显示相关模块的驱动的结合，他包含了 backlight、panel、rgb、lvds、dsi、edp、lvds、hdmi、vop 等等显示通路上的依赖模块。只有这些相互依赖的模块都加载起来，整个 drm 系统才能启动成功。

因为这些复杂的依赖关系，在 drm 系统初始化的过程中，可能会出现某个资源暂时未就绪，而导致某个模块暂时无法顺利加载的情况，为了解决这种问题，drm 驱动利用了 Linux 驱动 中的 `deferred probe` 机制，当发现某个依赖的资源未就绪的时候，驱动返回 `-EPROBE_DEFER(-517)`，然后退出。Linux kernel 会在稍后再次尝试加载这个驱动，直到依赖的资源就绪，驱动顺利加载为止。

```
[1.747190] rockchip-drm display-subsystem: bound fdd90000.vop (ops vop2_component_ops)
[1.747877] dwhdmi-rockchip fde80000.hdmi: registered ddc I2C bus driver
[1.748022] rockchip-drm display-subsystem: bound fde80000.hdmi (ops dw_hdmi_rockchip_ops)
[1.748676] dwhdmi-rockchip fdea0000.hdmi: registered ddc I2C bus driver
[1.748807] rockchip-drm display-subsystem: bound fdea0000.hdmi (ops dw_hdmi_rockchip_ops)
[1.748840] dw-mipi-dsi2 : [drm:dw_mipi_dsi2_bind] *ERROR* Failed to find panel or bridge: -517
[1.755174] panel-simple-dsi fde20000.dsi.0: failed to get power regulator: -517
[1.759296] brd: module loaded
[1.764374] loop: module loaded
[1.764528] zram: Added device: zram0
[1.764698] system_heap: orders[0] = 6
[2.248871] imx415 5-001a: supply dovdd not found, using dummy regulator
[2.416673] rockchip-drm display-subsystem: bound fdd90000.vop (ops vop2_component_ops)
[2.418711] dwhdmi-rockchip fde80000.hdmi: registered ddc I2C bus driver
```

```
[2.420336] rockchip-drm display-subsystem: bound fde80000.hdmi (ops dw_hdmi_rockchip_ops)
[2.421725] dwhdmi-rockchip fdea0000.hdmi: registered ddc I2C bus driver
[2.422291] rockchip-drm display-subsystem: bound fdea0000.hdmi (ops dw_hdmi_rockchip_ops)
[2.422318] dw-mipi-dsi2 : [drm:dw_mipi_dsi2_bind] *ERROR* Failed to find panel or bridge: -517
[2.433888] input: adc-keys as /devices/platform/adc-keys/input/input3

[2.466237] rockchip-drm display-subsystem: bound fdd90000.vop (ops vop2_component_ops)
[2.468705] dwhdmi-rockchip fde80000.hdmi: registered ddc I2C bus driver
[2.469751] rockchip-drm display-subsystem: bound fde80000.hnd fde20000.dsi (ops dw_mipi_dsi2_ops)
[2.472282] rockchip-drm display-subsystem: bound fde50000.dp (ops dw_dp_component_ops)
[2.472319] rockchip-drm display-subsystem: bound fde60000.dp (ops dw_dp_component_ops)
[2.531892] rockchip-drm display-subsystem: [drm] fb0: rockchipdrmfb frame buffer device
[2.532850] [drm] Initialized rockchip 3.0.0 20140818 for display-subsystem on minor 0
```

从上面的 log 我们可以看到，在第 6 行和第 18 行，mipi dsi 驱动因为找不到 panel 或者 bridge 这个依赖资源，而返回 `-EPROBE_DEFER` 退出，一直到第 23 行，dsi 驱动获取到依赖的资源 bind 成功，最终看到整个 drm 驱动完成加载的标准 log。

```
[2.532850] [drm] Initialized rockchip 3.0.0 20140818 for display-subsystem on minor 0
```

3.2.4 DTS 配置

3.2.4.1 基础配置

在一颗 SOC 上，可能有多个 VOP，HDMI，eDP，DP，MIPI，Panel 模块，根据具体产品定义，一款产品可能只需要使用到其中一部分模块来组成显示通路。具体使用哪些模块，以及这些模块之间如何衔接则通过 dts 配置。

在每一个支持 drm 显示功能的 soc 的核心 dtsi 里面，都会有如下 `display_subsystem` 节点：

```
        display_subsystem: display-subsystem {
                compatible = "rockchip,display-subsystem";
                memory-region = <&drm_logo>, <&drm_cubic_lut>;
                memory-region-names = "drm-logo", "drm-cubic-lut";
                ports = <&vop_out>;
                devfreq = <&dmc>;

                route {
                        route_dsi0: route-dsi0 {
                                status = "disabled";
                                logo,uboot = "logo.bmp";
                                logo,kernel = "logo_kernel.bmp";
                                logo,mode = "center";
                                charge_logo,mode = "center";
                                connect = <&vp0_out_dsi0>;
                        };
                        route_dsi1: route-dsi1 {
                                status = "disabled";
                                logo,uboot = "logo.bmp";
                                logo,kernel = "logo_kernel.bmp";
                                logo,mode = "center";
                                charge_logo,mode = "center";
                                connect = <&vp0_out_dsi1>;
                        };
                        route_edp: route-edp {
                                status = "disabled";
                                logo,uboot = "logo.bmp";
                                logo,kernel = "logo_kernel.bmp";
                                logo,mode = "center";
                                charge_logo,mode = "center";
                                connect = <&vp0_out_edp>;
                        };
                        route_hdmi: route-hdmi {
```

```
                                status = "disabled";
                                logo,uboot = "logo.bmp";
                                logo,kernel = "logo_kernel.bmp";
                                logo,mode = "center";
                                charge_logo,mode = "center";
                                connect = <&vp1_out_hdmi>;
                        };
                        route_lvds: route-lvds {
                                status = "disabled";
                                logo,uboot = "logo.bmp";
                                logo,kernel = "logo_kernel.bmp";
                                logo,mode = "center";
                                charge_logo,mode = "center";
                                connect = <&vp1_out_lvds>;
                        };
                        route_rgb: route-rgb {
                                status = "disabled";
                                logo,uboot = "logo.bmp";
                                logo,kernel = "logo_kernel.bmp";
                                logo,mode = "center";
                                charge_logo,mode = "center";
                                connect = <&vp2_out_rgb>;
                        };
                };
        };
```

该节点控制 `rockchip_drm_drv.c` 驱动的加载。

memory-region 控制 logo 显示 buffer 的分配和传递，该属性在系统启动时会在 u-boot 中被动态修改，然后传递给内核。

route_xxx 节点控制着各个接口开机 logo 功能是否开启，默认处于关闭状态。

以 route_rgb 为例，解释相关属性的功能：

```
route_rgb: route-rgb {
    status = "disabled";
        logo,uboot = "logo.bmp";
        logo,kernel = "logo_kernel.bmp";
        logo,mode = "center";
        charge_logo,mode = "center";
        connect = <&vp2_out_rgb>;
};
```

`logo,uboot`、 `logo,kernel` 描述 U-Boot 阶段和 Kernel 阶段 logo 图片的名字，需要和真实的图片资源匹配。

`logo,mode` `charege_logo,mode` 控制 logo 图片的显示模式，有 `center` 和 `fullscreen` 两种模式。

`connect` 描述该显示接口和 VOP 的哪个 Video Port(VOP2.0 架构) 或者哪个 VOP（VOP1.0 架构)连接 。一般根据实际使用情况配置。

`display_subsystem` 节点的大部分属性不需要修改，一般只需要根据实际产品需求，开关对应的 logo 节点即可。

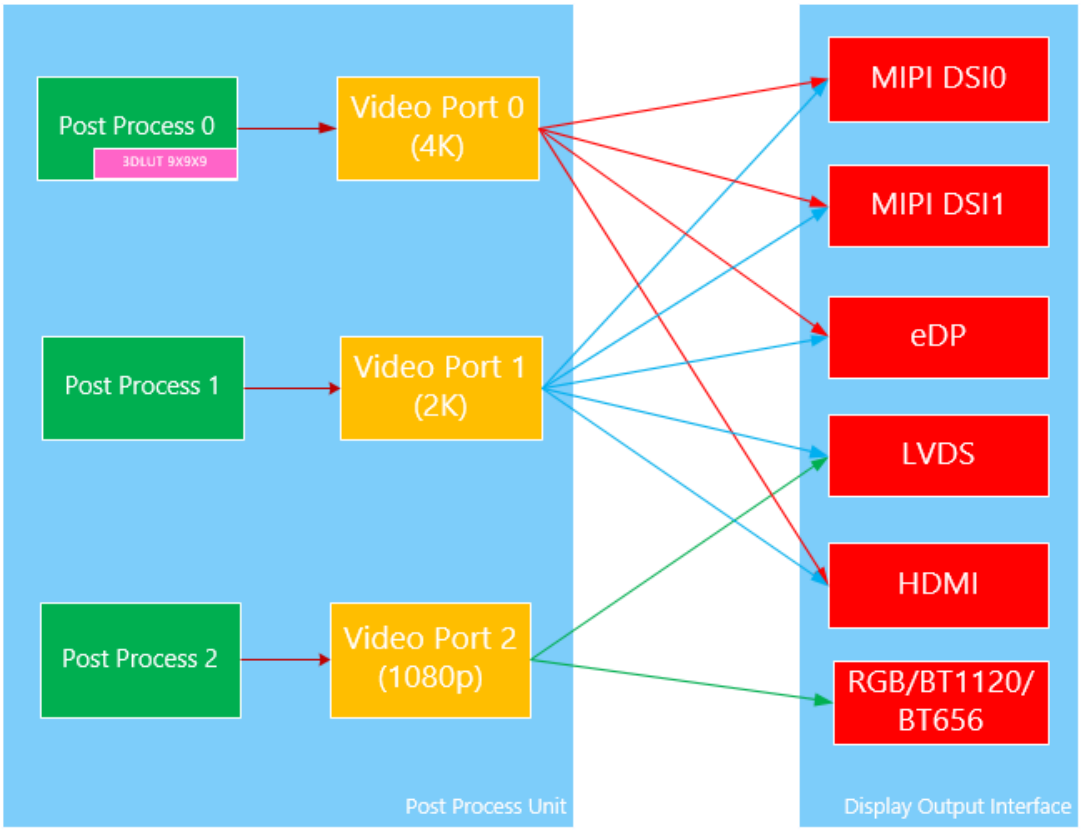
`vop` 节点：

```
vop: vop@fe040000 {
    compatible = "rockchip,rk3568-vop";
    reg = <0x0 0xfe040000 0x0 0x3000>, <0x0 0xfe044000 0x0 0x1000>;
    reg-names = "regs", "gamma_lut";
    rockchip,grf = <&grf>;
    interrupts = <GIC_SPI 148 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru ACLK_VOP>, <&cru HCLK_VOP>, <&cru DCLK_VOP0>, <&cru DCLK_VOP1>, <&cru DCLK_VOP2>;
    clock-names = "aclk_vop", "hclk_vop", "dclk_vp0", "dclk_vp1", "dclk_vp2";
    iommus = <&vop_mmu>;
    power-domains = <&power RK3568_PD_VO>;
    status = "disabled";
```

```
    vop_out: ports {
        ......
        vp0: port@0 {
            ......
            vp0_out_dsi0: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&dsi0_in_vp0>;
            };

            vp0_out_dsi1: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&dsi1_in_vp0>;
            };

            vp0_out_edp: endpoint@2 {
                reg = <2>;
                remote-endpoint = <&edp_in_vp0>;
            };

            vp0_out_hdmi: endpoint@3 {
                reg = <3>;
                remote-endpoint = <&hdmi_in_vp0>;
            };
        };

        vp1: port@1 {
            ......
            vp1_out_dsi0: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&dsi0_in_vp1>;
            };

            vp1_out_dsi1: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&dsi1_in_vp1>;
            };

            vp1_out_edp: endpoint@2 {
                reg = <2>;
                remote-endpoint = <&edp_in_vp1>;
            };

            vp1_out_hdmi: endpoint@3 {
                reg = <3>;
                remote-endpoint = <&hdmi_in_vp1>;
            };

            vp1_out_lvds: endpoint@4 {
                reg = <4>;
                remote-endpoint = <&lvds_in_vp1>;
            };
        };

        vp2: port@2 {
            ......
            vp2_out_lvds: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&lvds_in_vp2>;
            };

            vp2_out_rgb: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&rgb_in_vp2>;
            };
```

```
        };
    };
};
```

该节点描述 VOP 硬件资源，控制着 vop 驱动的加载 `rockchip_drm_vop.c/rockchip_drm_vop2.c`， 它描述了如下的显示通路连接关系：



`vop_out: ports` 节点描述 VOP 的输出通道，vp0/1/2 对应 VOP 上 Video Port0/1/2 三个独立的输出通路。

`vp0/1/2 : port` 下的 endpoint 节点描述 VP 和显示接口的连接关系，以上面的 dts 描述为例：vp0 节点下有 `vp0_out_dsi0` ，`vp0_out_dsi1`， `vp0_out_edp`， `vp0_out_hdmi` 四个节点，说明 vp0 可以和 dsi0、dsi1、edp、hdmi 四个显示接口连接。

每个 endpoint 通过`remote-endpoint` 属性和对应的显示接口组成一个连接通路 ，比如和 hdmi 显示接口的连接：

```
hdmi: hdmi@fe0a0000 {
    compatible = "rockchip,rk3568-dw-hdmi";
    ......
    status = "disabled";

    ports {
        ......
        port@0 {
            reg = <0>;
            ......
            hdmi_in_vp0: endpoint@0 {
                reg = <0>;
                remote-endpoint = <&vp0_out_hdmi>;
                status = "disabled";
            };

            hdmi_in_vp1: endpoint@1 {
                reg = <1>;
                remote-endpoint = <&vp1_out_hdmi>;
                status = "disabled";
            };
        };
    };
};
```

结合上面的 dts 描述我们可以知道，在 rk3568 上，hdmi 可以和 vop 的 vp0，vp1 连接。

需要注意的是，一个显示接口在同一个时刻只能和一个 vp 连接，所以在具体的板级配置中，需要在 dts 中把要使用的通路打开，把不使用的通路设置为 `disabled` 状态。

比如在某款产品上，希望 HDMI 连接在 vp0 上，则 dts 中需要做如下设置：

```
&hdmi {
        status = "okay";
};

&hdmi_in_vp0 {
        status = "okay";
};

&hdmi_in_vp1 {
        status = "disabled";
};

&route_hdmi {
        status = "okay";
        connect = <&vp0_out_hdmi>;
};
```

3.2.4.2 指定图层分配策略

VOP2 采用统一显示架构，各个独立的 Video Port 共享 VOP 内部的所有图层资源，而且这些图层需要排他性的使用，即某个图层在同一时刻只能为其中一个 Video Port 所独占。

为了充分合理的使用所有图层资源，我们会根据当前产品 dts 配置的接口类型和数量在 U-Boot 中生成了一种默认的图层分配策略，如果有些产品没有开 U-Boot logo 显示或者对图层使用有特殊的需求，可以参考下面的写法在 dts 根据需求自行指定图层分配策略：

rockchip,plane-mask：指定分配给该 VP 的图层 ID 掩码集合，图层 ID 定义在 `dt-bindings/display/rockchip_vop.h` 中。

rockchip,primary-plane：指定 primary 图层，当前 VP 的 primary 图层一定是 rockchip,plane-mask 中的一个，我们一般选用 Smart 或者 Esmart 图层。

图层分配的基本原则是：把所有图层（rk3568 有 6 个图层，rk3588 有 8 个图层）平均分配给各个使用的 VP，不使用的 VP 一般不分配图层。由于 VOP 内部不同类型（Cluster，Esmart，Smart）的图层，性能，限制不同，一般推荐各种类型的图层平均搭配分配。

如下是一个 RK3568 上的典型图层分配参考， RK3568 VOP 一共有 6 个图层（2 Cluster + 2 Esmart + 2 Smart），该配置支持三屏异显。一般，我们尽量给使用场景最多的屏幕(主屏) 对应的 VP 分配三个以上的图层（在该应用案例下是 VP1），其他接口尽量分配不少于两个图层。

```
#include <dt-bindings/display/rockchip_vop.h>        // 图层 ID 定义头文件

&vp0 {
    rockchip,plane-mask = <(1 << ROCKCHIP_VOP2_CLUSTER1 | 1 << ROCKCHIP_VOP2_SMART1)>;
    rockchip,primary-plane = <ROCKCHIP_VOP2_SMART1>;
};

&vp1 {
    rockchip,plane-mask = <(1 << ROCKCHIP_VOP2_CLUSTER0 | 1 << ROCKCHIP_VOP2_ESMART0 | 1 <<
ROCKCHIP_VOP2_SMART0)>;
    rockchip,primary-plane = <ROCKCHIP_VOP2_SMART0>;
};

&vp2 {
    rockchip,plane-mask = <(1 << ROCKCHIP_VOP2_ESMART1)>;
    rockchip,primary-plane = <ROCKCHIP_VOP2_ESMART1>;
};
```

配置生效后，系统启动的时候可以从 U-Boot 和 Linux kernel 的启动 log 中看到对应的 plane mask 解析信息。

```
Rockchip UBOOT DRM driver version: v1.0.1
VOP have 3 active VP
vp0 have layer nr:2[1 5 ], primary plane: 5
vp1 have layer nr:3[0 2 4 ], primary plane: 4
vp2 have layer nr:1[3 ], primary plane: 3
Using display timing dts
dsi@fe060000:  detailed mode clock 132000 kHz, flags[8000000a]
    H: 1080 1095 1097 1127
    V: 1920 1935 1937 1952
bus_format: 100e
VOP update mode to: 1080x1920p0, type: MIPI0 for VP1
VOP VP1 enable Smart0[654x270->654x270@213x825] fmt[2] addr[0x7df04000]
final DSI-Link bandwidth: 876 Mbps x 4
disp info 0, type:11, id:0
xfer: num: 2, addr: 0x50
xfer: num: 2, addr: 0x50
```

```
[2.314574] panel-simple-dsi fe060000.dsi.0: Specify missing connector_type
[2.315764] rockchip-vop2 : [drm:vop2_bind] vp0 assign plane mask: 0x22, primary plane phy id: 5
[2.315807] rockchip-vop2 : [drm:vop2_bind] vp1 assign plane mask: 0x15, primary plane phy id: 4
[2.315828] rockchip-vop2 : [drm:vop2_bind] vp2 assign plane mask: 0x8, primary plane phy id: 3
[2.316713] rockchip-drm display-subsystem: bound fe040000.vop (ops vop2_component_ops)
[2.317966] rockchip-drm display-subsystem: bound fe0c0000.edp (ops rockchip_dp_component_ops)
[2.318378] dwhdmi-rockchip : Detected HDMI TX controller v2.11a with HDCP (DWC HDMI 2.0 TX PHY)
[2.319625] dwhdmi-rockchip : registered DesignWare HDMI I2C bus driver
[2.321857] rockchip-drm display-subsystem: bound fe0a0000.hdmi (ops dw_hdmi_rockchip_ops)
[2.322069] rockchip-drm display-subsystem: bound fe060000.dsi (ops dw_mipi_dsi_rockchip_ops)
```

3.2.4.2.1 RK3566 图层分配策略

RK3566 IC 实现上有主图层和镜像图层的区别，即镜像图层 Cluster1 只能从主图层 Cluster0 对应的地址取数，同理 Esmart1/Smart1 只能从 Esmart0/Smart0 对应的地址取数，所以我们需要保证主图层被优先使用。正常产品的 U-Boot 显示驱动中会根据显示接口的类型设置好 plane-mask 属性，如果有些产品没有开 U-Boot logo 显示，可以在 dts 中按以下规则配置：

1. 只有一个屏显示，可以使用如下配置：

```
#include <dt-bindings/display/rockchip_vop.h>   //图层 ID 定义头文件

&vpx {  //x 取决于使用的 vp id，如 vp0
    rockchip,plane-mask = <(1 << ROCKCHIP_VOP2_CLUSTER0 | 1 << ROCKCHIP_VOP2_ESMART0 | 1 <<
ROCKCHIP_VOP2_SMART0 | 1 << ROCKCHIP_VOP2_CLUSTER1 | 1 << ROCKCHIP_VOP2_ESMART1 | 1 << ROCKCHIP_VOP2_SMART1)>;
    rockchip,primary-plane = <ROCKCHIP_VOP2_SMART0>;
};
```

2. 有两个屏显示【RK3566 目前只有 android 产品支持双显，且要求两个屏刷新帧率一致】，那我们让不支持热插拔设备(即始终连接显示的通路)使用主图层，另一个通路使用镜像图层：

```
#include <dt-bindings/display/rockchip_vop.h>   //图层 ID 定义头文件

&vpx {  //x 取决于使用的 vp id，如 vp0
    rockchip,plane-mask = <(1 << ROCKCHIP_VOP2_CLUSTER0 | 1 << ROCKCHIP_VOP2_ESMART0 | 1 <<
ROCKCHIP_VOP2_SMART0)>;
    rockchip,primary-plane = <ROCKCHIP_VOP2_SMART0>;
};  //单显或者双显时不支持热插拔的主显示设备使用主图层

&vpx {  //x 取决于使用的 vp id，如 vp1
    rockchip,plane-mask = <(1 << ROCKCHIP_VOP2_CLUSTER1 | 1 << ROCKCHIP_VOP2_ESMART1 | 1 <<
ROCKCHIP_VOP2_SMART1)>;
    rockchip,primary-plane = <ROCKCHIP_VOP2_SMART1>;
};
```

3.2.4.3 把某个图层设置为鼠标层

应用如果使用 Atomic 显示接口，可以不区分图层的类型(primary, overlay, cursor)，只需要根据图层支持的格式，使用任意可以使用的图层做各种类型的显示。

但是在 Linux 系统(非 Android) 下，还有一些应用使用 legacy 的 API，这些 API 对图层类型比较在意，比如使用 primary 图层显示桌面背景，使用 overlay 图层播放视频，使用 cursor 图层显示鼠标。

Rockchip drm 驱动默认只注册 primary 图层和 overlay 图层，不注册 cursor 图层，如果一些特殊的 Linux 系统希望使用 cursor 图层，可以在 dts 中对应的 vp 节点下设置 `cursor-win-id` 属性，为该 VP 对应的 crtc 分配一个 cursor 图层。

```
&vp0 {
    cursor-win-id = <ROCKCHIP_VOP2_CLUSTER0>;
};
```

- 因为 RK3566/RK3568 的 Cluster 图层只支持 AFBC 格式（不支持任何非 AFBC 压缩的 RGB/YUV 格式，在任何场景下把非 AFBC 格式的显示数据送到 Cluster 图层都是非法的），所以如果在 Linux 系统（buildroot，Debian 等非 Android 系统）上，指定 Cluster 图层为鼠标层的话，需要配合 Linux SDK 提供的 `libdrm-cursor` 库，该库可以针对具体的图层特性，做对应的数据格式转换，具体细节可参考 《Rockchip_Developer_Guide_Debian_CN.pdf》。

3.2.4.3.1 图层和 VP 之间的的连接关系

对于 RK356X/RK3588/RK3562 每一个图层和任意 VP 都有连接的，配置上没有特殊要求；对于 RK3528 和 RK3576 并不是每一个图层都可以连接到任意 VP 上，所以配置 VP 的 cursor 图层的时候需要选择能连接到当前 VP 的图层，以下是不同平台图层和 VP 的连接关系：

1. RK3528

VP	图层
VP0	Cluster0、Esmart0、Esmart1、Esmart2
VP1	Esmart2、Esmart3

2. RK3576

VP	图层
VP0	Cluster0、Cluster1、Esmart0、Esmart2
VP1	Cluster0、Cluster1、Esmart1、Esmart3
VP2	Esmart0、Esmart1、Esmart2、Esmart3

3.2.4.4 esmart 图层分割

RK3528 和 RK3576 几个 esmart 图层共享 linebuffer，可以根据产品形态对 linebuffer 进行分割来支持不同的图层数量和性能：

3.2.4.4.1 RK3528 esmart 图层分割

RK3528 平台的 esmart0/1/2/3 共享 linebuffer，默认配置为：VOP3_ESMART_4K_2K_2K_MODE，即此时除了固定的 cluster 图层，还注册 esmart0[4k]，esmart2[2k]，esmart3[2k]，也可以根据产品需求在 dts 中做修改，如要改成 2 个支持 4k 的图层，可以按以下配置：

```
&vop {
    esmart_lb_mode = /bits/ 8 <1>;
};
```

配置说明：

esmart_lb_mode	val	图层数量和性能
VOP3_ESMART_4K_4K_MODE	1	注册 cluster[4k]，esmart0[4k]，esmart2[4k]
VOP3_ESMART_4K_2K_2K_MODE	2	注册 cluster[4k]，esmart0[4k]，esmart2[2k]，esmart3[2k]
VOP3_ESMART_2K_2K_2K_2K_MODE	3	注册 cluster[4k]，esmart0[2k]，esmart1[2k]，esmart2[2k]，esmart3[2k]

3.2.4.4.2 RK3576 esmart 图层分割

RK3576 平台除了 Cluster0/1[4k] 和 esmart0/1[4k] 是固定的配置之外，esmart2[2k] 和 esmart3[2k] 也是共享 linebuffer 的设计，默认配置是 2 个 2k 的图层，如要改成 1 个支持 4k 的图层，可以按以下配置：

```
&vop {
    esmart_lb_mode = /bits/ 8 <4>;
};
```

配置说明：

esmart_lb_mode	val	图层数量和性能
VOP3_ESMART_4K_4K_4K_MODE	4	注册 cluster0[4k]，cluster1[4k]，esmart0[4k]，esmart1[4k]，esmart2[4k]
VOP3_ESMART_4K_4K_2K_2K_MODE	5	注册 cluster0[4k]，cluster1[4k]，esmart0[4k]，esmart1[4k]，esmart2[2k]，esmart3[2k]

3.2.4.5 禁止图层迁移

RK3568 和 RK3588 不支持运行时图层在各个 VP/CRTC 之间自由迁移，因此在大部分 Linux（非 Android） 系统上， 图层都应该被配置为被固定的 CRTC 独占，不允许在 CRTC 之间动态迁移，即需要确保在 dts 的 vop 节点下加上 `disable-win-move` 属性。

```
&vop {
        disable-win-move;
}
```

系统启动后，可以通过 `modetest` 查看图层的 `possible crtcs` 属性，每一个图层的 `possible crtcs` 属性值都只应该有一个 bit 被置位，否则就不符合预期。

3.2.5 色域相关配置说明

Linux 6.1 内核 DRM 驱动框架加了色域相关的标准定义，原来 DRM 驱动是借用 V4L2 的色域定义，这不是很正规，而且 V4L2 的定义也不是很好理解，所以在 Linux 6.1 内核将图层 Plane 色域定义切到 DRM 标准定义，以下是 RK 平台基于 DRM 驱动框架几个内核版本的色域配置说明：

3.2.5.1 Plane 色域配置说明

1. Linux 6.1 之前内核版本

使用 RK 私有属性：COLOR_SPACE

配置值	色域	说明
V4L2_COLORSPACE_SMPTE170M / V4L2_COLORSPACE_470_SYSTEM_M / V4L2_COLORSPACE_470_SYSTEM_BG	BT.601L	V4L2 定义
V4L2_COLORSPACE_REC709 / V4L2_COLORSPACE_SMPTE240M / V4L2_COLORSPACE_DEFAULT	BT.709L	V4L2 定义
V4L2_COLORSPACE_JPEG	BT.601F	V4L2 定义
V4L2_COLORSPACE_BT2020	BT.2020L	V4L2 定义
V4L2_COLORSPACE_BT709F	BT.709F	RK 私有定义
V4L2_COLORSPACE_BT2020F	BT.2020F	RK 私有定义

2. Linux 6.1 及之后内核版本

使用 DRM 框架定义的两个属性： COLOR_ENCODING 和 COLOR_RANGE

- COLOR_ENCODING

配置值	色域	说明
DRM_COLOR_YCBCR_BT601	BT.601	DRM 定义
DRM_COLOR_YCBCR_BT709	BT.709	DRM 定义
DRM_COLOR_YCBCR_BT2020	BT.2020	DRM 定义

- COLOR_RANGE

配置值	range	说明
DRM_COLOR_YCBCR_FULL_RANGE	Full	DRM 定义
DRM_COLOR_YCBCR_LIMITED_RANGE	Limit	DRM 定义

3.2.5.2 Connector 色域配置说明

1. Linux 5.10 之前内核版本

- 色域使用 RK 私有属性：hdmi_output_colorimetry

属性值	色域	说明
HDMI_COLORIMETRY_NONE	Default	RK 私有定义
RK_HDMI_COLORIMETRY_BT2020	BT.2020	RK 私有定义

- range 使用 RK 私有定义：hdmi_quant_range

配置值	range	说明
HDMI_QUANTIZATION_RANGE_DEFAULT	Default	RK 私有定义
HDMI_QUANTIZATION_RANGE_LIMITED	Limit	RK 私有定义
HDMI_QUANTIZATION_RANGE_FULL	Full	RK 私有定义

2. Linux 5.10 内核版本

- 色域使用 DRM 定义的标准属性：Colorspace

配置值	色域	说明
Default=0	Default	DRM 定义
DRM_MODE_COLORIMETRY_SMPTE_170M_YCC	SMPTE_170M_YCC	DRM 定义
DRM_MODE_COLORIMETRY_BT709_YCC	BT709_YCC	DRM 定义
DRM_MODE_COLORIMETRY_XVYCC_601	XVYCC_601	DRM 定义
DRM_MODE_COLORIMETRY_XVYCC_709	XVYCC_709	DRM 定义
DRM_MODE_COLORIMETRY_SYCC_601	SYCC_601	DRM 定义
DRM_MODE_COLORIMETRY_OPYCC_601	opYCC_601	DRM 定义
DRM_MODE_COLORIMETRY_OPRGB	opRGB	DRM 定义
DRM_MODE_COLORIMETRY_BT2020_CYCC	BT2020_CYCC	DRM 定义
DRM_MODE_COLORIMETRY_BT2020_RGB	BT2020_RGB	DRM 定义
DRM_MODE_COLORIMETRY_BT2020_YCC	BT2020_YCC	DRM 定义
DRM_MODE_COLORIMETRY_DCI_P3_RGB_D65	P3_RGB_D65	DRM 定义
DRM_MODE_COLORIMETRY_DCI_P3_RGB_THEATER	P3_RGB_Theater	DRM 定义

目前 DRM HDMI 驱动只区分 BT2020 的几个色域，如果应用没指定 BT2020 色域，驱动会根据不同的分辨率指定 HDMI 输出的色域。

- range 使用 RK 私有定义：hdmi_quant_range

配置值	range	说明
HDMI_QUANTIZATION_RANGE_DEFAULT	Default	RK 私有定义
HDMI_QUANTIZATION_RANGE_LIMITED	Limit	RK 私有定义
HDMI_QUANTIZATION_RANGE_FULL	Full	RK 私有定义

3. Linux 6.1 内核版本

- 色域和 Linux 5.10 内核版本一样，使用 DRM 定义的标准属性：Colorspace；
- range 使用 RK 私有属性，属性名字从 “hdmi_quant_range“ 改成 ”quant_range“，主要是为了和 DP 复用：

配置值	range	说明
HDMI_QUANTIZATION_RANGE_DEFAULT	Default	RK 私有定义
HDMI_QUANTIZATION_RANGE_LIMITED	Limit	RK 私有定义
HDMI_QUANTIZATION_RANGE_FULL	Full	RK 私有定义

3.2.5.3 其他说明

1. DP 的配置和 HDMI 基本一致，部分功能可能还缺失，后续会继续完善；
2. 不同的内核版本可以使用 Libdrm 标准接口： drmGetVersion() 判断：

内核版本	版本号
Linux 4.4	1.0
Linux 4.19	2.0
Linux 5.10	3.0
Linux 6.1	4.0

3. 目前 DRM 框架定义的 Plane 属性还只考虑 YUV 格式的数据， 对 RGB 格式的色域未加定义，所以对于 RGB 格式我们先借用 YUV 的色域定义，即：

   普通 UI (SRGB + Full Range) 配置属性：COLOR_ENCODING=DRM_COLOR_YCBCR_BT709，COLOR_RANGE=DRM_COLOR_YCBCR_FULL_RANGE；

4. Display feature

4.1 各平台 VOP 基础特性

SOC	VOP-version	VOP base feature										
		8K	4K	MMU	i-MODE	A/I FBDC	MULTI AREA	BCSH	gamma	3D-LUT	POST-SCALE	X-mirror
RK3066/PX2	V1.0	×	×	×	×	×	×	×	√	×	×	×
RK3188/PX3	V1.0	×	×	×	×	×	×	×	√	×	×	×
RK3126/RK3126C	V1.0	×	×	√	×	×	×	√	√	×	×	×
RK3128/PX3SE	V1.0	×	×	√	√	×	×	√	√	×	×	×
RK3036	V1.0	×	×	√	√	×	×	√	√	×	×	×
RK322X/RK312XH	V1.0	×	√	√	√	×	×	√	×	×	√	×
RK322XH/RK332X	V1.0	×	√	√	√	×	×	√	×	×	√	×
SOFIA 3GR	V1.0	×	×	√	×	×	×	√	√	×	×	×
RV1108	V1.0	×	×	×	√	×	×	√	√	×	×	×
RK3288	V1.0	×	√	√	×	×	√	√	√	×	√	×
RK3368/PX5	V1.0	×	√	√	×	√	√	√	√	×	√	×
RK3399	V1.0	×	√	√	√	√	√	√	√	×	√	×
RK3326/PX30	V1.0	×	×	√	√	√	√	√	√	×	×	×
RK3308	V1.0	×	×	×	×	×	×	√	√	×	×	×
RK1808	V1.0	×	×	√	×	×	×	√	√	×	×	×
RV1109/RV1126	V1.0	×	×	√	×	×	×	√	√	×	×	×
RV1103/RV1106	V1.0	×	×	×	√	×	×	√	√	×	×	×
RK356X	V2.0	×	√	√	√	√	√	√	√	√	√	√
RK3588	V2.0	√	√	√	√	√	√	√	√	√	×	√
RK3528	V3.0	×	√	√	√	√	√	√	√	√	√	√

RK3562	V3.0	×	×	√	√	×	√	√	√	√	√	√
RK3576	V3.0	×	√	√	√	√	√	√	√	√	√	√
RK3506	V1.0	×	×	×	√	×	×	√	√	×	×	×
RV1126B	V1.0	×	×	√	√	×	×	√	√	×	×	×

4.2 各平台显示接口最大输出分辨率和协议标准

平台	显示接口	最大输出分辨率	协议标准
RK3036	HDMI	1920x1080@60hz	支持 HDMI 1.4a 协议标准
	CVBS	720x480i@60hz/720x576i@50hz	支持 NTSC/PAL 标准输出
RK312X/PX3SE	RGB	1280x800@60hz	支持 RGB666/sRGB888
	LVDS	1280x800@60hz	支持 VESA 和 JEIDA LVDS 数据格式
	MIPI	1920x1080@60hz	支持 DSI v1.0，DCS v1.0，DPHY v1.0 协议标准
	HDMI	1920x1080@60hz	支持 HDMI 1.4a 协议标准
	CVBS	720x480i@60hz/720x576i@50hz	支持 NTSC/PAL 标准输出
RK322X/RK312XH	HDMI	4096x2160@60hz	支持 HDMI 1.4a 和 2.0 协议标准
	CVBS	720x480i@60hz/720x576i@50hz	支持 NTSC/PAL 标准输出
RK3288	RGB	1920x1080@60hz	支持 GB888/RGB666/sRGB888
	LVDS	单通道：1280x800@60hz 双通道：1920x1080@60hz	支持 VESA 和 JEIDA LVDS 数据格式
	eDP	2560x1600@60hz	支持 DP1.2a 和 eDP1.3 协议标准
	MIPI	单通道：1920x1080@60hz 双通道：2560x1600@60hz	支持 DSI v1.1，DCS v1.1，DPHY v1.1 协议标准
	HDMI	VOP BIG: 3840x2160@60hz VOP LIT: 1920x1080@60hz	支持 HDMI 1.4a 和 2.0 协议标准
RK3308	RGB	1280x800@60hz	支持 RGB888/RGB666/sRGB888/MCU
RK1808	RGB	1280x800@60hz	支持 RGB888/RGB666/sRGB888/MCU
	MIPI	1280X800@60hz	支持 DSI v1.0，DCS v1.0，DPHY v1.0 协议标准
RK322XH/RK332X	HDMI	3840x2160@60hz	支持 HDMI 1.4a 和 2.0 协议标准
	CVBS	720x480i@60hz/720x576i@50hz	支持 NTSC/PAL 标准输出
RK3326/px30	RGB	1280x800@60hz	支持 RGB888/RGB666/sRGB888/MCU
	LVDS	1280x800@60hz	支持 VESA 和 JEIDA LVDS 数据格式
	MIPI	1920x1080@60hz	支持 DSI v1.0，DCS v1.0，DPHY v1.0 协议标准
RK3368/PX5	RGB	1280x800@60hz	支持 RGB888/RGB666/sRGB888/MCU
	LVDS	1280x800@60hz	支持 VESA 和 JEIDA LVDS 数据格式
	MIPI	1920x1080@60hz	支持 DSI v1.0，DCS v1.0， DPHY v1.0 协议标准
	eDP	2560x1600@60hz	支持 DP1.2a 和 eDP1.3 协议标准
	HDMI	4096x2160@60hz	支持 HDMI 1.4a 和 2.0 协议标准

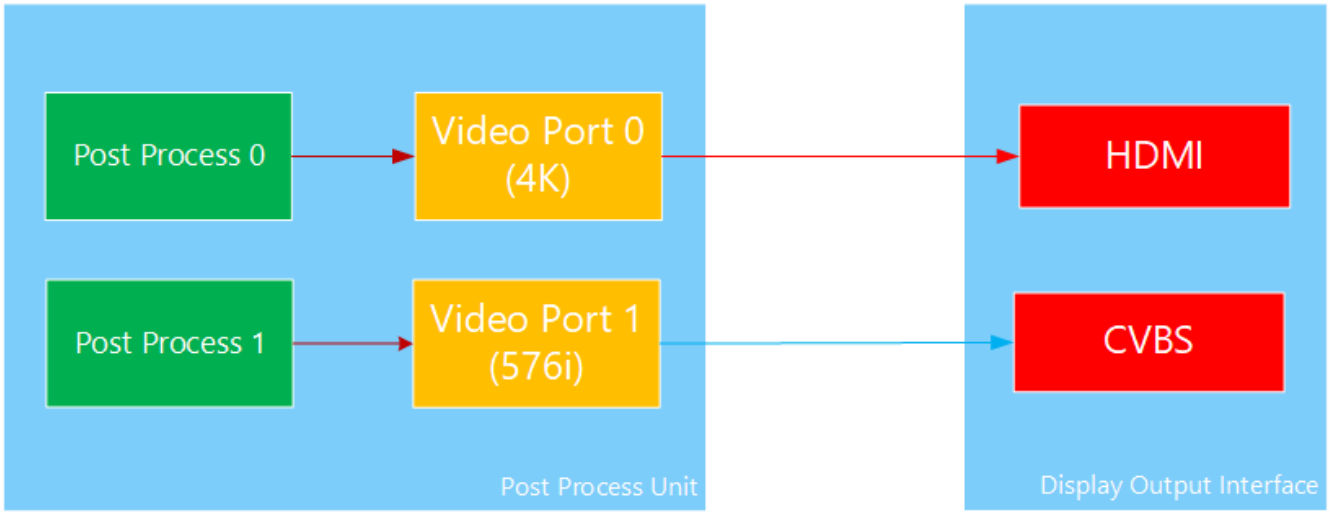
RK3399	MIPI	单通道：1280x800@60hz 双通道：2560x1600@60hz	支持 DSI v1.1，DCS v1.1，DPHY v1.1 协议标准准
	eDP	2560x1600@60hz	支持 DP1.2a 和 eDP1.3 协议标准
	HDMI	VOP BIG: 4096x2160@60hz VOP LIT: 2560x1600@60hz	支持 HDMI 1.4a 和 2.0a 协议标准
	DP	VOP BIG: 3840x2160@60hz VOP LIT: 2560x1600@60hz	支持 DP 1.2 协议标准
RV1109/RV1126	RGB	1920x1080@60hz	支持 RGB888/RGB666/sRGB888/MCU/BT.1120
	MIPI	1920x1080@60hz	支持 DSI v1.1，DCS v1.1， DPHY v1.2 协议标准
RK356X	RGB	1920x1080@60hz	支持 RGB888/RGB666/BT.656/BT.1120
	MIPI	单通道：1920x1080@60hz 双通道：2560x1600@60hz	支持 DSI v1.1，DCS v1.1，DPHY v1.1 协议标准准
	eDP	2560x1600@60hz	支持 DP 1.2a 和 eDP 1.3 协议标准
	HDMI	4096x2160@60hz	支持 HDMI 1.4a 和 2.0a 协议标准
RK3588	RGB	1920x1080@60hz	支持 BT.656/BT.1120
	MIPI	3840x2160@60hz	双 MIPI，支持 DSI v1.1，DCS v1.1， DPHY v2.0，CPHY V1.1 协议标准
	eDP[0]	3840x2160@60hz	双 eDP，支持 DP1.2a 和 eDP1.3 协议标准
	HDMI	7680x4320@60hz	双 HDMI，支持 HDMI 2.1 协议标准
	DP	7680x4320@30hz	双 DP，支持 DP1.4 协议标准
RV1103/RV1106	RGB	1280x720@60hz	支持 RGB666/RGB565/sRGB888/MCU/BT.656/BT.1120
RK3528	HDMI	4096x2160@60hz	支持 HDMI 2.0 协议标准
	CVBS	720x480i@60hz/720x576i@50hz	支持 NTSC/PAL 标准输出
RK3562	RGB	1920x1080@60hz	支持 RGB888/RGB666/RGB565/sRGB888/MCU/BT.656/BT.1120
	MIPI	2048x1080@60hz	支持 DSI v1.1，DCS v1.1， DPHY v1.2 协议标准
	LVDS	1280x800@60hz	支持 VESA 和 JEIDA LVDS 数据格式
RK3576	HDMI	4096x2160@120hz	支持 HDMI 2.1 协议标准
	DP	4096x2160@120hz	支持 DP1.4 协议标准
	MIPI	2560x1600@60hz	支持 DSI v1.1，DCS v1.4， DPHY v2.0 协议标准
	eDP	4096x2160@60hz	支持 DP1.2 和 eDP1.3 协议标准
	RGB	1920x1080@60hz	支持 RGB888/RGB666/RGB565/sRGB888/MCU/BT.656/BT.1120
RK3506	RGB	1280x720@60hz	支持 RGB888/RGB666/RGB565/sRGB888/MCU/BT.656/BT.1120
	MIPI	1280x720@60hz	支持 DSI v1.1，DCS v1.1， DPHY v1.2 协议标准
RV1126B	RGB	1920x1080@60hz	支持 RGB888/RGB666/RGB565/sRGB888/MCU/BT.656/BT.1120
	MIPI	1920x1080@60hz	支持 DSI v1.1，DCS v1.1， DPHY v1.2 协议标准

Note:

1. 在 RK3576/RK3588 上，eDP 和 HDMI 的 PHY 是 combo 的，即在同一个产品上，使用了 HDMI0 就不能使用 eDP0，HDMI1 和 eDP1 同理。

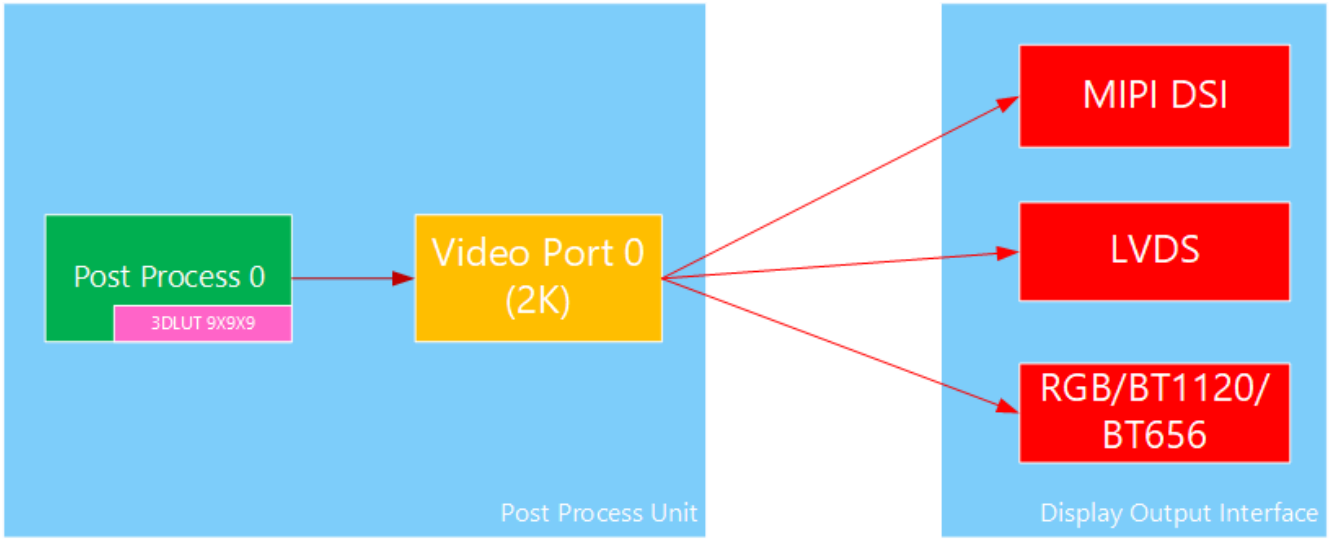
4.3 VOP2 平台显示通路

4.3.1 RK3528 VP 和各显示接口的连接关系



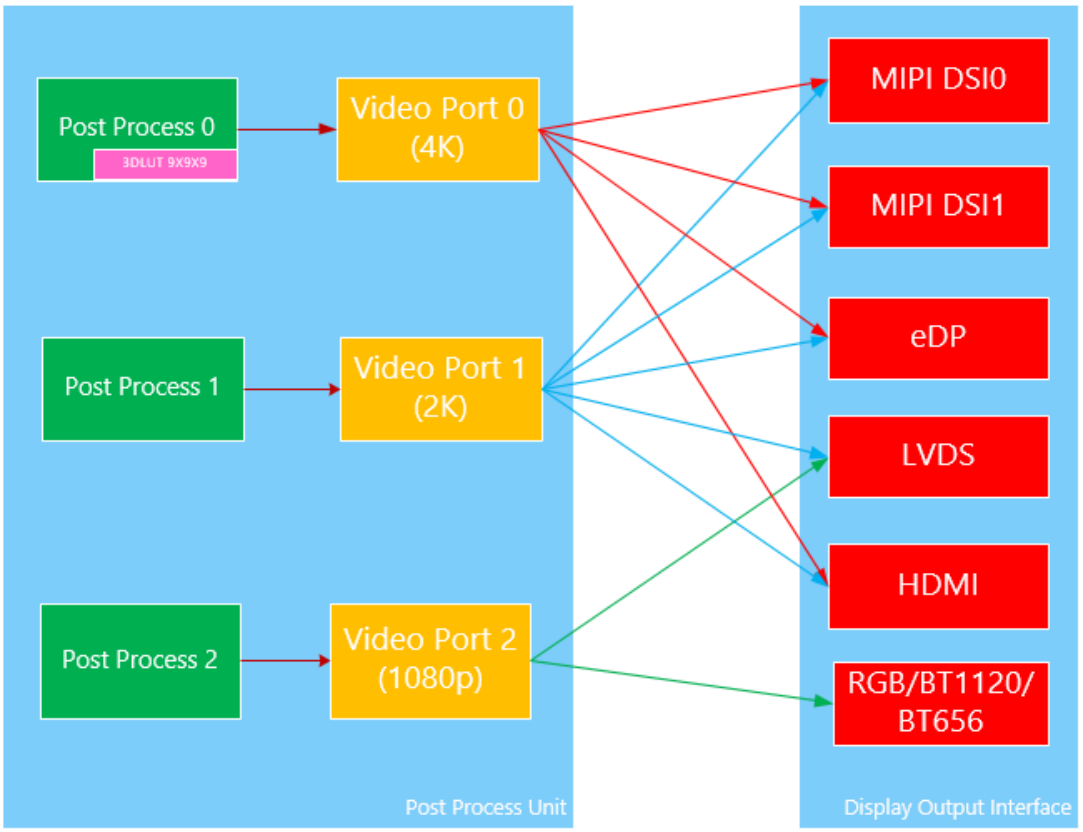
其中， VP0 的 4K 分辨率为 4096x2304 ，VP1 的 576i 分辨率为 720x576 ，更详细的内容请参考 《Rockchip RK3528 datasheet》 **Display Interface** 和 **Video Output Processor** 章节。

4.3.2 RK3562 VP 和各显示接口的连接关系



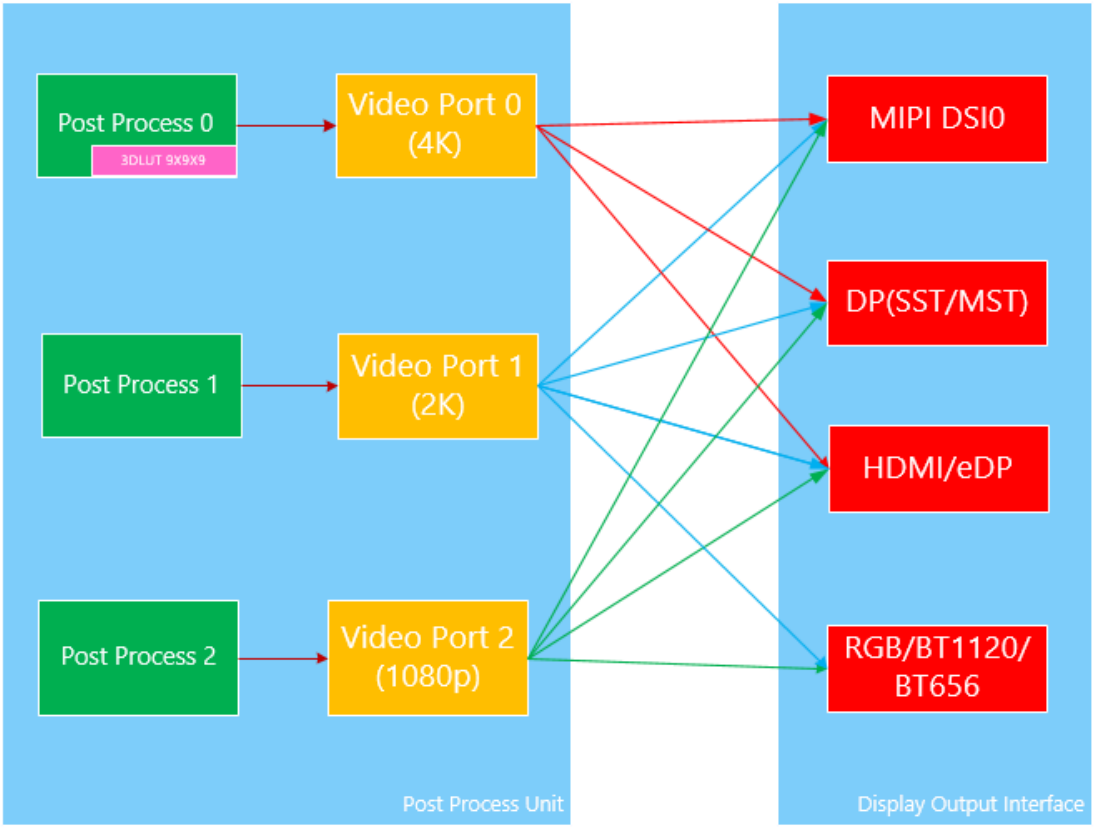
其中， VP0 的 2K 分辨率为 2048x1080 ，更详细的内容请参考 《Rockchip RK3562 datasheet》 **Display Interface** 和 **Video Output Processor** 章节。

4.3.3 RK3568 VP 和各显示接口的连接关系



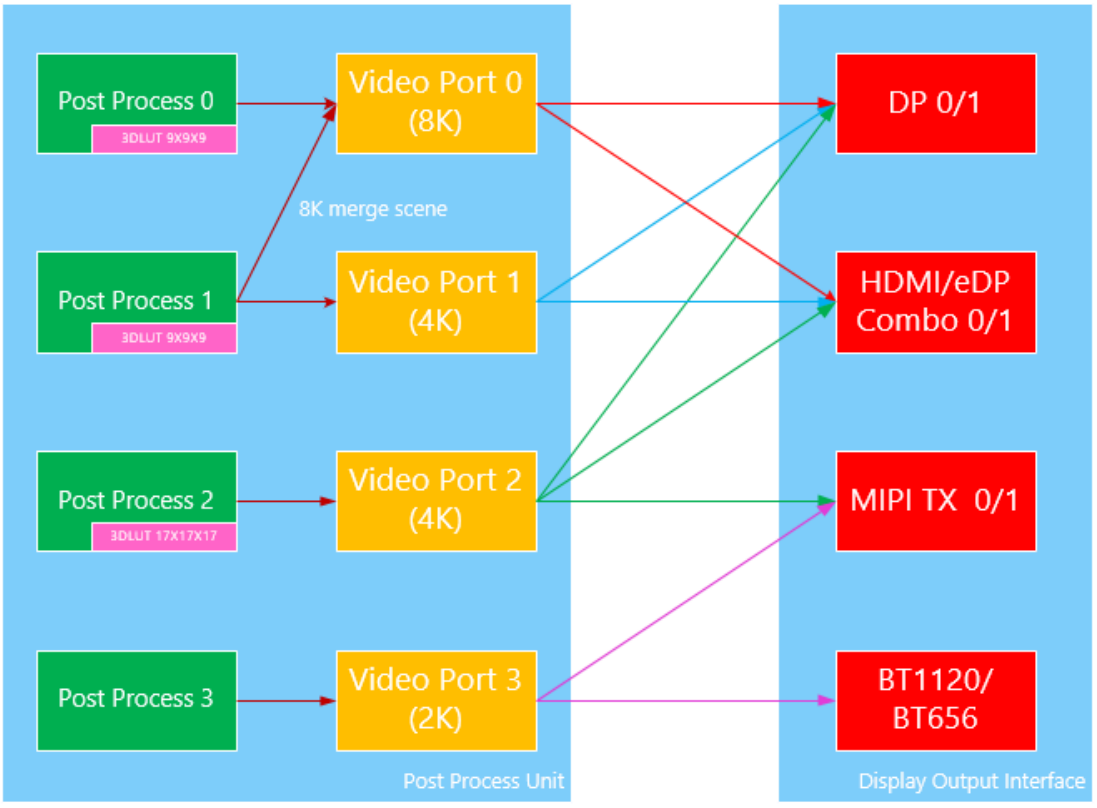
更详细的内容请参考 《Rockchip RK3568 datasheet》 **Display Interface** 和 **Video Output Processor** 章节。

4.3.4 RK3576 VP 和各显示接口的连接关系



需要注意的是，上图 VP1 的 2K 对应的分辨率是2560x1600，更详细的内容请参考 《Rockchip RK3576 datasheet》 **Display Interface** 和 **Video Output Processor** 章节。

4.3.5 RK3588 VP 和各显示接口的连接关系



需要注意的是，RK3588 的 HDMI 和 DP 支持 8K 输出，但是在 8K 输出模式下，一个显示接口需要同时占用 VP0 和 VP1 。所以如果产品上需要支持 8K 显示输出，VP1 上要注意不要连接其他显示接口。

更详细的内容请参考 《Rockchip RK3588 datasheet》 **Display Interface** 和 **Video Output Processor** 章节。

4.4 延伸芯片的显示规格

4.4.1 RK3566

相比于 RK3568， RK3566 只支持双屏同显，对屏幕的要求如下：

- 要求两个屏幕的刷新率相同
- 要求两个屏幕的显示方向一致，即都为横屏或者都为竖屏

有关 RK3566 双屏显示的使用，详见 RK3566 图层分配策略 一节。

4.4.2 RK3576s

相比于 RK3576，RK3576s 只保留了 Video Port0，RGB/BT.1120/BT.656/MCU 显示接口不可用。

4.4.3 RK3588s

相比于 RK3588，RK3588s 没有 HDMI1，DP1，eDP1 显示接口。

4.5 VOP 平台 DCLK 时钟分配策略

4.5.1 RK3399 VOP DCLK 分配策略

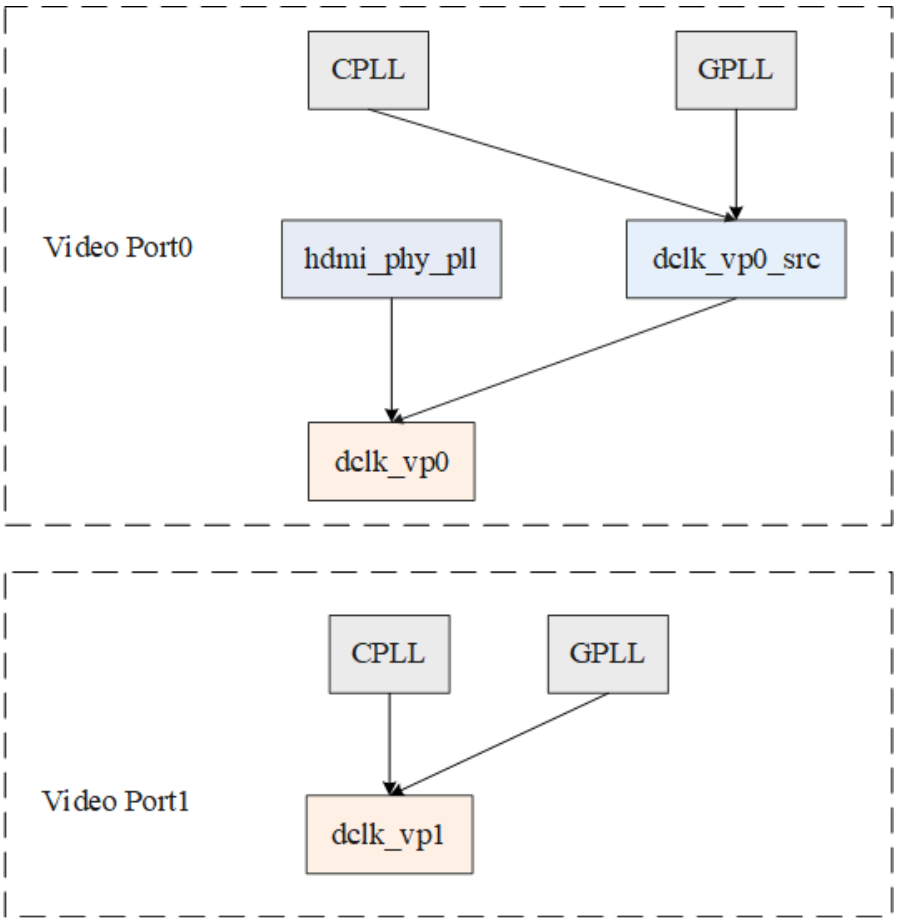
RK3399 有两个 VOP，分别是 VOPB[VOP0] 和 VOPL[VOP1]，两个 VOP 可以使用的 PLL 有 VPLL/CPLL ，其中 VPLL 是可以用于显示专用的 PLL，特殊的一点是对于 HDMI 通路 VOP 的 DCLK 一定要从 VPLL 分出来，需要在 DTS 正确指定，如果 DTS 配置错误，驱动会把 HDMI 那一路切到 VPLL，并做一次分辨率切换流程，从现象上看就是走了显示关闭再打开的流程， DTS 参考配置：

```
&hdmi_in_vopl {
    status = "disabled";
};

&hdmi_in_vopb {
    status = "okay";
};

&route_hdmi {
    connect = <&vopb_out_hdmi>;
};

&vopb {
    status = "okay";
    assigned-clocks = <&cru DCLK_VOP0_DIV>;
    /* The dclk parent for VOP->HDMI must from VPLL */
    assigned-clock-parents = <&cru PLL_VPLL>;
};

&vopl {
    status = "okay";
    assigned-clocks = <&cru DCLK_VOP1_DIV>;
    assigned-clock-parents = <&cru PLL_CPLL>;
};
```

4.5.2 RK3528 VOP DCLK 分配策略

4.5.2.1 RK3528 DCLK 概述

RK3528 有 2 个 Video Port，Video Port 的 dclk tree 如下图所示：



对于 Video Port0 上的 dclk，可以指定 hdmi_phy_pll 或者 dclk_vp0_src 作为其 parent clock。对于 dclk_vp0_src，可以指定 CPLL， GPLL 中的一个作为其 parent clock。

对于 Video Port1 上的 dclk， 可以指定 CPLL，GPLL 中的一个作为其 parent clock。

hdmi_phy_pll 的特点如下：

- 支持任意分频

CPLL，GPLL 具有以下特点：

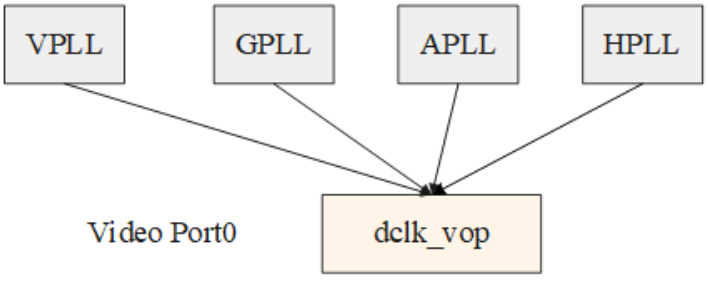
- 与其他 IP 模块共享 PLL
- 不支持任意分频，输出频率为 PLL 频率的整数分频

默认情况下，HDMI 使用 hdmi_phy_pll 作为 dclk 的 parent clock，可以满足各种频率要求。 CVBS 只有固定的几个输出分辨率，所需要的只有固定的几个 dclk 频点，使用 CPLL 或 GPLL 作为 parent clock 可以满足这几种输出分辨率的频率要求。

4.5.3 RK3562 VOP DCLK 分配策略

4.5.3.1 RK3562 DCLK 概述

RK3562 只有 1 个 Video Port，Video Port 的 dclk tree 如下图所示：



dclk 可以指定 VPLL，GPLL，APLL 中的一个作为其 parent clock。HPLL 当前分配给 audio 进行独占使用， VOP 不可使用。

VPLL 具有以下特点：

- 可以分配给 VOP 独占使用
- 支持输出任意频率
- VPLL 为 VP0 的默认 parent clock

GPLL，APLL 具有以下特点：

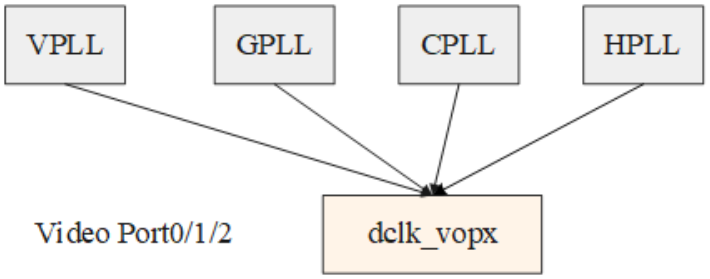
- 与其他 IP 模块共享 PLL
- 不支持任意分频，输出频率为 PLL 频率的整数分频

默认情况下，使用 VPLL 作为 VP0 的 parent clock 可以满足各种频率要求。

4.5.4 RK3568 VOP DCLK 分配策略

4.5.4.1 RK3568 DCLK 概述

RK3568 有 3 个 Video Port, 每个 Video Port 的 dclk tree 如下图所示：



对于所有 Video Port 上的 dclk ，均可以指定 VPLL，GPLL，CPLL，HPLL 中的一个作为其 parent clock。

VPLL、HPLL 具有以下特点：

- 可以分配给 VOP 独占使用
- 支持输出任意频率
- HPLL 在 DTS 中默认分配给 VP0， VPLL 在 DTS 中默认分配给 VP1：

```
assigned-clocks = <&cru DCLK_VOP0>, <&cru DCLK_VOP1>, <&cru DCLK_VOP2>;
assigned-clock-parents = <&pmucru PLL_HPLL>, <&cru PLL_VPLL>, <&cru PLL_GPLL>;
```

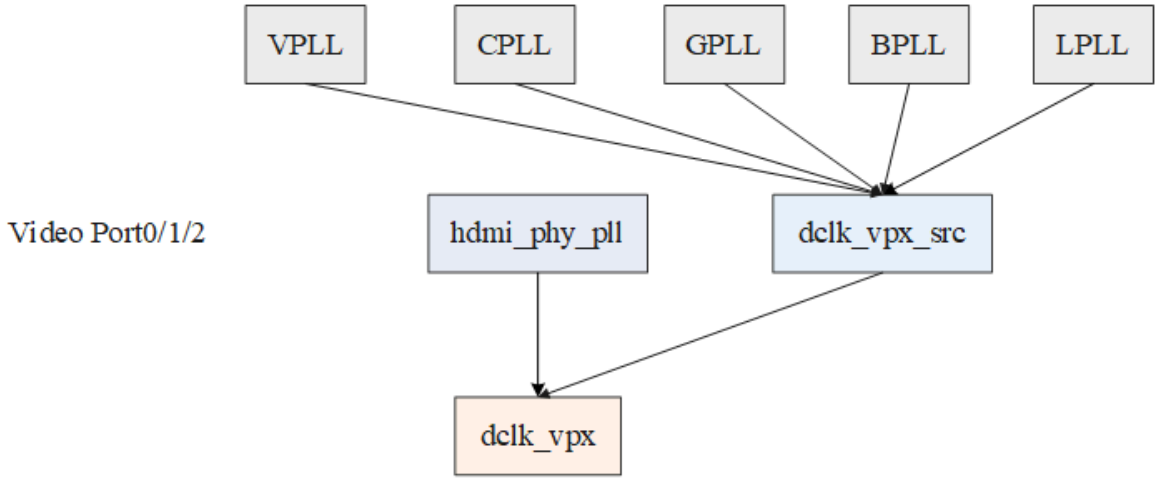
GPLL，CPLL 具有以下特点：

- 与其他 IP 模块共享 PLL
- 不支持任意分频，输出频率为 PLL 频率的整数分频

4.5.5 RK3576 VOP DCLK 分配策略

4.5.5.1 RK3576 DCLK 概述

RK3576 有 3 个 Video Port, 每个 Video Port 的 dclk tree 如下图所示：



对于所有 Video Port 上的 dclk ，可以指定 hdmi_phy_pll 或者 dclk_vpx_src0/1/2 作为其 parent clock。 对于 dclk_vpx_src0/1/2，可以指定 VPLL， CPLL， GPLL， BPLL， LPLL 中的一个作为其 parent clock。

其中，VPLL，CPLL，GPLL，BPLL，LPLL 为系统 CRU 上的 PLL，hdmi_phy_pll 为 HDMI PHY 上的 PLL。

VPLL 具有以下特点：

- 该 PLL 可以分配给 VOP 独占使用
- 支持输出任意频率
- 在 DTS 中默认分配给 VP2：

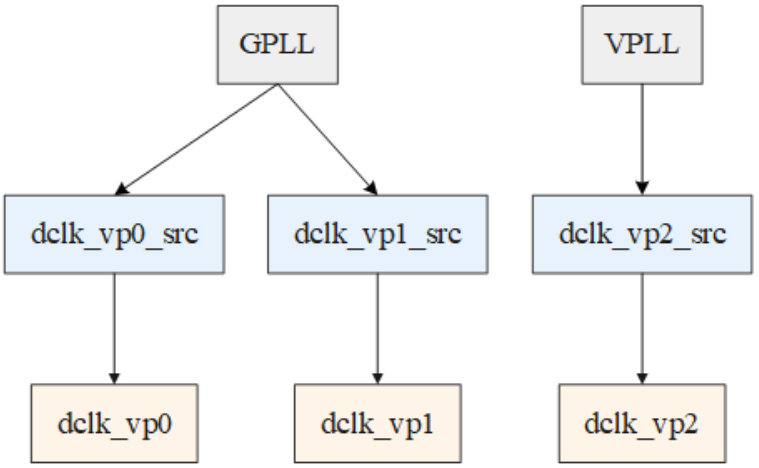
```
&vp2 {
    assigned-clocks = <&cru DCLK_VP2_SRC>;
    assigned-clock-parents = <&cru PLL_VPLL>;
};
```

CPLL，GPLL，BPLL，LPLL 具有以下特点：

- 与其他 IP 模块共享 PLL
- 不支持任意分频，输出频率为 PLL 频率的整数分频
- dclk_vp0/1/ 默认与 GPLL 绑定

其中， BPLL 与 LPLL 较少使用。BPLL 默认供 CPU 使用，若 VOP 需要使用，需要把 CPU 的 PLL 切换到 PVTPLL，此外， BPLL 分频只能取到到 2、3、4、6、8 的倍数。

各个 Video Port 的默认 parent clock 的绑定关系如下：



hdmi_phy_pll 的特点如下：

1. 支持任意分频
2. HDMI 不工作时 VOP 独占 PLL，HDMI 工作时与 HDMI PHY 共享 PLL
3. HDMI 和 EDP 共用 PHY，EDP 工作时，PHY 上的 PLL 无法被 VOP 使用

其中，特点 2 和 3 导致使用时有一些限制，举例说明如下：

1. 当 HDMI 工作时，且 Video Port0 绑定到 HDMI，各 Video Port 可以使用 hdmi_phy_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0_DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy_pll	DCLK <= 600MHz, 可用; DCLK > 600MHz, 不可用	不可用	不可用

2. 当 EDP 工作时，且 Video Port0 绑定到 EDP，各 Video Port 可以使用 hdmi_phy_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0_DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy_pll	不可用	不可用	不可用

由于 CPLL，GPLL，BPLL，LPLL 作为 Video Port dclk 源时，无法实现任意分频。当需要准确分频时，且满足使用的限制条件时，可以使用 hdmi_phy_pll 作为 Video Port dclk 的 parent clock。hdmi_phy_pll 的使用策略目前有三种：

1. 静态分配策略。即 hdmi_phy_pll 和 Video Port 绑定，Video Port 固定使用 hdmi phy pll 作为 dclk 源;
2. 和 HDMI 接口绑定的动态分配策略。即 hdmi_phy_pll 和 HDMI 接口绑定，hdmi_phy_pll 只能分配给使用 HDMI 接口的 Video Port, 这个 Video Port 可以动态选择 dclk 源用 CRU PLL 还是 hdmi phy pll。
3. 动态分配策略。hdmi phy pll 开放给所有可用的 VP 和 接口。 这种动态分配的策略最为复杂，需要应用考虑接口和 Video Port 使能是否会导致hdmi phy pll 使用冲突。目前只提供给 NVR 产品使用。

一般产品上使用的 dclk 源的分配方案，前两种即可满足需求。

4.5.5.2 RK3576 VOP DCLK 静态分配策略

静态分配策略有如下限制：

1. 一般情况下，一个 hdmi_phy_pll 只能指定给一个 Video Port(包括 VP0, VP1, VP2);
2. HDMI 接口工作时， 对应 hdmi_phy_pll 只能指定给这个 HDMI 接口对应的 Video Port, 或者不被用做 dclk 源；
3. HDMI 接口工作时，对应 hdmi_phy_pll 指定给这个 HDMI 接口对应的 Video Port 时，输出的分辨率的 pixel clock 不能超过 600MHz;
4. EDP 接口(和 HDMI 接口共用 phy)工作时, 对应的 hdmi_phy_pll 无法被作为 Video Port 的 dclk 源使用；

静态分配即通过在 dtsi 中绑定 Video Port dclk parent 的方式, 在 dtsi 中配置好绑定关系后，就不再发生变更。

```
&hdptxphy_hdmi {
    status = "okay";
};

&vp0 {
    assigned-clocks = <&cru DCLK_VP0>;
    assigned-clock-parents = <&hdptxphy_hdmi>;
};
```

如上，即绑定 dclk_vp0 的 parent clock 为 hdmi_phy_pll 。

通过 clock tree，可以确认配置是否生效，如下：

```
rk3588_s:/ # cat /sys/kernel/debug/clk/clk_summary | grep "hdmiphy" -A 4
       clk_hdmiphy_pixel0            1        2        0        132000000   0          0     50000      ?
  27da0000.hdmi                   link_clk

  display-subsystem               hdmi0_phy_pll

  hdmi@27da0000                   no_connection_id

  deviceless                      no_connection_id
         dclk_vp0                    1        3        0        132000000   0          0     50000      Y
     27d00000.vop                    dclk_vp0
```

如上，parent clock 绑定生效，并且当前 dclk_vp0 设置的 clock rate 为 132MHz，这里的 clock rate 与 log 打印中获取的 clock rate 是一样的。

如下 log 中的最后一行`set dclk_vop0 to 132000000, get 132000000`即设置 132MHz, 实际输出的也是 132MHz, 与从 clk tree 上获取的相同。

```
[    5.598827][  T331] rockchip-vop2 27d00000.vop: [drm:vop2_crtc_atomic_enable] Update mode to 1080x1920p60,
type: 16(if:MIPI0, flag:0x0) for vp0 dclk: 132000000
[    5.599012][  T331] rockchip-hdptx-phy-hdmi 2b000000.hdmiphy: hdptx_ropll_cmn_config bus_width:0
rate:742500
[    5.599185][  T331] rockchip-hdptx-phy-hdmi 2b000000.hdmiphy: hdptx phy pll locked!
[    5.599237][  T331] rockchip-hdptx-phy-hdmi 2b000000.hdmiphy: hdptx_ropll_cmn_config bus_width:0
rate:1320000
[    5.599415][  T331] rockchip-hdptx-phy-hdmi 2b000000.hdmiphy: hdptx phy pll locked!
[    5.599423][  T331] rockchip-vop2 27d00000.vop: [drm:vop2_crtc_atomic_enable] set dclk_vp0 to 132000000,
get 132000000
```

4.5.5.3 RK3576 VOP DCLK 和 HDMI 接口绑定的动态分配策略

hdmi_phy_pll 只能分配给使用 HDMI 接口的 Video Port, 这个 Video Port 可以根据 HDMI 接口的工作模式，dclk 源在 CRU PLL 和 hdmi phy pll 之间动态切换。对应的配置如下：

```
&hdptxphy_hdmi {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi>;
    clock-names = "hdmi0_phy_pll";
};
```

对于上述配置，如果 HDMI0 绑定 VP0，则对应 dclk 源会根据分辨率的 pixel clock 切换，如下表：

	pixel clock > 600MHz	**pixel clock < 600MHz**
VP0-->HDMI0	CRU PLL	hdmi0 phy pll

4.5.5.4 RK3576 VOP DCLK 动态分配策略

要使用动态绑定策略，需要在 dtsi 中进行如下配置：

```
&hdptxphy_hdmi {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi>;
    clock-names = "hdmi0_phy_pll";
};

&vop {
    rockchip,extend-phy-pll-shared-mode;
};
```

从 dts 配置可以看出，动态分配测试和第二个策略相比，只是在 `vop` 节点下多配置了一个 `rockchip,extend-phy-pll-shared-mode` 属性。

由于可任意分频的 PLL 源有限，并且 HDMI PHY 上 的 PLL 使用还有一定的限制，因此动态分配的策略比较复杂，同时，动态分配的策略对业务场景有一定的限制。

每次使能一个 Video Port 时，都会按照如下的策略进行动态的 Video Port dclk 的绑定：

1. HDMI (可能也包含 DP 或 MIPI 挂在同一个 Video Port 下) 接到一个 Video Port(0/1/2)，下面称 VP_A，并且 dclk 在 600MHz 以下:
    - 配置了 hdmi_phy_pll 且 未被其他 Video Port 使用时， VP_A 使用 hdmi_phy0_pll;
    - 配置了 hdmi_phy_pll 且 被其他 Video Port 使用时，报错;
2. 只有 DP 接到一个 Video Port(0/1/2) 时:
    - Video Port 为 Video Port2 时, 使用默认配置的 dclk parent clock，即 VPLL;
    - hdmi_phy_pll 未被其他 Video Port 使用时，优先使用 hdmi_phy_pll ，否则使用默认配置的 dclk parent clock;

如果要求的 dclk 大于 4k@60Hz, 使用默认的 pll。

根据动态配置策略，为尽可能多的利用 hdmi_phy0_pll, 实际使用时有如下建议：

- HDMI 对应的 VP 尽量不要将 parent clock 设置为 VPLL 。

对于 RK3576 的一些应用场景，可以给出如下的建议 Video Port 和 显示接口的配置：

1 个 HDMI 接口和 1 个 DP 接口输出时：

Video Port	**接口配置1**	**接口配置2**
Video Port0	HDMI	其他显示接口
Video Port1	其他显示接口	HDMI
Video Port2	DP	DP

在这种配置下，DP 在 VP2 上使用 VPLL ，HDMI 在 VP0 上使用 hdmi_phy_pll ，两个接口都可以得到精确的频率。

如果 DP 的 parent clock 不是 VPLL，那么尝试获取 hdmi_phy_pll ，若此时 hdmi_phy_pll 已经被 HDMI 占用，会出现以下的提示：

```
[   71.673928][  T397] rockchip-vop2 27d00000.vop: [drm:vop2_crtc_atomic_enable] Update mode to 2560x1440p60,
type: 1lag:0x0) for vp1 dclk: 241500000
[   71.674040][  T397] [drm] No free hdmi phy pll for DP, use default parent
[   71.674088][  T397] rockchip-vop2 27d00000.vop: [drm:vop2_crtc_atomic_enable] set dclk_vp1 to 241500000,
get 23760
```

在这种情况下，DP 将使用当前默认的 clock parent，并且可能无法获取到准确的时钟频率。

4.5.5.5 RK3576 VOP DCLK 分配策略选用

上述的三种策略，主要是为了满足 HDMI 和 DP 接口可以任意分频的需求，大部分应用场景 HDMI 接口动态分配策略，即可以满足要求。下面列举部分场景的 DCLK 分配策略。

(1) 三个 Port 输出， DP 和 HDMI 要求满足任意分频

Video Port	Interface
VP0	HDMI
VP1	DP
VP2	DSI

使用和 HDMI 接口绑定的动态切换策略，HDMI 接口 pixe clock 不超过 600MHz 时对应的 VP0 dclk 源选用 hdmi phy pll, 满足任意分频，pixel clock 大于等于 600MHz 时，使用 CRU PLL，整数分频，过滤部分不支持的分辨率。DP 接口对应的 VP1 dclk 源使用 VPLL, 满足任意分频需求。

dts 配置如下：

```
&hdptxphy_hdmi {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi>;
    clock-names = "hdmi0_phy_pll";
};

&vp1 {
    assigned-clocks = <&cru DCLK_VP1_SRC>;
    assigned-clock-parents = <&cru PLL_VPLL>;
};
```

(2)双4K 配置场景, DP 接口为主屏, 这种场景支持 VP0 输出到 DP + HDMI , 或 VP0 输出到 DP， VP1 输出到 HDMI，并且 VP0 和 VP1 输出的 pixel clock 都在 600MHz 一下，这两种场景切换，并且都需要支持任意分频，即：

Video Port	Interface
VP0	DP + HDMI

或

Video Port	Interface
VP0	DP
VP1	HDMI

这种场景下 ，由于 HDMI 接口可能会切换 VP，使用静态分配策略，VP0 dclk 源固定使用 VPLL, VP1 dclk 源固定使用 hdm phy pll, dts 配置如下：

```
&hdptxphy_hdmi {
    status = "okay";
};

&vp0 {
    assigned-clocks = <&cru DCLK_VP0_SRC>;
    assigned-clock-parents = <&cru PLL_VPLL>;
};

&vp1 {
    assigned-clocks = <&cru DCLK_VP1>;
    assigned-clock-parents = <&hdptxphy_hdmi>;
};
```

(3) 接口任意切换 VP 并满足不同接口任意分频需求的场景。即不同的接口可能就在不同的 VP 间切换使用，这属于比较复杂的使用场景(目前仅 NVR 场景使用)。如下 HDMI/DP/DSI 均可切换使用 VP0/1/2 的这种场景：

Video Port	Interface
VP0, VP1, VP2	HDMI
VP0, VP1, VP2	DP
VP0, VP1, VP2	DSI

这种场景使用如下动态分配策略。dts 配置如下：

```
&hdptxphy_hdmi {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi>;
    clock-names = "hdmi0_phy_pll";
};

&vop {
    rockchip,extend-phy-pll-shared-mode;
};

&vp2 {
    assigned-clocks = <&cru DCLK_VP2_SRC>;
    assigned-clock-parents = <&cru PLL_VPLL>;
};
```

如上的配置，假设初始时，只有 DP 使用 VP2, HDMI 使用 VP0 时， dclk 源绑定关系如下：

route	dclk source
VP0->HDMI	hdmi phy pll
VP2->DP	VPLL

之后如果 HDMI 拔出，DP 接到 VP1, DSI 接到 VP2, 则 DP 使用 hdmi phy pll, DSI 使用 VPLL:

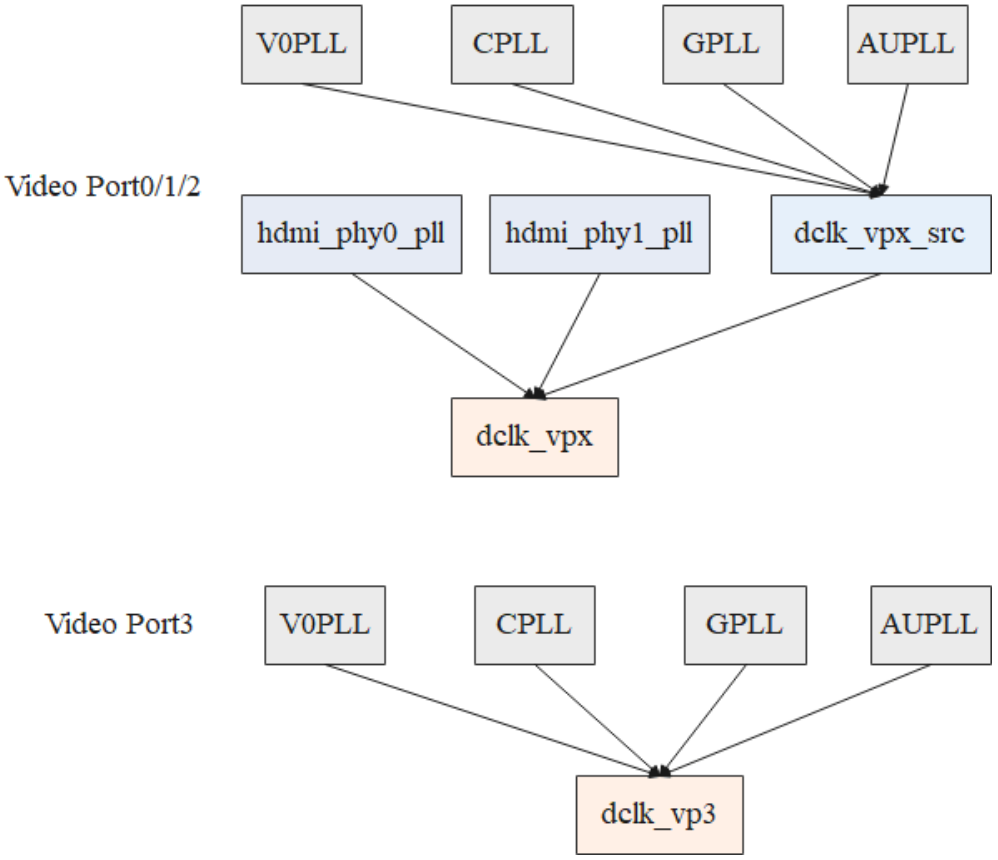
route	dclk source
VP1->DP	hdmi phy pll
VP2->DSI	VPLL

上述动态分配策略的这种场景，HDMI 接口使用 VP 时选用对应的 hdmi phy pll，需要保证 hdmi phy pll 未被其他接口使用, 非 HDMI 接口使用 VP 时，如果对应的 VP 源不是 VPLL，并且 hdmi phy pll 也未被其他接口占用，可以使用 hdmi phy pll 作为 dclk 源。使用动态分配策略时，需要应用来保证使能 HDMI 接口前，对应的 hdmi phy pll 未被其他接口占用。

4.5.6 RK3588 VOP DCLK 分配策略

4.5.6.1 RK3588 DCLK 概述

RK3588 有 4 个 Video Port, 每个 Video Port 的 dclk tree 如下图所示：



对于 dclk_vp0/1/2， 可以指定 hdmi_phy0_pll, hdmi_phy1_pll, dclk_vpx_src0/1/2 中的一个作为其 parent clock, 对 dclk_vpx_src0/1/2，可以指定

V0PLL, CPLL, GPLL, AUPLL 中的一个作为其 parent clock。

对于 dclk_vp3，可以指定 V0PLL, CPLL, GPLL, AUPLL 中的一个作为其 parent clock。

V0PLL, CPLL, GPLL, AUPLL 为系统 CRU 上的 PLL，hdmi_phy0_pll 和 hdmi_phy1_pll 为 HDMI PHY 上的 PLL

V0PLL 的特点如下：

1. 该 PLL 一般分配给 VOP 独占使用
2. 支持任意频率
3. dts 中默认与 dclk_vp2 绑定，代码如：

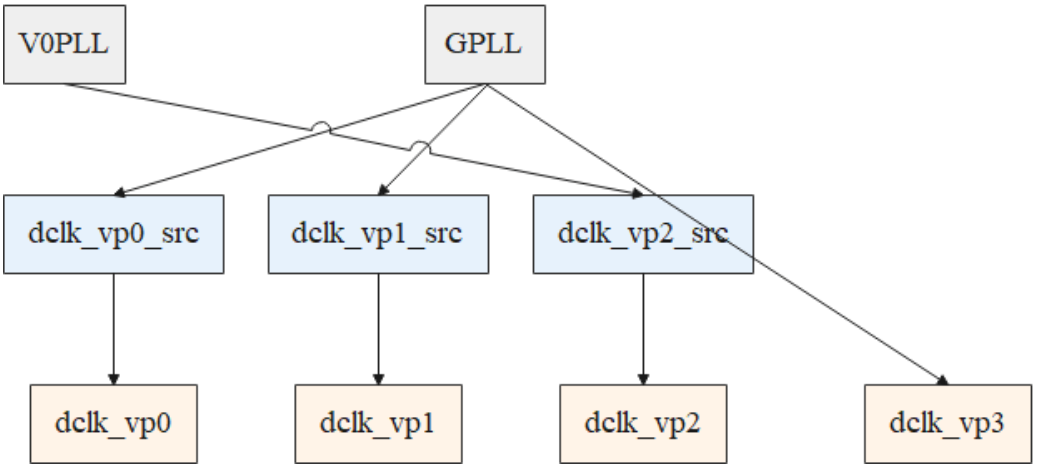
```
vp2: port@2 {
    assigned-clocks = <&cru DCLK_VOP2_SRC>;
    assigned-clock-parents = <&cru PLL_V0PLL>;
}
```

CPLL, GPLL, AUPLL 的特点如下：

1. 与其它 IP 模块共享 PLL
2. 不支持任意分频，输出频率为 PLL 频率的整数分频

3. dclk_vp0/1/3 默认与 GPLL 绑定

各个 Video Port 的默认 parent clock 的绑定关系如下：



hdmi_phy0_pll, hdmi_phy1_pll 的特点如下：

1. 支持任意分频
2. HDMI 不工作时 VOP 独占 PLL，HDMI 工作时与 HDMI PHY 共享 PLL
3. HDMI 和 EDP 共用 PHY，EDP 工作时，PHY 上的 PLL 无法被 VOP 使用

hdmi_phy0_pll, hdmi_phy1_pll 的第 2, 3 个特点导致使用时有一些限制，举例说明如下：

1. 当 HDMI0 工作时，且 Video Port0 绑定到 HDMI0，各 Video Port 可以使用 hdmi_phy0_pll 和 hdmi_phy1_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0_DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy0_pll	DCLK <= 600MHz, 可用; DCLK > 600MHz, 不可用	不可用	不可用
hdmi_phy1_pll	可用	可用	可用

2. 当 HDMI1 工作时，且 Video Port0 绑定到 HDMI1，各 Video Port 可以使用 hdmi_phy0_pll 和 hdmi_phy1_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0 DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy0_pll	可用	可用	可用
hdmi_phy1_pll	DCLK <= 600MHz, 可用 DCLK > 600MHz, 不可用	不可用	不可用

3. 当 HDMI0 和 HDMI1 同时工作时，且都绑定到 Video Port0 时， Video Port 可以使用 hdmi_phy0_pll 和 hdmi_phy1_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0_DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy0_pll	DCLK <= 600MHz, 可用; DCLK > 600MHz, 不可用	不可用	不可用
hdmi_phy1_pll	DCLK <= 600MHz, 可用; DCLK > 600MHz, 不可用	不可用	不可用

4. 当 EDP0 工作时，且 Video Port0 绑定到 EDP0，各 Video Port 可以使用 hdmi_phy0_pll 和 hdmi_phy1_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0_DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy0_pll	不可用	不可用	不可用
hdmi_phy1_pll	可用	可用	可用

5. 当 EDP1 工作时，且 Video Port0 绑定到 EDP1，各 Video Port 可以使用 hdmi_phy0_pll 和 hdmi_phy1_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0 DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy0_pll	可用	可用	可用
hdmi_phy1_pll	不可用	不可用	不可用

6. 当 EDP0 和 EDP1 同时工作时，且都绑定到 Video Port0 时， Video Port 可以使用 hdmi_phy0_pll 和 hdmi_phy1_pll 的场景(绑定到 Video Port1/2 时类似)

HDMI PHY PLL	VP0_DCLK	VP1_DCLK	VP2_DCLK
hdmi_phy0_pll	不可用	不可用	不可用
hdmi_phy1_pll	不可用	不可用	不可用

由于 CPLL，GPLL，AUPLL 作为 Video Port dclk 源时，无法实现任意分频。当需要准确分频时，且满足使用的限制条件时，可以使用 hdmi_phy0_pll 或 hdmi_phy1_pll 作为 Video Port dclk 的 parent clock。hdmi_phy_pll 的使用策略目前有三种：

1. 静态分配策略。即 hdmi_phy_pll 和 Video Port 绑定，Video Port 固定使用 hdmi phy pll 作为 dclk 源;
2. 和 HDMI 接口绑定的动态分配策略。即 hdmi_phy_pll 和 HDMI 接口绑定，hdmi_phy_pll 只能分配给使用 HDMI 接口的 Video Port, 这个 Video Port 可以动态选择 dclk 源用 CRU PLL 还是 hdmi phy pll。
3. 动态分配策略。hdmi phy pll 开放给所有可用的 VP 和 接口。 这种动态分配的策略最为复杂，需要应用考虑接口和 Video Port 使能是否会导致hdmi phy pll 使用冲突。目前只提供给 NVR 产品使用。

一般产品上使用的 dclk 源的分配方案，前两种即可满足需求。

4.5.6.2 RK3588 VOP DCLK 静态分配策略

静态分配策略有如下限制：

1. 一般情况下，一个 hdmi_phy_pll 只能指定给一个 Video Port(包括 VP0, VP1, VP2);
2. HDMI 接口工作时， 对应 hdmi_phy_pll 只能指定给这个 HDMI 接口对应的 Video Port, 或者不被用做 dclk 源；
3. HDMI 接口工作时，对应 hdmi_phy_pll 指定给这个 HDMI 接口对应的 Video Port 时，输出的分辨率的 pixel clock 不能超过 600MHz;
4. EDP 接口(和 HDMI 接口共用 phy)工作时, 对应的 hdmi_phy_pll 无法被作为 Video Port 的 dclk 源使用；

静态分配即通过在 dtsi 中绑定 Video Port dclk parent 的方式, 在 dtsi 中配置好绑定关系后，就不再发生变更。

kernel 5.10 的参考配置如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&hdptxphy_hdmi_clk0 {
    status = "okay";
};

&hdptxphy_hdmi_clk1 {
    status = "okay";
};

&vp0 {
    assigned-clocks = <&cru DCLK_VOP0>;
```

```
    assigned-clock-parents = <&hdptxphy_hdmi_clk0>;
};

&vp1 {
    assigned-clocks = <&cru DCLK_VOP1>;
    assigned-clock-parents = <&hdptxphy_hdmi_clk1>;
};
```

kernel 6.1 及以上版本的参考配置如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&vp0 {
    assigned-clocks = <&cru DCLK_VOP0>;
    assigned-clock-parents = <&hdptxphy_hdmi0>;
};

&vp1 {
    assigned-clocks = <&cru DCLK_VOP1>;
    assigned-clock-parents = <&hdptxphy_hdmi1>;
};
```

如上，即绑定 dclk_vp0 的 parent clock 为 hdmi_phy0_pll, dclk_vp1 的 parent clock 为 hdmi_phy1_pll 。

通过 clock tree，可以确认配置是否生效，如下：

```
rk3588_s:/ # cat /sys/kernel/debug/clk/clk_summary | grep "hdmiphy" -A 1
       clk_hdmiphy_pixel1             0        1        0           0          0     0  50000
          dclk_vop1                   0        3        0           0          0     0  50000
       clk_hdmiphy_pixel0             1        1        0   148500000          0     0  50000
          dclk_vop0                   2        5        0   148500000          0     0  50000
```

如上，parent clock 绑定生效，并且当前 dclk_vp0 设置的 clock rate 为 148.5MHz，这里的 clock rate 与 log 打印中获取的 clock rate 是一样的。

如下 log 中的最后一行`set dclk_vop0 to 148500000, get 148500000`即设置 148.5MHz, 实际输出的也是 148.5MHz, 与从 clk tree 上获取的相同。

```
[  270.974623][  T381] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] Update mode to 1920x1080p60,
type: 11(if:800) for vp0 dclk: 148500000
[  270.974837][  T381] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] dclk_out0 div: 0 dclk_core0
div: 2
[  270.974891][  T381] rockchip-hdptx-phy-hdmi fed60000.hdmiphy: hdptx_ropll_cmn_config bus_width:16a8c8
rate:1485000
[  270.975264][  T381] rockchip-hdptx-phy-hdmi fed60000.hdmiphy: hdptx phy pll locked!
[  270.975304][  T381] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] set dclk_vop0 to 148500000,
get 148500000
```

4.5.6.3 RK3588 VOP DCLK 和 HDMI 接口绑定的动态分配策略

hdmi_phy_pll 只能分配给使用 HDMI 接口的 Video Port, 这个 Video Port 可以根据 HDMI 接口的工作模式，dclk 源在 CRU PLL 和 hdmi phy pll 之间动态切换。对应的 kernel 5.10 配置如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
```

```
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&hdptxphy_hdmi_clk0 {
    status = "okay";
};

&hdptxphy_hdmi_clk1 {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi_clk0>, <&hdptxphy_hdmi_clk1>;
    clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};
```

对应的 kernel 6.1 及以上版本如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi0>, <&hdptxphy_hdmi1>;
    clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};
```

对于上述配置，如果 HDMI0 绑定 VP0, HDMI1 绑定 VP1, 则对应 dclk 源会根据分辨率的 pixel clock 切换，如下表：

	pixel clock > 600MHz	**pixel clock < 600MHz**
VP0-->HDMI0	CRU PLL	hdmi0 phy pll
VP1-->HDMI1	CRU PLL	hdmi1 phy pll

4.5.6.4 RK3588 VOP DCLK 动态分配策略

动态绑定的方式对应 kernel 5.10 版本如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&hdptxphy_hdmi_clk0 {
    status = "okay";
};

&hdptxphy_hdmi_clk1 {
    status = "okay";
};
```

```
&display_subsystem {
    clocks = <&hdptxphy_hdmi_clk0>, <&hdptxphy_hdmi_clk1>;
    clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};

&vop {
    rockchip,extend-phy-pll-shared-mode;
};
```

对应 kernel 6.1 及之后的版本版本如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi0>, <&hdptxphy_hdmi1>;
    clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};

&vop {
    rockchip,extend-phy-pll-shared-mode;
};
```

从 dts 配置可以看出，动态分配测试和第二个策略相比，只是在 `vop` 节点下多配置了一个 `rockchip,extend-phy-pll-shared-mode` 属性。

由于可任意分频的 PLL 源有限，并且 HDMI PHY 上 的 PLL 使用还有一定的限制，因此动态分配的策略比较复杂，同时，动态分配的策略对业务场景有一定的限制。

每次使能一个 Video Port 时，都会按照如下的策略进行动态的 Video Port dclk 的绑定：

1. HDMI0 和 HDMI1 (可能也包含 DP 或 MIPI 挂在同一个 Video Port 下) 接到同一个 Video Port(0/1/2)并且 dclk 在 600MHz 以下:

   1.1 hdmi_phy0_pll 和 hdmi_phy1_pll 均未被其他 Video Port 使用时，使用 hdmi_phy0_pll 为当前 Video Port dclk 的 parent clock , 并且 hdmi_phy0_pll 和 hdmi_phy1_pll 不可被其他的 Video Port 使用;

   1.2 hdmi_phy0_pll 或 hdmi_phy0_pll 被其他 Video Port 使用时，则报错；

2. HDMI0 (可能也包含 DP 或 MIPI 挂在同一个 Video Port 下) 接到一个 Video Port(0/1/2)，下面称 VP_A，并且 dclk 在 600MHz 以下:

   2.1 hdmi_phy0_pll 和 hdmi_phy1_pll 均存在时，且 hdmi_phy0_pll 未被其他 Video Port 使用时，VP_A 使用 hdmi_phy0_pll ，同时 hdmi_phy0_pll 不可被其他的 Video Port 使用;

   2.2 hdmi_phy0_pll 和 hdmi_phy1_pll 均存在时，且 hdmi_phy0_pll 正在被其他的 Video Port(VP_B) 使用时, hdmi_phy1_pll 空闲时，VP_B 要把 hdmi_phy0_pll 的 pll 让出来，去使用 hdmi_phy1_pll ，VP_A 使用 hdmi_phy0_pll ;

   2.3 hdmi_phy0_pll 和 hdmi_phy1_pll 均存在且均被其他的 Video Port 使用时, 报错；

   2.4 只有配置 hdmi_phy0_pll 且 未被其他 Video Port 使用时， VP_A 使用 hdmi_phy0_pll;

   2.5 只有配置 hdmi_phy0_pll 且 被其他 Video Port 使用时，报错;

3. HDMI1 (可能也包含 DP 或 MIPI 挂在同一个 Video Port 下) 接到一个 Video Port(0/1/2) 并且 dclk 在 600MHz 以下时，策略和 2 类似，不再详细描述；

4. 只有 DP接到一个 Video Port(0/1/2) 时:

   4.1 Video Port 为 Video Port2 时, 使用默认配置的 dclk parent clock;

   4.2 hdmi_phy0_pll 或 hdmi_phy1_pll 未被其他 Video Port 使用时，优先使用 hdmi_phy0_pll 或 hdmi phy1_pll，否则使用默认配置的 dclk parent clock;

如果要求的 dclk 大于 4k@60Hz, 使用默认的 pll。

根据动态配置策略，为尽可能多的利用 hdmi_phy0_pll 和 hdmi_phy1_pll , 实际使用时有如下建议：

1. HDMI0 和 HDMI1 尽量不要挂在 Video Port2 上；
2. HDMI0 和 HDMI1 尽量不要挂在同一个Video Port上。

对于一些应用场景，可以给出如下的建议 Video Port 和 显示接口的配置：

2 个 HDMI 接口和 1 个 DP 接口输出时：

Video Port	接口配置1	接口配置2
Video Port0	HDMI0	HDMI1
Video Port1	HDMI1	HDMI0
Video Port2	DP0/DP1	DP0/DP1

2 个 DP 接口和 1 个 HDMI 接口输出时:

Video Port	接口配置1	搭配方案2	接口配置1	接口配置2
Video Port0	HDMI0/1	HDMI0/1	DP0	DP1
Video Port1	DP0	DP1	HDMI0/1	HDMI0/1
Video Port2	DP1	DP0	DP1	DP0

更多的应用场景，需要参考动态分配策略进行合理的配置。

动态分配策略可能会出现 HDMI PHY PLL 都被占用的情况，这个时候 log 中会打印 HDMI PHY PLL 的占用信息：

```
[  220.692900][  T383] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] Update mode to 1920x1080p60,
type: 11(if:1000) for vp1 dclk: 148500000
[  220.693048][  T383] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] dclk_out1 div: 0 dclk_core1
div: 2
[  220.693094][  T383] [drm:vop2_clk_set_parent_extend.isra.61] *ERROR* hdmi1 phy pll is used by vp0:vp2
```

如上，即 hdmi_phy0_pll 已经被 Video Port0 占用，hdmi_phy1_pll 已经被 Video Port2 占用。

当只配置一个 HDMI PHY PLL 时，例如

```
&display_subsystem {
        clocks = <&hdptxphy_hdmi_clk0>;
        clock-names = "hdmi0_phy_pll";
};
```

当出现如下 log 时，即说明 hdmi_phy0_pll 已经被 Video Port2 占用。

```
[   35.059306][  T386] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] Update mode to 1920x1080p60,
type: 11(if:800) for vp0 dclk: 148500000
[   35.059416][  T386] rockchip-vop2 fdd90000.vop: [drm:vop2_crtc_atomic_enable] dclk_out0 div: 0 dclk_core0
div: 2
[   35.059447][  T386] [drm:vop2_clk_set_parent_extend.isra.61] *ERROR* hdmi0: phy pll is used by vp2
```

4.5.6.5 RK3588 VOP DCLK 分配策略选用

上述的三种策略，主要是为了满足 HDMI 和 DP 接口可以任意分频的需求，大部分应用场景 HDMI 接口动态分配策略，即可以满足要求。下面列举部分场景的 DCLK 分配策略。

(1) 四个 Video Port 输出， DP 和 HDMI 要求满足任意分频

Video Port	Interface
VP0	HDMI0
VP1	HDMI1
VP2	DP0
VP3	DSI0

使用HDMI 接口动态切换策略，HDMI 接口 pixe clock 小于等于 600MHz 时 dclk 源选用 hdmi phy pll, 满足任意分频，pixel clock 大于等于 600MHz 时，使用 CRU PLL，整数分频，过滤部分不支持的分辨率。DP0 接口对应的 VP2 使用 VPLL, 满足任意分频需求。

kernel 6.1 及以上版本 dts 配置如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi0>, <&hdptxphy_hdmi1>;
    clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};
&vp2 {
    assigned-clocks = <&cru DCLK_VP2_SRC>;
    assigned-clock-parents = <&cru PLL_V0PLL>;
};
```

(2) 4 个 Video Port 输出，DP 和 HDMI 要求满足任意分频

Video Port	Interface
VP0	HDMI0
VP1	DP0
VP2	DP1
VP3	DSI0

使用HDMI 接口动态切换策略，HDMI0 接口 pixe clock 不超过 600MHz 时 VP0 dclk 源选用 hdmi phy pll, 满足任意分频，pixel clock 大于等于 600MHz 时，使用 CRU PLL，整数分频，过滤部分不支持的分辨率。DP0 接口使用 hdmi phy1 pll 为 VP1 dclk 源, DP1 接口使用 V0PLL 为 VP2 dclk 源, 满足任意分频需求。

kernel 6.1 及以上版本 dts 配置如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi0>;
    clock-names = "hdmi0_phy_pll";
};

&vp1 {
```

```
    assigned-clocks = <&cru DCLK_VP1>;
    assigned-clock-parents = <&hdptxphy_hdmi1>;
};

&vp2 {
    assigned-clocks = <&cru DCLK_VP2_SRC>;
    assigned-clock-parents = <&cru PLL_V0PLL>;
};
```

(3) 接口任意切换 VP 并满足不同接口任意分频需求的场景。即不同的接口可能就在不同的 VP 间切换使用，这属于比较复杂的使用场景(目前仅 NVR 场景使用)。如下 HDMI/DP/DSI 均可切换使用 VP0/1/2 的这种场景：

Video Port	Interface
VP0, VP1, VP2	HDMI0
VP0, VP1, VP2	HDMI1
VP0, VP1, VP2	DP0
VP0, VP1, VP2	DP1

这种场景使用如下动态分配策略。kernel 6.1 及以上版本 dts 配置如下：

```
&hdptxphy_hdmi0 {
    status = "okay";
};

&hdptxphy_hdmi1 {
    status = "okay";
};

&display_subsystem {
    clocks = <&hdptxphy_hdmi0>, <&hdptxphy_hdmi1>;
    clock-names = "hdmi0_phy_pll", "hdmi1_phy_pll";
};

&vop {
    rockchip,extend-phy-pll-shared-mode;
};

&vp2 {
    assigned-clocks = <&cru DCLK_VP2_SRC>;
    assigned-clock-parents = <&cru PLL_V0PLL>;
};
```

如上的配置，假设初始时，HDMI0 使用 VP0, HDMI1 使用 VP1, DP0 使用 VP1，如下：

route	dclk source
VP0->HDMI0	hdmi phy0 pll
VP1->HDMI1	hdmi phy1 pll
VP2->DP0	V0PLL

之后如果 HDMI0 和 HDMI1 都拔出，HDMI1 先重新接到 VP0 上，DP1 再接到 VP1，dclk 源分配如下:

route	dclk source
VP0->HDMI1	hdmi phy1 pll
VP1->DP1	hdmi phy0 pll
VP2->DP0	V0PLL

上述动态分配策略的这种场景，HDMI 接口使用 VP 时选用对应的 hdmi phy pll，并需要保证 hdmi phy pll 未被其他接口使用, 非 HDMI 接口使用 VP 时，如果对应的 VP 源不是 V0PLL，并且 hdmi phy pll 也未被其他接口占用，可以使用 hdmi phy pll 作为 dclk 源。使用动态分配策略时，需要应用来保证使能 HDMI 接口前，对应的 hdmi phy pll 未被其他接口占用。

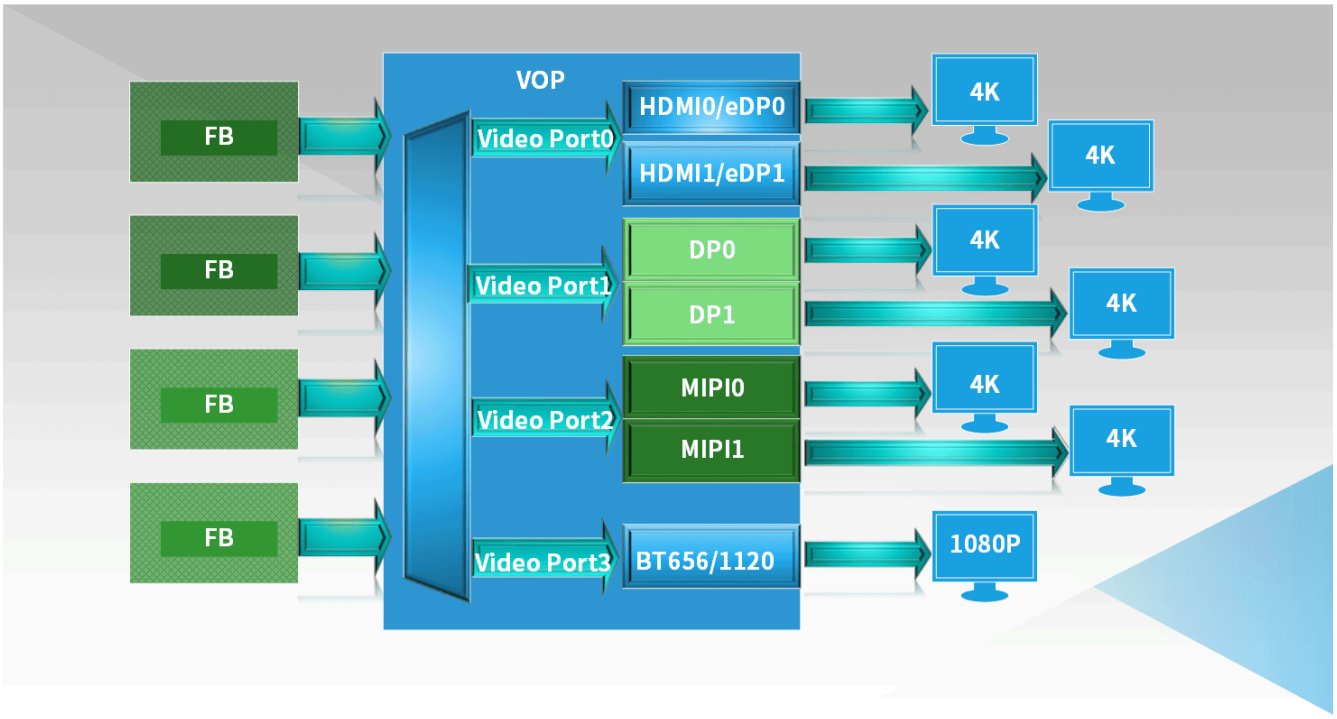
5. VOP 2.0 架构下的多屏显示

根据前面的介绍，对于采用 VOP 1.0 架构的芯片，如果要实现多屏异显，一般需要多个 VOP，所以对于 VOP 1.0 架构的平台，只有 RK3288，RK3399，PX30 这些包含双 VOP 的芯片才支持双屏异显。

对于 VOP 2.0 架构的芯片，由于采用了统一显示架构，可以更高效的利用 IP 资源，在最基础的条件下，VOP 内部包含几个 Video Port 就能实现几路独立的显示输出。根据 **VOP2 平台显示通路** 章节的内容可以知道，RK3568 有三路独立的 Video Port，RK3588 平台有四路独立的 Video Port, 所以在最基础的条件下，RK3568 和 RK3588 可以分别实现 3 路和 4 路独立的显示输出。

5.1 Connector-mirror

VOP2 的 Connector-mirror 技术支持一个 Video Port 同时驱动多路显示接口，输出相同的显示时序并显示相同的内容。



如图所示，在 RK3588 上，通过 connector-mirror 技术，把两路 HDMI/eDP 连接在 VP0 上，把两路 DP 连接在 VP1 上，把两路 MIPI DSI 连接在 VP2 上，VP3 通过 BT656，BT1120 可以同时输出 7 路，四组独立的显示输出，其中每一组（同一个 VIdeo Port 上的两个显示接口）输出的显示时序相同，且显示内容相同。

在这种应用模式下，每一组显示通路输出的最大分辨率受对应的 Video Port 和显示接口的最大分辨率限制。

这种显示特性可以通过 dts 配置开启，在 dts 里面只要把两个显示接口挂接在同一个 VP 上即可：

```
&hdmi0 {
        status = "okay";
};
```

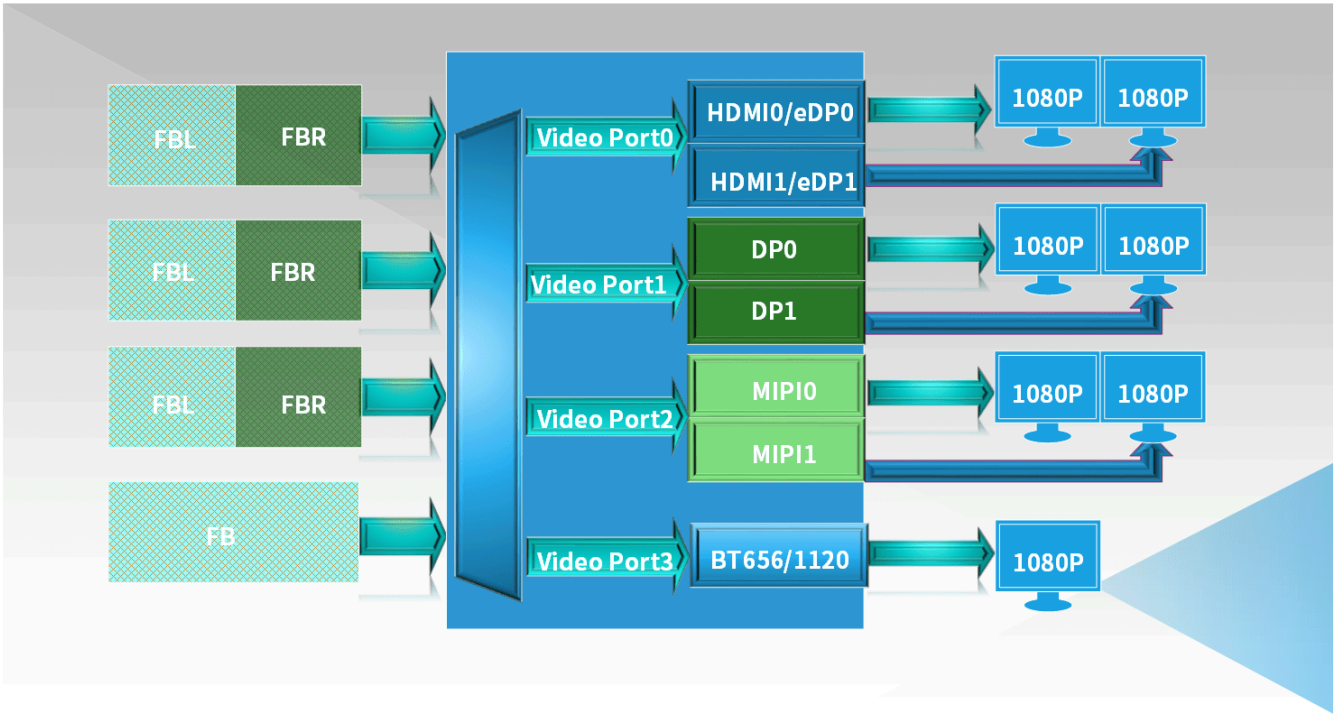
```
&hdmi1 {
        status = "okay";
};

&hdmi0_in_vp0 {
        status = "okay";
};

&hdmi1_in_vp0 {
        status = "okay";
};

&hdmi0_in_vp1 {
        status = "disabled";
};

&hdmi0_in_vp2 {
        status = "disabled";
};

&hdmi1_in_vp1 {
        status = "disabled";
};

&hdmi1_in_vp2 {
        status = "disabled";
};
```

该配置把 HDMI0 和 HDMI1 挂接在 VP0 上，开启 connector-mirror 功能。

目前 NVR SDK 支持这种功能，Android 需要更新到 1.4.15 以后的 hwc 版本才支持这种特性。

5.2 Connector-split

VOP2 提供的 Connector-split 功能是一种类似 mipi 双通道模式的技术，可以让一路 Video Port 输出按照水平方向平分成左右两路，同时驱动两个显示接口，显示时序相同，内容独立的画面。



如上图所示，如果在 VP0 上开启 split 模式，则 VP0 的输出可以同时驱动两个显示接口(HDMI0/1、eDP0/1 或者其他能与 VP0 连接的显示接口)，两个显示接口上显示的内容为 VP0 输出的内容水平方向左右平分，比如 VP0 以 3840x1080 的分辨率输出，则两个显示接口各显示 1920x1080 的输出。

通过这种技术，可以在 RK3588 上扩展出 7 路独立的显示输出。

需要注意的是，每一个 VP 上参与 split 输出的两个显示接口，输出的时序，帧率必须相同。

通过 Connector-split 技术，可以扩展出更多的多屏异显功能，目前只有 RK3588/RK3576 支持该功能， Connector-split 根据接口类型又分以下两种模式：

1. 相同显示接口的 split 模式：`rockchip,split-mode`

   (1) 单个 VP 接 2 个接口/时序完全相同的屏；

   (2) 两个屏分别显示左右半屏图像；

   (3) DTS 配置/EDID 读取单个屏的时序，驱动通过调用接口：`drm_mode_convert_to_split_mode()` 将水平方向的时序 *2 后传给 VOP；

   (4) 软件注册 1 个 connetor，应用层看到 1 个显示设备；

这种模式下，dts 配置文件中把参与 split 的两个显示接口挂接在同一个 VP 上，且打开左边显示接口的 `rockchip,split-mode` 属性。

比如 ，按照如下配置，打开 hdmi0 和 hdmi1 在 VP0 上的 split 功能：

```
&hdmi0 {
        status = "okay";
};

&hdmi1 {
        status = "okay";
        rockchip,split-mode;
};

&hdmi0_in_vp0 {
        status = "okay";
};

&hdmi1_in_vp0 {
        status = "okay";
};

&hdmi0_in_vp1 {
        status = "disabled";
};

&hdmi0_in_vp2 {
        status = "disabled";
};

&hdmi1_in_vp1 {
        status = "disabled";
};

&hdmi1_in_vp2 {
        status = "disabled";
};
```

在驱动实现上，为了方便上层应用适配，尽量和 MIPI 双通道技术接近，屏蔽底层实现差异，每个 Video Port 上进行 split 的两个显示接口只会向 drm 系统注册一个 encoder 和 connector，所以在用户空间，每一 CRTC(Video Port) 上只会看到一个 connector 设备，这个信息可以通过 modetest 的输出确认。

对于 Android 应用，希望每一个屏幕都对应一个独立的显示设备，针对这种需求，Rockchip 平台的 Android hwc 有做针对性的优化，具体请参考 《DrmHwc2 多屏拼接异显功能说明》文档。

2. 不同显示接口的 split 模式：`rockchip,dual-connector-split`

(1) 单个 VP 接 2 个接口不一样但是时序完全一样的屏；

(2) 两个屏分别显示左右半屏图像；

(3) DTS 配置/EDID 读取单个屏的时序，驱动通过调用接口：`drm_mode_convert_to_split_mode()` 将水平方向的时序 *2 后传给 VOP；

(4) 软件注册 2 个 connector，应用层看到 2 个显示设备；

和 `rockchip,split-mode` 的主要区别是使用 2 个不同的显示接口，由于 DRM 框架需要对 2 个显示接口的驱动分别进行管理，所以注册了 2 个 connector，应用层看到的是 2 个显示设备；这种模式下，通过在对应接口的节点下配置属性：`rockchip,dual-connector-split` 就可以打开该功能，同时可以使用属性：`rockchip,left-display` 来标注左边的那个屏。

比如 ，按照如下配置，打开 HDMI0 和 DP0 在 VP0 上的 split 功能，同时 HDMI0 显示左半屏的图像：

```
&hdmi0 {
        status = "okay";
        rockchip,dual-connector-split;
        rockchip,left-display;
};

&dp0 {
        status = "okay";
        rockchip,dual-connector-split;
};

&hdmi0_in_vp0 {
        status = "okay";
};

&dp0_in_vp0 {
        status = "okay";
};

&hdmi0_in_vp1 {
        status = "disabled";
};

&hdmi0_in_vp2 {
        status = "disabled";
};

&dp0_in_vp1 {
        status = "disabled";
};

&dp0_in_vp2 {
        status = "disabled";
};
```

6. 硬件相关

6.1 RGB 输出/TTL 模式硬件连接

6.1.1 VOP 1.0 RGB 接口硬件连接方式

1. 判断是 VOP 1.0 还是 VOP 2.0 的设计，可以从 3.1 章节的 VOP version 中查询；
2. 对于 SOC 支持 24bit RGB 输出的硬件连接方式

interface	RGB parallel						
display mode index	mode0	mode1	mode2	mode3	mode4	mode5	
display mode	RGB parallel 24 bit	RGB parallel 18 bit	RGB parallel 18 bit	RGB parallel 16 bit	RGB parallel 16 bit	serial 3x8/4x8	serial 3x6/4x6
dclk	dclk						
vsync	vsync						
hsync	hsync						
den	den						
data	data[23:0]	data[23:18] data[15:10] data[7:2]	data[17:0]	data[23:19] data[15:10] data[7:3]	data[15:0]	data[7:0]	data[7:2]
D23	R7	R5	—	R4	—	—	—
D22	R6	R4	—	R3	—	—	—
D21	R5	R3	—	R2	—	—	—
D20	R4	R2	—	R1	—	—	—
D19	R3	R1	—	R0	—	—	—
D18	R2	R0	—	—	—	—	—
D17	R1	—	R5	—	—	—	—
D16	R0	—	R4	—	—	—	—
D15	G7	G5	R3	G5	R4	—	—
D14	G6	G4	R2	G4	R3	—	—
D13	G5	G3	R1	G3	R2	—	—
D12	G4	G2	R0	G2	R1	—	—
D11	G3	G1	G5	G1	R0	—	—
D10	G2	G0	G4	G0	G5	—	—
D9	G1	—	G3	—	G4	—	—
D8	G0	—	G2	—	G3	—	—
D7	B7	B5	G1	B4	G2	D7	D5
D6	B6	B4	G0	B3	G1	D6	D4
D5	B5	B3	B5	B2	G0	D5	D3
D4	B4	B2	B4	B1	B4	D4	D2
D3	B3	B1	B3	B0	B3	D3	D1
D2	B2	B0	B2	—	B2	D2	D0
D1	B1	—	B1	—	B1	D1	—
D0	B0	—	B0	—	B0	D0	—

3. 对于 SOC 支持 18bit RGB 输出的硬件连接方式

interface	RGB parallel			
display mode index	mode2	mode4	mode5	
display mode	RGB parallel 18 bit	RGB parallel 16 bit	serial 3x8/4x8	serial 3x6/4x6
dclk	dclk			
vsync	vsync			
hsync	hsync			
den	den			
data	data[17:0]	data[15:0]	data[7:0]	data[7:2]
D17	R5	—	—	—
D16	R4	—	—	—
D15	R3	R4	—	—
D14	R2	R3	—	—
D13	R1	R2	—	—
D12	R0	R1	—	—
D11	G5	R0	—	—
D10	G4	G5	—	—
D9	G3	G4	—	—
D8	G2	G3	—	—
D7	G1	G2	D7	D5
D6	G0	G1	D6	D4
D5	B5	G0	D5	D3
D4	B4	B4	D4	D2
D3	B3	B3	D3	D1
D2	B2	B2	D2	D0
D1	B1	B1	D1	—
D0	B0	B0	D0	—

4. 对于 VOP 1.0 中 MCU 接口 DATA 线连接方式和 RGB parallel 的连接方式一致，需要注意的是，RGB parallel 中的 4 个 时钟信号复用成 MCU 接口的控制信号，以下是具体的对应关系：

dclk	mcu_rs	1表示发送的是数据，0表示发送的是命令
vsync	mcu_csn	片选信号，低有效
hsync	mcu_wrn	写使能信号，上升沿有效
den	mcu_rdn	1：表示发数据到屏，0：表示从屏读数据

6.1.2 VOP 2.0 及之后的版本 RGB 接口硬件连接方式

interface	RGB parallel		
display mode index	mode0	mode1	mode3
display mode	RGB parallel 24 bit	RGB parallel 18 bit	RGB parallel 16 bit
dclk	dclk		
vsync	vsync		
hsync	hsync		
den	den		
data	data[23:0]	data[23:18] data[15:10] data[7:2]	data[23:19] data[15:10] data[7:3]
D23	R7	R5	R4
D22	R6	R4	R3
D21	R5	R3	R2
D20	R4	R2	R1
D19	R3	R1	R0
D18	R2	R0	—
D17	R1	—	—
D16	R0	—	—
D15	G7	G5	G5
D14	G6	G4	G4
D13	G5	G3	G3
D12	G4	G2	G2
D11	G3	G1	G1
D10	G2	G0	G0
D9	G1	—	—
D8	G0	—	—
D7	B7	B5	B4
D6	B6	B4	B3
D5	B5	B3	B2
D4	B4	B2	B1
D3	B3	B1	B0
D2	B2	B0	—
D1	B1	—	—
D0	B0	—	—

6.1.3 不同 display mode index 对应的软件配置

display mode index	bus format
mode0	MEDIA_BUS_FMT_RGB888_1X24
mode1	MEDIA_BUS_FMT_RGB666_1X24_CPADHI
mode2	MEDIA_BUS_FMT_RGB666_1X18
mode3	MEDIA_BUS_FMT_RGB565_1X24_CPADLO
mode4	MEDIA_BUS_FMT_RGB565_1X16
mode5	MEDIA_BUS_FMT_RGB888_3X8

6.1.4 BT.656 和 BT.1120 的硬件连接方式

SOC TX 引脚	Clock	D15	D14	D13	D12	D11	D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	软件 bus_format 配置
BT.656 接法	Clock	-	-	-	-	-	-	-	-	D7	D6	D5	D4	D3	D2	D1	D0	MEDIA_BUS_FMT_UYVY8_2X8
BT.1120 接法1	Clock	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	C7	C6	C5	C4	C3	C2	C1	C0	MEDIA_BUS_FMT_YUYV8_1X16
BT.1120 接法2	Clock	C7	C6	C5	C4	C3	C2	C1	C0	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	MEDIA_BUS_FMT_UYVY8_1X16

根据接收端的 Y/U/V 处理顺序不同，如果出现显示颜色不对，还可以在 DTS 中调整不同的 bus_format 来适配，BT.656 和 BT.1120 分别有以下四种配置：

BT.656：

```
#define MEDIA_BUS_FMT_UYVY8_2X8          0x2006
#define MEDIA_BUS_FMT_VYUY8_2X8          0x2007
#define MEDIA_BUS_FMT_YUYV8_2X8          0x2008
#define MEDIA_BUS_FMT_YVYU8_2X8          0x2009
```

BT.1120：

```
#define MEDIA_BUS_FMT_UYVY8_1X16        0x200f
#define MEDIA_BUS_FMT_VYUY8_1X16        0x2010
#define MEDIA_BUS_FMT_YUYV8_1X16        0x2011
#define MEDIA_BUS_FMT_YVYU8_1X16        0x2012
```

MEDIA_BUS_FMT 的定义可以参考：

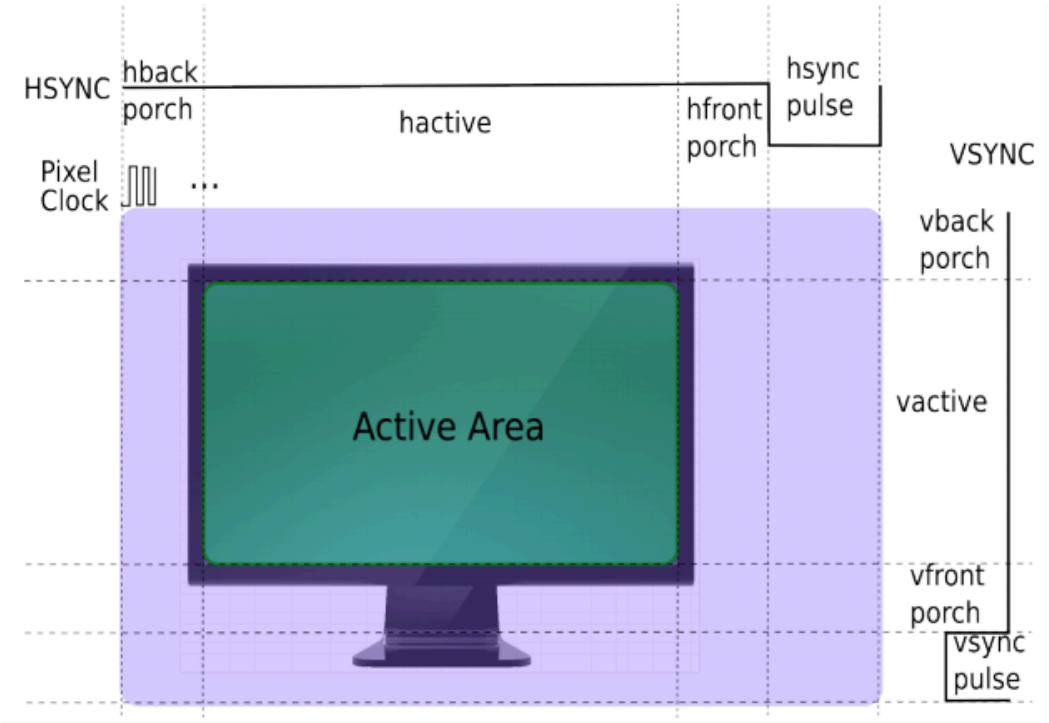
```
./include/uapi/linux/media-bus-format.h
```

6.2 LVDS Data Mapping

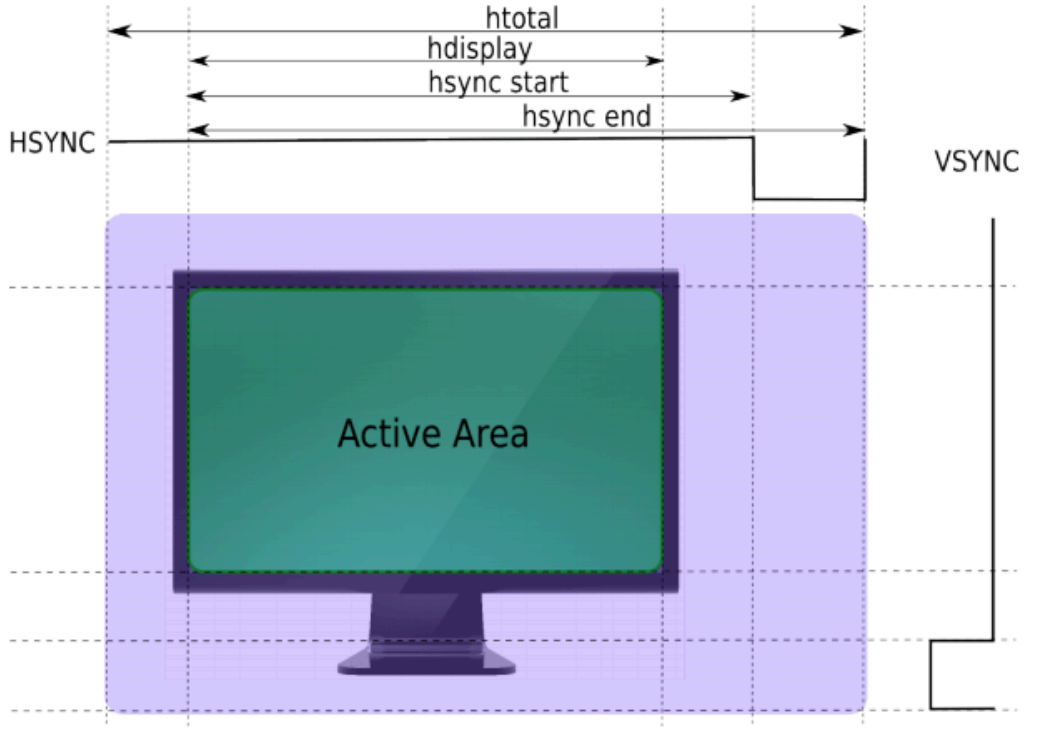
bus-format	Timeslot	Data organization			
		Lane3	Lane2	Lane1	Lane0
MEDIA_BUS_FMT_RGB666_1X7X3_SPWG	0	—	DEN	B1	G0
	1	—	VSYNC	B0	R5
	2	—	HSYNC	G5	R4
	3	—	B5	G4	R3
	4	—	B4	G3	R2
	5	—	B3	G2	R1
	6	—	B2	G1	R0
MEDIA_BUS_FMT_RGB888_1X7X4_SPWG	0	GND	DEN	B1	G0
	1	B7	VSYNC	B0	R5
	2	B6	HSYNC	G5	R4
	3	G7	B5	G4	R3
	4	G6	B4	G3	R2
	5	R7	B3	G2	R1
	6	R6	B2	G1	R0
MEDIA_BUS_FMT_RGB888_1X7X4_JEIDA	0	GND	DEN	B3	G2
	1	B1	VSYNC	B2	R7
	2	B0	HSYNC	G7	R6
	3	G1	B7	G6	R5
	4	G0	B6	G5	R4
	5	R1	B5	G4	R3
	6	R0	B4	G3	R2

7. 扫描时序说明

7.1 常见的扫描时序图



7.2 DRM 对扫描时序的定义



7.3 软件配置的对应关系和转换

```
          Active                Front           Sync           Back
         Region                 Porch                          Porch
<----------------><----------><----------><----------><----------->
 //////////////////////|
 //////////////////////| |
//////////////////////  |..................              ................
                                                 ----------
<---- [hv]display ---->
<---------- [hv]sync_start ---------->
<---------------- [hv]sync_end ---------------->
<---------------------- [hv]total ---------------------->*
```

```
void drm_display_mode_from_videomode(const struct videomode *vm,
                                     struct drm_display_mode *dmode)
{
        dmode->hdisplay = vm->hactive;
        dmode->hsync_start = dmode->hdisplay + vm->hfront_porch;
        dmode->hsync_end = dmode->hsync_start + vm->hsync_len;
        dmode->htotal = dmode->hsync_end + vm->hback_porch;

        dmode->vdisplay = vm->vactive;
        dmode->vsync_start = dmode->vdisplay + vm->vfront_porch;
        dmode->vsync_end = dmode->vsync_start + vm->vsync_len;
        dmode->vtotal = dmode->vsync_end + vm->vback_porch;

        dmode->clock = vm->pixelclock / 1000;

        if (vm->flags & DISPLAY_FLAGS_HSYNC_HIGH)
                dmode->flags |= DRM_MODE_FLAG_PHSYNC;
        ...

        drm_mode_set_name(dmode);
}

void drm_display_mode_to_videomode(const struct drm_display_mode *dmode,
                                   struct videomode *vm)
{
        vm->hactive = dmode->hdisplay;
        vm->hfront_porch = dmode->hsync_start - dmode->hdisplay;
        vm->hsync_len = dmode->hsync_end - dmode->hsync_start;
        vm->hback_porch = dmode->htotal - dmode->hsync_end;

        vm->vactive = dmode->vdisplay;
        vm->vfront_porch = dmode->vsync_start - dmode->vdisplay;
        vm->vsync_len = dmode->vsync_end - dmode->vsync_start;
        vm->vback_porch = dmode->vtotal - dmode->vsync_end;

        vm->pixelclock = dmode->clock * 1000;

        if (dmode->flags & DRM_MODE_FLAG_PHSYNC)
                vm->flags |= DISPLAY_FLAGS_HSYNC_HIGH;
        ...
}
```

7.4 查看当前配置的时序

```
rk3568_r:/ # cat /d/dri/0/summary
Video Port0: DISABLED
Video Port1: ACTIVE
    Connector: DSI-1
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[0] color_space[0]
    Display mode: 1080x1920p60
        clk[132000] real_clk[132000] type[48] flag[a]
        H: 1080 1095 1097 1127
        V: 1920 1935 1937 1952
    Cluster0-win0: ACTIVE
        win_id: 4
        format: AB24 little-endian (0x34324241)[AFBC] SDR[0] color_space[0]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 0
        src: pos[0, 0] rect[1080 x 1920]
        dst: pos[0, 0] rect[1080 x 1920]
        buf[0]: addr: 0x0000000000edb000 pitch: 4352 offset: 0
Video Port2: DISABLED
```

8. 带宽的计算方法

8.1 图像的带宽

以 1080P ARGB 格式的图像数据为例：

p0	p0	p0	p0	p1	p1	p1	p1	p2	p2	p2	p2	p3	p3	p3	p3
p4	p4	p4	p4	p5	p5	p5	p5	p6	p6	p6	p6	p7	p7	p7	p7
p8	p8	p8	p8	p9	p9	p9	p9	p10	p10	p10	p10	p11	p11	p11	p11
p12	p12	p12	p12	p13	p13	p13	p13	p14	p14	p14	p14	p15	p15	p15	p15

ARGB 格式一个像素占用的内存大小：4 Byte

1080P ARGB 格式的数据占用内存：1920 x 1080 x 4Byte/pixel = 8,100 Kbyte

如果按 60fps 刷新，占用的带宽是： 8,100 x 60fps = 474.6 Mbyte/s

如果数据格式改成 YUV420SP(NV12):

Single Frame YUV420: NV12

Y1	Y2	Y3	Y4	Y5	Y6
Y7	Y8	Y9	Y10	Y11	Y12
Y13	Y14	Y15	Y16	Y17	Y18
Y19	Y20	Y21	Y22	Y23	Y24
U1	V1	U2	V2	U3	V3
U4	V4	U5	V5	U6	V6

Position in byte stream:

Y1	Y2	Y3	Y4	Y5	Y6	Y7	Y8	Y9	Y10	Y11	Y12	Y13	Y14	Y15	Y16	Y17	Y18	Y19	Y20	Y21	Y22	Y23	Y24	U1	V1	U2	V2	U3	V3	U4	V4	U5	V5	U6	V6

NV12 格式一个像素占用的内存大小：1.5 Byte

1080P NV12 格式的数据占用内存：1920 x 1080 x 1.5Byte/pixel = 3,037.5 Kbyte

如果按 60fps 刷新，占用的带宽是： 3,037.5 x 60fps = 178 Mbyte/s

8.2 显示接口的带宽

```
disp_timings0: display-timings {
        native-mode = <&dsi0_timing0>;
        dsi0_timing0: timing0 {
                clock-frequency = <132000000>;
                hactive = <1080>;
                vactive = <1920>;
                hfront-porch = <15>;
                hsync-len = <2>;
                hback-porch = <30>;
                vfront-porch = <15>;
                vsync-len = <2>;
                vback-porch = <15>;
                hsync-active = <0>;
                vsync-active = <0>;
                de-active = <0>;
                pixelclk-active = <1>;
        };
};
```

以上面这张图配置的时序为例，当前这个时序下，按 60 帧刷新需要的 dclk 是：131994240 hz，dts 实际按取整 132000000 hz 配置：

```
htotal = hfp + hsync + hbp + hactive = 15 + 2 + 30 + 1080 = 1,127
vtotal = vfp + vsync + vbp + vactive = 15 + 2 + 15 + 1920 = 1,952
dclk = htotal x vtotal x fps = 1127 x 1952 x 60fps = 131,994,240
```

MIPI 接口上传输的频率是：

```
132M x 3(RGB) x 8(bpc) / 4(lane) / 0.9 = 880 Mbps
```

其中：

x3(RGB)：是每一个 pixel 有 RGB 3 个分量；

x8(bpc)：是每一个分量的位深是 8bit；

/4(lane)：是这么多数据量在 4 lane 上传输，/4 是计算每 lane 的数据量；

/0.9：是考虑 mipi 时序的传输效率；

9. 常用的 debug 手段

9.1 dump 当前的显示状态

9.1.1 使用命令

```
cat /sys/kernel/debug/dri/0/summary
```

```
/ # cat /sys/kernel/debug/dri/0/summary
VOP [ff900000.vop]: ACTIVE
    Connector: eDP
        overlay_mode[0] bus_format[1009] output_mode[f] color_space[0]
    Display mode: 1536x2048p60
        clk[200000] real_clk[200000] type[0] flag[a]
        H: 1536 1548 1564 1612
        V: 2048 2056 2060 2068
    win0-0: ACTIVE
        format: XR24 little-endian (0x34325258) SDR[0] color_space[0]
        csc: y2r[0] r2r[0] r2y[0] csc mode[0]
        zpos: 0
        src: pos[0x0] rect[1536x2048]
        dst: pos[0x0] rect[1536x2048]
        buf[0]: addr: 0x0000000000000000 pitch: 6144 offset: 0
    win1-0: DISABLED
    win2-0: DISABLED
    win2-1: DISABLED
    win2-2: DISABLED
    win2-3: DISABLED
    win3-0: DISABLED
    win3-1: DISABLED
    win3-2: DISABLED
    win3-3: DISABLED
    post: sdr2hdr[0] hdr2sdr[0]
    pre : sdr2hdr[0]
    post CSC: r2y[0] y2r[0] CSC mode[1]
VOP [ff8f0000.vop]: ACTIVE
    Connector: DSI
        overlay_mode[0] bus_format[100a] output_mode[0] color_space[0]
    Display mode: 1280x720p29
        clk[96000] real_clk[96000] type[8] flag[a]
        H: 1280 2280 2480 2680
        V: 720 920 1120 1220
    win0-0: ACTIVE
        format: XR24 little-endian (0x34325258) SDR[0] color_space[0]
        csc: y2r[0] r2r[0] r2y[0] csc mode[0]
        zpos: 0
        src: pos[0x0] rect[1280x720]
        dst: pos[0x0] rect[1280x720]
        buf[0]: addr: 0x0000000000000000 pitch: 6144 offset: 0
    win2-0: DISABLED
    win2-1: DISABLED
```

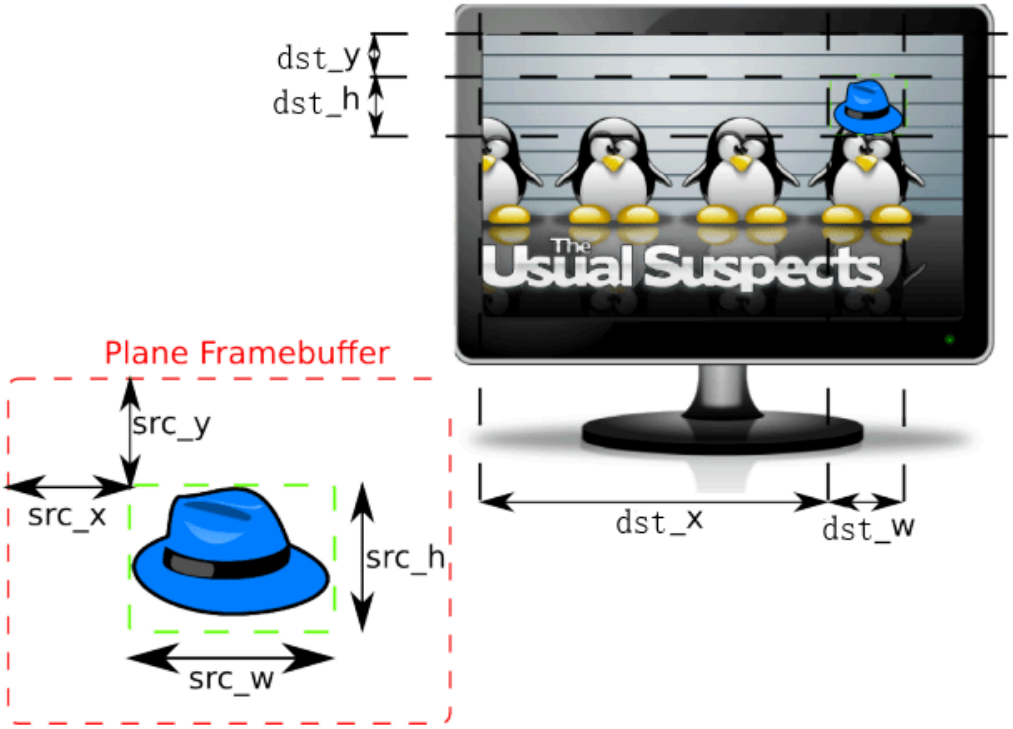
Copyright © 2018 Fuzhou Rockchip Electronics Co., Ltd.

需要注意的是该命令依赖内核的 debugfs 功能，如果内核没有开启 debugfs 功能或者系统没有挂载 debugfs 节点，需要确保内核打开 CONFIG_DEBUG_FS 选项，并通过如下命令手动挂载：

```
mount -t debugfs none /sys/kernel/debug
```

9.1.2 参数说明

1. 两个红色方框表示两个显示设备使用的 vop 分别是 ff900000.vop 和 ff8f0000.vop；
2. 绿色部分表示 connector 信息，两个显示设备分别为 eDP 屏和 MIPI 屏；
3. 粉色部分为显示模式，可以知道具体的时序、DCLK 以及帧率，上图中两个设备分别为分辨率为 1536x2048p60 的 eDP 屏和分辨率 1280x720p29 的 MIPI 屏；
4. 蓝色部分是 VOP 图层信息，第一个显示设备打开 win0 图层，大小为 1536x2048 格式为 XRGB 第二个显示设备打开 win0 图层，大小 1280x720 格式为 XRGB， src 和 dst 表示源数据和显示的大小和位置，如果 src 和 dst 的大小不一致，VOP 会进行缩放处理，如下图所示:



5. 橙色部分为 VOP HDR、CSC 的一些状态信息；SDR 表示应用送下来的是 SDR 格式的数据，如果是 HDR 格式的数据，会看到 HDR 的关键字。
6. VOP2 平台的 summary 信息基本和之前平台的一致，只是把之前 VOP 改成了 Video Port，图层名字从原来的 winx， 变成了 Cluster-winx 或者 Esmart-winx:

```
[root@RK3588:/]#
[root@RK3588:/]#
[root@RK3588:/]#
[root@RK3588:/]# cat /sys/kernel/debug/dri/0/summary
Video Port0: DISABLED
Video Port1: DISABLED
Video Port2: DISABLED
Video Port3: ACTIVE
    Connector: DSI-1
        bus_format[100a]: RGB888_1X24
        overlay_mode[0] output_mode[0] color_space[0], eotf:0
    Display mode: 1080x1920p60
        clk[132000] real_clk[132000] type[48] flag[a]
        H: 1080 1095 1099 1129
        V: 1920 1935 1937 1952
    Esmart3-win0: ACTIVE
        win_id: 11
        format: XR24 little-endian (0x34325258) SDR[0] color_space[0] glb_alpha[0xff]
        rotate: xmirror: 0 ymirror: 0 rotate_90: 0 rotate_270: 0
        csc: y2r[0] r2y[0] csc mode[0]
        zpos: 3
        src: pos[0, 0] rect[1080 x 1920]
        dst: pos[0, 0] rect[1080 x 1920]
        buf[0]: addr: 0x0000000000000000 pitch: 4320 offset: 0
```

7. 每个图层下边有一个 format 属性，他可能是 AB24、XR24、NV12 之类的标准 fourcc 值，表示的是这个图层所显示的 framebuffer 中数据的格式，具体的含义可以参考 drm fourcc。

9.1.3 dump VOP 当前的配置信息

由于 VOP2 架构复杂，寄存器相关配置量非常大，我们提供了以下两个方法 dump VOP 当前的配置信息：

1. 通过 cat DRM 的设备节点获取配置信息，如：

   (1) 获取 VOP 中正在工作的模块的寄存器配置

```
cat /sys/kernel/debug/dri/0/active_regs
```

(2) 获取 VOP 所有子模块的寄存器配置

```
cat /sys/kernel/debug/dri/0/regs
```

以上两个命令可以 dump VOP 的相关寄存器配置，再配合上一个章节的 cat /sys/kernel/debug/dri/0/summary，基本可以 dump 出当前 VOP 的显示状态信息。

需要注意的是，这个功能在 Linux-6.1 及之后的内核默认就支持，在 Linux-5.10 和 Linux-4.19 内核需要更新到较新的版本才支持，如果你是这2个版本的内核，同时又不方便更新的，那可以使用下面的方法2。

2. 使用 vop2_dump.sh 脚本

   这个脚本可以更方便的 dump vop 相关的显示状态信息，大部分 SDK 里面默认有带这个脚本，如果没有，可以通过如下地址获取：vop2_dump。

   需要注意的是，该脚本也依赖 `debugfs` ，并且需要内核打开 `CONFIG_DEVMEM` 选项，以方便该脚本利用 `io` 命令读取寄存器配置。

9.2 dump 当前显示的 buffer

有时候我们发现屏幕上显示的内容异常，这种异常有可能是 VOP ——> 显示接口 ——> 屏幕/显示器，这条链路上有异常，也有可能是上面的应用绘制的显示数据就是异常的。这时候我们可以把 VOP 图层显示的 buffer 的数据 dump 出来，然后用 7yuv 之类的软件查看，这些数据是否正常。如果这些数据本身就是异常的，那就说明是 userspace 送下来的数据就是有问题的，反之则要排查 VOP 到屏幕之间的这条显示通路。

9.2.1 使用说明

首选需要确保 Linux kernel make menuconfig 打开 CONFIG_ROCKCHIP_DRM_DEBUG 选项。

```
/sys/kernel/debug/dri/0/ff900000.vop/vop_dump # cat dump
  echo dump    > dump to dump one frame
  echo dumpon  > dump to start vop keep dumping
  echo dumpoff > dump to stop keep dumping
  echo dumpn   > dump n is the number of dump times
  dump path is /data/vop_buf
  if fd err = -3 try rm -r /data/vopbuf echo dump1 > dump can fix it
  if fd err = -28 save needed data try rm -r /data/vopbuf
```

1. dump 一帧当前显示的 buffer

   echo dump > /sys/kernel/debug/dri/0/**ff900000.vop**/vop_dump/dump

2. 连续 dump n 帧显示的 buffer

   echo dumpn > /sys/kernel/debug/dri/0/**ff900000.vop**/vop_dump/dump

3. dump 出来的文件保存在 /data/vop_buf/，可以使用 7yuv 软件查看

4. 以上路径加粗部分 [**ff900000.vop**] 在不同平台不一样，具体路径可以和 cat /sys/kernel/debug/dri/0/summary 节点的 VOP/Video Port 对应，比如对于 RK3588 路径会变成：/sys/kernel/debug/dri/0/**video_port0**/dump

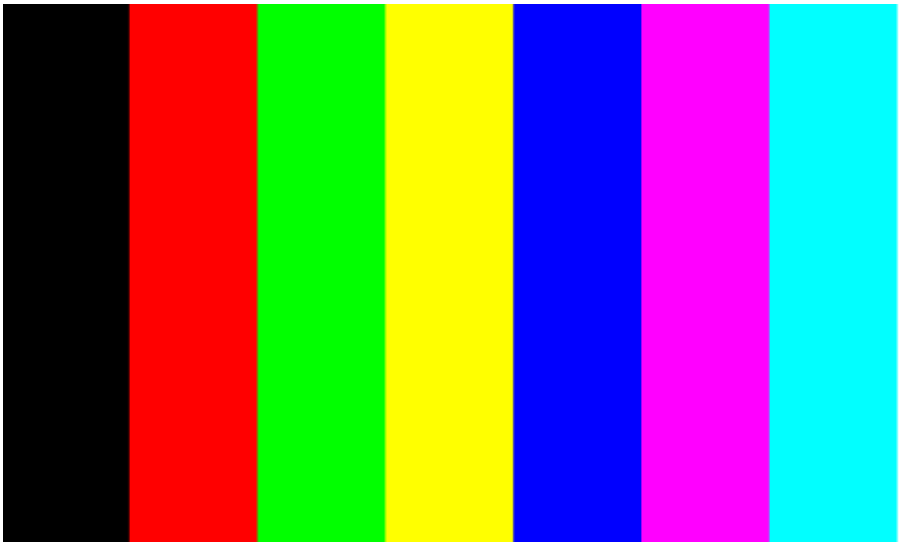
9.3 使用 VP 自测模式显示彩条

使用 VOP2 架构的平台，每个 VP 都支持在自测模式下显示彩条，以 VP0 为例，命令如下：

```
Enable horizontal color bar:
    echo 1 > /sys/kernel/debug/dri/0/video_port0/color_bar
Enable vertical color bar:
    echo 2 > /sys/kernel/debug/dri/0/video_port0/color_bar
Disable color bar:
    echo 0 > /sys/kernel/debug/dri/0/video_port0/color_bar
```

1. color_bar 设置为 1 显示效果

2. color_bar 设置为 2 显示效果



对于产品中遇到屏幕无法显示或者显示异常的场景，可以使用 VP 自测模式按以下流程调试：

1. 如果使用 VP 的自测模式能正常显示彩条，则可以排除显示接口和屏端的异常，重点关注是不是 VOP 及 userspace 异常；
2. 如果使用 VP 的自测模式无法正常显示彩条，则重点关注 VOP 及 VOP 之后模块可能存在异常。

9.4 调整 DRM 打印 Log 等级抓 Log

DRM 根据不同的接口设定以下几个打印等级，可以通过 DRM 的 debug 节点决定开关哪个接口的打印，比如我要查看 ATOMIC 的打印，就你可以输入命令：

```
echo 0x10 > /sys/module/drm/parameters/debug
```

如果想看到所有的调试信息， 可以输入命令：

```
echo 0xff > /sys/module/drm/parameters/debug
```

其中 0x20 Bit5 对应的是 vsync 相关的信息，这个打印的内容非常多，如果调试的时候不关注这个信息，一般不要设置这个 bit。

其他更多的打印等级可以查看下面的定义：

```
Enable debug output, where each bit enables a debug category.
Bit 0 (0x01)  will enable CORE messages (drm core code)
Bit 1 (0x02)  will enable DRIVER messages (drm controller code)
Bit 2 (0x04)  will enable KMS messages (modesetting code)
Bit 3 (0x08)  will enable PRIME messages (prime code)
Bit 4 (0x10)  will enable ATOMIC messages (atomic code)
Bit 5 (0x20)  will enable VBL messages (vblank code)
Bit 7 (0x80)  will enable LEASE messages (leasing code)
Bit 8 (0x100) will enable DP messages (displayport code)
```

这里面有很多打印，默认是不打印到串口的，所以打开了对应的调试开关后，默认在串口终端只会看到部分调试信息，要获取完整 log 信息需要通过 `dmesg` 命令查看。

9.5 查看当前显示时钟

时钟是影响显示模块的关键因素之一，所以我们经常要获取当前系统提供的时钟是多少，可以通过以下命令获取：

1. 获取整个时钟树

```
cat /sys/kernel/debug/clk/clk_summary
```

2. 如果只关注 VOP 的时钟

```
cat /sys/kernel/debug/clk/clk_summary | grep vop
```

对于 HDMI/DP/VGA/CVBS 等显示接口，每个分辨率对应的 DCLK 是有严格的标准要求，如果 DCLK 时钟不对可能会出现无法显示或者出现显示兼容性问题，而对于 eDP/LVDS/MIPI/RGB 等显示接口一般有一定的余量。

在 VOP2 驱动使能对应的显示接口的时候，会有如下的打印，显示接口需要的时钟频率以及从系统中获取到的时钟频率，如果二者不匹配的时候，就要特别注意。

```
rockchip-vop2: [vop2_crtc_atomic_enable] Update mode to 3840x2160p60, type: 10(if:200) for vp2 dclk: 533280000
fdd90000.vop: [drm:vop2_crtc_atomic_enable] dclk_out2 div: 2 dclk_core2 div: 2
fdd90000.vop: [drm:vop2_crtc_atomic_enable] set dclk_vop2 to 533280000, get 533279987
```

这个 log 显示的是，VP2 上整在使能一个 DP 接口(通过 type 判断)，显示 4K 的分辨率，需要的 dclk 为 533280000 HZ，但是系统分配到的是 533279987 HZ，因为 DP 对 dclk 的精度要求非常高，这种情况下，大概率屏幕是点不亮的。

9.6 强行开关显示设备

以 LVDS 为例：

关 LVDS: echo off > /sys/class/drm/card0-LVDS-1/status

开 LVDS: echo on > /sys/class/drm/card0-LVDS-1/status

9.7 查看 DRM buffer 使用情况

通过下面的命令可以知道通过 DRM GEM 申请的 buffer 使用情况：

```
cat /sys/kernel/debug/dri/0/mm_dump
```

```
/ # cat /sys/kernel/debug/dri/0/mm_dump
0x0000000000000000-0x0000000000c00000: 12582912: used
0x0000000000c00000-0x0000000100000000: 4282384384: free
total: 4294967296, used 12582912 free 4282384384
```

9.8 查看 GPIO 状态

在项目刚开始 bring up 点屏阶段，经常需要通过控制 GPIO 的状态来控制屏或者背光的电源，我们可以通过以下命令确认软件配置的 GPIO 状态是否正确：

```
cat /sys/kernel/debug/gpio
```

```
GPIOs 0-31, platform/pinctrl, gpio0:
 gpio-1   (                    |vcc_sd              ) out lo
 gpio-4   (                    |bt_default_wake_host) in  lo
 gpio-5   (                    |GPIO Key Power      ) in  hi
 gpio-9   (                    |bt_default_reset    ) out lo
 gpio-10  (                    |reset               ) out hi

GPIOs 32-63, platform/pinctrl, gpio1:
 gpio-34  (                    |int-n               ) in  hi
 gpio-45  (                    |enable              ) out hi
 gpio-46  (                    |vsel                ) out lo
 gpio-49  (                    |vsel                ) out lo
```

9.9 modetest 的使用

modetest 是集成在 libdrm 中用来测试 drm 系统的命令行工具，它可以提供如下功能：

1. Dump drm 系统的基础状态

```
[root@RK356X:/]# modetest -M rockchip
Encoders:
id      crtc    type    possible crtcs  possible clones
151     0       Virtual 0x00000007      0x00000000
153     0       TMDS    0x00000002      0x00000000
155     71      TMDS    0x00000001      0x00000000
166     0       DPI     0x00000004      0x00000000

Connectors:
id      encoder status          name            size (mm)       modes   encoders
154     0       disconnected    eDP-1           0x0             0       153
  props:
        1 EDID:
                flags: immutable blob
                blobs:
                value:
        2 DPMS:
                flags: enum
                enums: On=0 Standby=1 Suspend=2 Off=3
                value: 0
156     155     connected       HDMI-A-1        530x300         31      155
  modes:
        name refresh (Hz) hdisp hss hse htot vdisp vss vse vtot)
  1920x1080 60 1920 2008 2052 2200 1080 1084 1089 1125 148500 flags: phsync, pvsync; type: preferred,
driver
  1920x1080 60 1920 2008 2052 2200 1080 1084 1089 1125 148352 flags: phsync, pvsync; type: driver
  1920x1080 50 1920 2448 2492 2640 1080 1084 1089 1125 148500 flags: phsync, pvsync; type: driver
  1280x720 60 1280 1390 1430 1650 720 725 730 750 74250 flags: phsync, pvsync; type: driver
  720x576 50 720 732 796 864 576 581 586 625 27000 flags: nhsync, nvsync; type: driver
  720x480 60 720 736 798 858 480 489 495 525 27027 flags: nhsync, nvsync; type: driver
  props:
167     0       disconnected    HDMI-A-2        0x0             0       166
  props:

CRTCs:
id      fb      pos     size
```

```
71      0       (0,0)   (1920x1080)
   60 1920 2008 2052 2200 1080 1084 1089 1125 148500 flags: phsync, pvsync; type: userdef
  props:
        72 PLANE_MASK:
                flags: immutable bitmask
                values: Cluster0=0x1 Cluster1=0x2 Esmart0=0x4 Esmart1=0x8 Smart0=0x10 Smart1=0x20
Cluster2=0x40 Cluster3=0x80 Esmart2=0x100 Esmart3=0x200
                value: 21
        26 GAMMA_LUT:
                flags: blob
                blobs:
                value:
        27 GAMMA_LUT_SIZE:
                flags: immutable range
                values: 0 4294967295
                value: 1024
87      0       (0,0)   (0x0)
   0 0 0 0 0 0 0 0 0 0 flags: ; type:
  props:
        88 PLANE_MASK:
                flags: immutable bitmask
                values: Cluster0=0x1 Cluster1=0x2 Esmart0=0x4 Esmart1=0x8 Smart0=0x10 Smart1=0x20
Cluster2=0x40 Cluster3=0x80 Esmart2=0x100 Esmart3=0x200
                value: 34
        26 GAMMA_LUT:
                flags: blob
                blobs:
                value:
        27 GAMMA_LUT_SIZE:
                flags: immutable range
                values: 0 4294967295
                value: 1024
103     0       (0,0)   (0x0)
   0 0 0 0 0 0 0 0 0 0 flags: ; type:
  props:
        104 PLANE_MASK:
                flags: immutable bitmask
                values: Cluster0=0x1 Cluster1=0x2 Esmart0=0x4 Esmart1=0x8 Smart0=0x10 Smart1=0x20
Cluster2=0x40 Cluster3=0x80 Esmart2=0x100 Esmart3=0x200
                value: 8
        26 GAMMA_LUT:
                flags: blob
                blobs:
                value:
        27 GAMMA_LUT_SIZE:
                flags: immutable range
                values: 0 4294967295
                value: 1024

Planes:
id      crtc    fb      CRTC x,y        x,y     gamma size      possible crtcs
57      0       0       0,0             0,0     0               0x00000007
  formats: XR24 AR24 XB24 AB24 RG24 BG24 RG16 BG16
  props:
        8 type:
                flags: immutable enum
                enums: Overlay=0 Primary=1 Cursor=2
                value: 1
        60 alpha:
                flags: range
                values: 0 65535
                value: 65535
        61 pixel blend mode:
                flags: enum
```

```
                enums: None=2 Pre-multiplied=0 Coverage=1
                value: 0
        62 zpos:
                flags: range
                values: 0 7
                value: 0
        63 NAME:
                flags: immutable bitmask
                values: Smart0-win0=0x1
                value: 1
        70 colorkey:
                flags: range
                values: 0 2164260863
                value: 0
73      0       0       0,0             0,0     0               0x00000007
  formats: XR24 AR24 XB24 AB24 RG24 BG24 RG16 BG16
  props:
        8 type:
                flags: immutable enum
                enums: Overlay=0 Primary=1 Cursor=2
                value: 1
        76 alpha:
                flags: range
                values: 0 65535
                value: 65535
        77 pixel blend mode:
                flags: enum
                enums: None=2 Pre-multiplied=0 Coverage=1
                value: 0
        78 zpos:
                flags: range
                values: 0 7
                value: 1
        79 NAME:
                flags: immutable bitmask
                values: Smart1-win0=0x2
                value: 2
        86 colorkey:
                flags: range
                values: 0 2164260863
                value: 0
89      0       0       0,0             0,0     0               0x00000007
  formats: XR24 AR24 XB24 AB24 RG24 BG24 RG16 BG16 NV12 NV16 NV24 NA12 NA16 NA24 YVYU VYUY
  props:
        8 type:
                flags: immutable enum
                enums: Overlay=0 Primary=1 Cursor=2
                value: 1
        92 alpha:
                flags: range
                values: 0 65535
                value: 65535
        93 pixel blend mode:
                flags: enum
                enums: None=2 Pre-multiplied=0 Coverage=1
                value: 0
        94 zpos:
                flags: range
                values: 0 7
                value: 2
        95 NAME:
                flags: immutable bitmask
                values: Esmart1-win0=0x4
                value: 4
```

```
105     0       0       0,0             0,0     0               0x00000007
  formats: XR24 AR24 XB24 AB24 RG24 BG24 RG16 BG16 NV12 NV16 NV24 NA12 NA16 NA24 YVYU VYUY
  props:
        8 type:
                flags: immutable enum
                enums: Overlay=0 Primary=1 Cursor=2
                value: 0
        108 alpha:
                flags: range
                values: 0 65535
                value: 65535
        109 pixel blend mode:
                flags: enum
                enums: None=2 Pre-multiplied=0 Coverage=1
                value: 0
        110 zpos:
                flags: range
                values: 0 7
                value: 3
        111 NAME:
                flags: immutable bitmask
                values: Esmart0-win0=0x8
                value: 8
```

第一部分的 Encoders 输出和第二部分的 Connectors 对应，从 Dump 输出我们可以看到：

Connectore eDP-1 的 id 为 154，它对应的 Encoder id 为 153， 且它处于 `disconnected` 状态， 说明底层驱动未检测到该 eDP 设备的连接，Encoder 153 的 `possible crtcs` 的值为 2 即 BIT(1)， 这是一个按位与的掩码，说明该 eDP 只能和 VP1 连接。

Connector HDMI-A-1 的 id 为 156，它对应的 Encoder id 为 155，且它处于 `connected` 的状态，说明底层驱动已经检测到了该 HDMI 接口上已经有显示设备连接，modes 是驱动上报的对应显示设备支持的分辨率。

CRTC 对应 VOP 2.0 中的 Video Port 或者 VOP 1.0 中的 vop。

Planes 对应图层，列出的信息包含该图层可以在哪几个 VP 之间切换（`possible crtcs`）以及所支持的格式(`formats`)。

2. 在对应的显示接口上输出各种颜色的彩条，可以利用该功能对 Drm 驱动进行简单的验证。

```
modetest -M rockchip -s 156@71:1920x1080
```

通过上述命令，可以在 HDMI-A-1 上显示类似下边的 smpte 彩条，其中 156 是 HDMI-A-1 的 id，71 是 VP0 的 id。

3. 需要临时手动修改 drm 一些属性的值

更多的使用方式，可以参考以下使用说明：

```
[root@RK3588:/]# modetest -h
usage: modetest [-acDdefMPpsCvrw]

 Query options:

        -c      list connectors
        -e      list encoders
        -f      list framebuffers
        -p      list CRTCs and planes (pipes)

 Test options:

        -P <plane_id>@<crtc_id>:<w>x<h>[+<x>+<y>][*<scale>][@<format>]  set a plane
        -s <connector_id>[,<connector_id>][@<crtc_id>]:[#<mode index>]<mode>[-<vrefresh>][@<format>]    set a mode
        -C      test hw cursor
        -v      test vsynced page flipping
        -r      set the preferred mode for all connectors
        -w <obj_id>:<prop_name>:<value> set property
        -a      use atomic API
        -F pattern1,pattern2    specify fill patterns

 Generic options:

        -d      drop master after mode set
        -M module       use the given driver
        -D device       use the given device

        Default is to dump all info.
```

这里需要说明的是，modetest 是通过 id 来指定 plane，crtc，connector 等 Drm 资源的，这些 id 可以通过 modetest 查询，也可以通过如下命令来查询：

```
cat /sys/kernel/debug/dri/0/state
```

9.10 xrandr 的使用

xrandr 是使用 Xserver 的图形系统上的一个显示配置工具，它可以用来 dump 系统的显示信息，也可以用来配置系统的显示输出。

```
root@linaro-alip:/# xrandr
Screen 0: minimum 320 x 200, current 1920 x 2048, maximum 8192 x 8192
eDP-1 connected primary 1536x2048+0+0 (normal left inverted right x axis y axis) 0mm x 0mm
   1536x2048     59.99*+
HDMI-1 connected 1920x1080+0+0 (normal left inverted right x axis y axis) 708mm x 398mm
   1920x1080     60.00*+  50.00    59.94    30.00    24.00    29.97    23.98
   1920x1080i    60.00    50.00    59.94
   1280x1024     60.02
   1440x900      59.90
   1360x768      60.02
   1280x720      60.00    50.00    59.94
   1024x768      60.00
   800x600       60.32
   720x576       50.00
   720x576i      50.00
   720x480       60.00    59.94
   720x480i      60.00    59.94
   640x480       60.00    59.94
DP-1 disconnected (normal left inverted right x axis y axis)
```

该工具是 Xserver 环境下的通用工具，详细的使用介绍可参考 Xrandr

9.11 显示进程的暂停、启动

有时候为了调试方便，希望应用停在某一个固定的场景，我们可以使用下面的命令暂停和恢复显示进程。

- Android Surfaceflinger 进程：

1. 暂停进程

```
kill -STOP `pgrep surfaceflinger`
```

2. 恢复进程

```
kill -CONT `pgrep surfaceflinger`
```

- Linux weston 进程：

1. 暂停进程

```
Kill -STOP `pgrep weston`
```

2. 恢复进程

```
kill -CONT `pgrep weston`
```

- Linux Xserver 进程：

1. 暂停进程

```
Kill -STOP `pgrep Xorg`
```

2. 恢复进程

```
kill -CONT `pgrep Xorg`
```

在日常的开发过程中，当我们希望临时通过 io 命令修改 VOP 的一些寄存器配置来查看某种显示效果的时候，特别注意提前使用类似的命令，停掉 usespace 中对应的显示主进程，否则显示进程的更新会随时覆盖本地的寄存器修改。

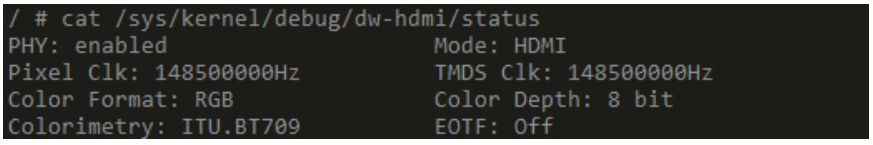
9.12 获取 EDID 信息

以 HDMI 为例：

```
cat /sys/class/drm/card0-HDMI-A-1/edid > /data/edid.bin
```

9.13 查看 HDMI 状态

```
cat /sys/kernel/debug/dw-hdmi/status
```



10. FAQ

10.1 VOP POST_BUF_EMPTY

可能会出现类似以下 log：

```
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
```

可能原因有：

1. 带宽不够

   如果系统带宽不够会导致 VOP 不能及时取到数据，从而报 post buf empty，可以做如下尝试：

   (1) 尝试将 ddr 固定最高频率;

   (2) 将屏的消隐期加长，提高一行的取数时间;

2. iommu 出错

   确认是否如下图所示的 iommu pagefault 错误，如果有，请先尝试更新到最新代码测试，如果最新代码还存在该问题，请提交 remine 系统，并附上相关 log；

```
rockchip-vop ff8f0000.vop: [drm:vop_isr] *ERROR* POST_BUF_EMPTY irq err
rk_iommu ff8f3f00.iommu: Page fault at 0x00000000f0000400 of type read
rk_iommu ff8f3f00.iommu: iova = 0x00000000f0000400: dte_index: 0x3c0 pte_index: 0x0
rk_iommu ff8f3f00.iommu: mmu_dte_addr: 0x00000000f1692000 dte@0x00000000f1692f00: 0x
: 0 page@0x0000000000000000 flags: 0x0

0x00000000: 00000000 03058896 20805800 0003d000
0x00000010: 0000000f 0800780c 00000000 00711c08
0x00000020: ed000000 00000000 00000000 00000000
0x00000030: 3a001085 00400000 00000000 01400147
0x00000040: 00082000 00000000 0101028d 0101028d
```

3. Logic 电压太低

   Logic 电压太低会导致 VOP 异常，可以尝试提高 100mv 测试；

4. AFBDC/IFBDC 对齐要求

   对于 PX30/RK3326、RK3368、RK3399、RK356X、RK3588 平台如果屏的分辨率非 16pixel 对齐可以尝试关闭 afbdc/ifbdc 功能，修改方法参考文档《FAQ-DRM-HWC》 1.3.1 章节。

10.2 显示效果调节

VOP 内部的 BCSH 模块支持 亮度、对比度、饱和度、色度的调节，可以通过 modetest 配置 connector 下的对应属性调节，对于 android 系统，有实现了基于安卓系统的属性配置默认值是 50，N 每次+1。

android 9.0 之前使用命令：

```
setprop persist.sys.brightness.main val
setprop persist.sys.contrast.main val
setprop persist.sys.saturation.main val
setprop persist.sys.hue.main val
setprop sys.display.timeline N+1
```

android 9.0 及之后使用命令：

```
setprop persist.vendor.brightness.main val
setprop persist.vendor.contrast.main val
setprop persist.vendor.saturation.main val
setprop persist.vendor.hue.main val
setprop vendor.display.timeline N+1
```

从 Android 11 开始 SDK 有集成了整套显示效果调节工具和文档，可以参考 Android SDK 中的以下文档说明：

RKDocs/common/display/Rockchip_Introduction_DisplayAdjust_APK_CN.pdf

10.3 屏无法点亮/休眠唤醒显示异常/不显示问题

按以下几个方面做进一步确认：

1. 确认是否有背光；
2. 确认屏的相关电源及复位控制是否正常；
3. 确认上下电时序是否满足屏的 spec 要求；

10.4 RK3308/RV1106/RV1103 如何打开显示功能

由于 RK3308/RV1106/RV1103 轻量化的设计理念，支持的内存较小，所以默认不支持显示功能。如果需要开显示，需要做以下配置：

1. U-Boot 修改

   RK3308 U-Boot RGB/MCU 接口显示参考配置 rk3308-display.config：

   ```
   make rk3308_defconfig rk3308-display.config && ./make.sh
   ```

   RV1106/RV1103 U-Boot RGB/MCU 接口显示参考配置 rv1106-display.config：

   ```
   make rv1106_defconfig rv1106-display.config && ./make.sh
   ```

2. kernel 修改

   RK3308 VOP 不支持 IOMMU，所以分配内存需要从预留的 cma buffer 分配，默认 CMA_SIZE 为 16M，如果出现分配内存失败，可以参考如下方法修改 CMA_SIZE：

   ```
   @@ -784,7 +786,12 @@ CONFIG_ARCH_SUPPORTS_MEMORY_FAILURE=y
    # CONFIG_MEMORY_FAILURE is not set
    # CONFIG_TRANSPARENT_HUGEPAGE is not set
    # CONFIG_CLEANCACHE is not set
   -# CONFIG_CMA is not set
   +CONFIG_CMA=y
   +# CONFIG_CMA_INACTIVE is not set
   +# CONFIG_CMA_DEBUG is not set
   +# CONFIG_CMA_DEBUGFS is not set
   +# CONFIG_CMA_SYSFS is not set
   +CONFIG_CMA_AREAS=7
    # CONFIG_ZPOOL is not set
    # CONFIG_ZBUD is not set
    # CONFIG_ZSMALLOC is not set
   @@ -4917,6 +4925,18 @@ CONFIG_DMA_NONCOHERENT_MMAP=y
    CONFIG_DMA_COHERENT_POOL=y
    CONFIG_DMA_REMAP=y
    CONFIG_DMA_DIRECT_REMAP=y
   +CONFIG_DMA_CMA=y
   +# CONFIG_DMA_PERNUMA_CMA is not set
   +
   +#
   ```

```
+# Default contiguous memory area size:
+#
+CONFIG_CMA_SIZE_MBYTES=32
+CONFIG_CMA_SIZE_SEL_MBYTES=y
+# CONFIG_CMA_SIZE_SEL_PERCENTAGE is not set
+# CONFIG_CMA_SIZE_SEL_MIN is not set
+# CONFIG_CMA_SIZE_SEL_MAX is not set
+CONFIG_CMA_ALIGNMENT=8
 # CONFIG_DMA_API_DEBUG is not set
 CONFIG_SGL_ALLOC=y
 CONFIG_CPU_RMAP=y
```

RK3308 kernel RGB 转 MIPI 显示（bridge 芯片 RK618）参考配置 rk3308bs_mipi_display.config：

```
make ARCH=arm64 rk3308_linux_defconfig rk3308bs_mipi_display.config
```

RV1106/RV1103 kernel RGB/MCU 接口显示参考配置 rv1106-evb.config：

```
make ARCH=arm rv1106_defconfig rv1106-evb.config
```

10.5 关闭 iommu 的方法

关闭 vop iommu 后通过 DRM 申请的内存会从 CMA 内存分配，系统默认 CMA 内存大小 16M，需要根据场景需求调整到对应的大小，否则会出现分配内存失败。

```
diff --git a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi b/ar
index 915034fc3d0a..26625ff51219 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
@@ -3423,7 +3423,7 @@ vop: vop@fdd90000 {
                                "dclk_vp1",
                                "dclk_vp2",
                                "dclk_vp3";
-               iommus = <&vop_mmu>;
+               //iommus = <&vop_mmu>;
                power-domains = <&power RK3588_PD_VOP>;
                rockchip,grf = <&sys_grf>;
                rockchip,vop-grf = <&vop_grf>;
```

10.6 各种接口应用参考文档

模块	文档
RGB/MCU	Rockchip_Developer_Guide_RGB_MCU_CN.pdf
BT656/BT1120	Rockchip_Developer_Guide_BT656_TX_AND_BT1120_TX_CN.pdf
LVDS	Rockchip_Developer_Guide_LVDS_CN.pdf
MIPI DSI	Rockchip_Developer_Guide_MIPI_DSI2_CN.pdf
eDP	Rockchip_Developer_Guide_eDP_CN.pdf
DP	Rockchip_Developer_Guide_DisplayPort_CN.pdf
HDMI	Rockchip_Developer_Guide_HDMI_CN.pdf

10.7 RGB/MCU 屏帧率计算问题

已知：

```
htotal = hactive + hback-porch + hfront-porch + hsync-len
vtotal = vactive + vback-porch + vfront-porch + vsync-len
```

N 是每一个像素需要 N 个 cycly 发送完：

bus_format	一个 Pixel 要 N 个 Cycle 发送
MEDIA_BUS_FMT_RGB888_1X24 MEDIA_BUS_FMT_RGB666_1X18 MEDIA_BUS_FMT_RGB565_1X16	1
MEDIA_BUS_FMT_RGB888_3X8	3
MEDIA_BUS_FMT_RGB888_DUMMY_4X8	4

根据以上信息，可以计算帧率：

接口类型	帧率计算
RGB	fps = dclk / (htotal x vtotal x N)
MCU	fps = dclk / (htotal x vtotal x （mcu-pix-total + 1) x N)

10.8 如何编写第三方转换芯片驱动

有些第三方转换芯片需要软件驱动的，这个时候我们需要借助 DRM 框架 bridge 接口向框架注册 connector，比如 RGB2HDMI 的 SII902X 为例，此时 rockchip_rgb.c 充当 encoder 角色，sii902x.c 充当 connector 角色，具体可以参考 SII902X 驱动的实现：

```
drivers/gpu/drm/bridge/sii902x.c
```

10.9 RK3588 DSC 支持几个 slice，slice width 最大支持多少

RK3588 有 2 个 DSC，HDMI0 和 DSI0 使用 DSC0(DSC_8K)，可以支持的 slice 数量是： 1、2、4、8 ， 对应的 slice width 是 xres / 1、xres / 2、xres / 4、xres / 8， DSC_8K 最大可以支持的宽是 7680，所以最大的 slice_width 也是 7680； HDMI1 和 DSI1 使用 DSC1(DSC_4K)，可以支持的 slice 数量是：1、2，对应的 slice width 是 xres / 1、xres / 2，DSC_4K 最大可以支持的宽是 4096，所以最大的 slice_width 也是 4096。

10.10 超过 4kP60 对 aclk 的要求

对于类似 RK3588 这种支持的分辨率超过 4KP60 的【如 4KP120, 8KP30, 8KP60 等分辨率】，默认的 500M aclk 无法满足这些分辨率的需求，所以需要在 DTS 中设置 aclk 频率为 800M，否则会出现 VOP 性能不够导致的显示横条纹问题，修改方法如下：

```
&vop {
    assigned-clocks = <&cru ACLK_VOP>;
    assigned-clock-rates = <800000000>;
};
```

10.11 如何默认开启显示 mirror 功能

有些项目，由于模具和屏幕方向不一致，可能希望软件上默认就能对显示内容做镜像。

- 默认 Y 方向镜像显示是不支持的。
- 默认 X 方向镜像显示仅 VOP V2.0 及以上版本支持（详见《各平台 VOP 基础特性》章节），配置方法示例如下：

```
&vp1 {
    xmirror-enable;
};
```

10.12 RV1106/RV1103 小分辨率屏无法点亮/显示异常问题

对于 RV1106 和 RV1103 平台，dclk_vop 的时钟源可以为 GPLL 或 CPLL，两者的默认频率分别是 1188000 KHz 和 1000000 KHz，而 CRU 模块在设计上 dclk_vop 支持的最大分频系数为 32：

CRU_CLKSEL_CON23

Address: Operational Base + offset (0x035C)

Bit	Attr	Reset Value	Description
31:16	RW	0x0000	write_enable Write enable for lower 16 bits, each bit is individual. 1'b0: Write access disable 1'b1: Write access enable
15:14	RO	0x0	reserved
13:9	RW	0x00	clk_ref_pvtpll_0_div Divide clk_ref_pvtpll_0 by (div_con + 1).
8	RW	0x0	dclk_vop_src_sel dclk_vop_src clock mux. 1'b0: clk_gpll_mux 1'b1: clk_cpll_mux
7:3	RW	0x07	dclk_vop_src_div Divide dclk_vop_src by (div_con + 1).
2:0	RO	0x0	reserved

因此，dclk_vop 在两个 PLL 下的最小频率默认为 37125 KHz 和 31250 KHz，若屏端支持分辨率的 pixel clock 小于上述频率，可能会导致无法点亮或显示异常问题。若需支持较小分辨率的显示设备（比如 pixel clock 为 27000 KHz 的分辨率 720x480p60/720x576p60），只能通过修改时钟源 GPLL 或 CPLL 的频率以降低 dclk_vop 支持的的最小频率：

```
&vop {
    assigned-clocks = <&cru PLL_GPLL>;
    assigned-clock-rates = <594000000>;
};
```

需要注意的是：

- RGB/MCU 屏通常可以支持一定区间内的 clock 频率（对应 datasheet 中说明的 MIN/TYP/MAX clock rate），如果默认 GPLL/CPLL 下 dclk_vop 的最小频率可以满足 MAX clock rate 的要求，**则尽量不要去修改时钟源 GPLL/CPLL 的频率**。
- GPLL 和 CPLL 通常作为多个模块的时钟源，需要根据具体的应用场景，**确认时钟源频率的修改是否会对 VOP 外的其他模块造成影响**。

11. 如何更高效的定位问题

在日常开发中，如果发现了 Bug 或者 issue，科学的分析和沟通有助于问题的更快解决和收敛。对于显示相关的问题，我们建议发现问题后，开发者能做如下确认。

1. 看内核 log，或者应用层 log，是否能看到明显的异常提示
2. 如果能看到内核异常的 log 提示，根据 log 去搜索内核驱动，找到相关代码，看是根据什么判断逻辑输出这些异常 log 的。
3. 确认问题是概率的，还是必现的。
4. 如果是 HDMI、DP 这些支持多分辨率的设备，确认异常是在某个特定分辨率才会出现，还是所有分辨率都会出现，确认问题是特定显示器才会出现，还是所有显示器都会出现。
5. 如果是 HDMI，DP 兼容性相关的问题，请在控制变量的前提下找第三方设备做对比，看第三方设备在同样条件下是什么表现。
6. 如果是播放视频场景下出现的异常，确认问题是特定视频，APP 才会出现，还是任何视频，APP 都会出现。
7. 如果找不到任何方向，请确认这个异常是一直都存在还是更新特定代码或者版本后才存在，如果是后者，用二分法、控制变量法，找到更新哪个版本或者模块后，异常才出现。

12. 参考文档

1. Rockchip_drm_integration_helper-zh.pdf
2. Rockchip_DRM_Panel_Porting_Guide.pdf
3. Rockchip_Developer_Guide_HDMI_CN.pdf
4. RK3588_MIPI_DSI2_Developer_Guide_CN.pdf
5. Rockchip_BT656_TX_AND_BT1120_TX_Developer_Guide_CN.pdf
6. Rockchip RK3568 Datasheet.pdf
7. Rockchip rk fb development guide.pdf
8. Wikipedia for Direct_Rendering_Manager
9. Linux DRM Developer's Guide